

Knowledge-Driven Control Software Development Using a Capabilities Ontology

Thesis submitted in partial fulfillment
of the requirements for the degree of

Doctor of Philosophy
in
Computer Science and Engineering

by

Amar Banerjee
2018801016

`amar.banerjee@research.iiit.ac.in`

Advisor: Dr. Venkatesh Choppella

January, 2026



INTERNATIONAL INSTITUTE OF
INFORMATION TECHNOLOGY
HYDERABAD

International Institute of Information Technology Hyderabad
500 032, India

Copyright © Amar Banerjee, 2026
All Rights Reserved

International Institute of Information Technology Hyderabad
Hyderabad, India

CERTIFICATE

This is to certify that work presented in this thesis proposal titled *Knowledge-Driven Control Software Development Using a Capabilities Ontology* by *Amar Banerjee* has been carried out under my supervision and is not submitted elsewhere for a degree.

Date

Advisor: Dr. Venkatesh Choppella

Acknowledgments

First and foremost, I express my deepest gratitude to my teacher and advisor, Prof. Venkatesh Choppella, whose teaching, guidance, and mentorship have shaped my academic journey. His unwavering support has not only helped me grow as a researcher but has also contributed immensely to my professional and personal development. I would also like to sincerely thank Prof. Raghu Reddy, Prof. T. V. Prabhakar, and Prof. Vikaram Pudi, whose valuable comments and constructive feedback helped me refine and align many ideas that contributed to the final shape of this thesis.

I am deeply grateful to IIIT Hyderabad for providing me with the opportunity to pursue this research. Balancing professional responsibilities while undertaking a PhD is always challenging, but IIIT Hyderabad and its vibrant academic ecosystem have been highly supportive throughout this journey. I extend my thanks to the Dean, the academic administration, and all those who facilitated my progress at every stage.

I am sincerely thankful to Dr. Swaminathan Natarajan and Mr. Subhrojyoti Roy Chaudhuri from TCS Research, who encouraged and inspired me to embark on this challenging and rewarding research journey. Their insights and encouragement were instrumental in shaping my work.

I extend my heartfelt gratitude to my parents, who have stood as my constant source of inspiration and strength since my childhood. Their unconditional support has always been my greatest motivation.

Finally, I owe profound thanks to my wife, whose patience, sacrifices, and unwavering support during every difficult phase allowed me to complete this thesis. Her belief in me made this accomplishment possible.

Abstract

Control software plays a pivotal role in orchestrating the interactions of industrial systems, including sensors, actuators, robots, and machinery, to ensure real-time coordination and reliability. Developing such software, however, is a complex and resource-intensive process, requiring significant time, cost, and specialized domain expertise. For example, the control software development for the Square Kilometre Array (SKA) radio telescope, which manages thousands of devices such as antennas and sensors, required over 100 engineers and spanned over five years. Furthermore, hardware upgrades, maintenance, and evolving operational requirements necessitate frequent reconfiguration of control software in industrial systems. Due to production demands or machine replacements, such reconfigurations may occur multiple times per month.

Manual development and reconfiguration of control software are inherently time-consuming, resource-intensive, and heavily reliant on domain expertise. Automating these processes is essential for scalable, cost-effective, and rapid development. A significant challenge in automation lies in capturing and reusing domain-specific knowledge concerning device behaviors and interactions. Current generative AI (GenAI) technologies lack domain-specific knowledge, explainability, and robust reasoning capabilities to address the precision and constraints in reconfiguration scenarios.

This thesis addresses these challenges by automating control software development and reconfiguration using domain knowledge. The knowledge of control components is systematically captured through a Capability Ontology. Domain-specific languages (DSLs) were developed to leverage this knowledge, providing a syntactic interface to describe abstract requirements, device capabilities, and control designs. A synthesis algorithm automatically generates control designs, translated into executable software via a code-generation engine. These components are integrated into the *Knowledge-aided Integrated Development Environment (K-IDE)*.

K-IDE facilitates the creation of new device capabilities and workflows, enabling the generation of control software from scratch. Additionally, it supports reusing existing knowledge to reconfigure systems and produce updated software. By combining the synthesis algorithm with domain knowledge, K-IDE automates the development and reconfiguration of control software.

To bridge domain knowledge with executable artifacts, the thesis also introduces a domain-specific *Knowledge Representation Language (KRL)* that translates semantic models into multiple target representations such as code and visualization formats.

K-IDE was validated across three industrial setups: (i) a warehouse robotics system with over 200 components, (ii) an automated vehicle entry system comprising 150 devices, and (iii) a smart meeting room with 50 interconnected devices. Validation results demonstrated significant efficiency improvements, reducing development time by 60% and reconfiguration effort by 50% compared to manual development approaches. In addition to technical validation, the usability of K-IDE was evaluated through user interviews, questionnaires, and practical deployment, indicating substantial gains in efficiency, flexibility, error reduction, and ease of adoption across diverse engineering roles.

Contents

Chapter	Page
List of Abbreviations	xvi
1 Introduction: Control Systems in Modern Industrial Plants	1
1.1 Key Characteristics of Control Software	1
1.2 Differences Between Control Software and General-Purpose Software	2
1.2.1 Working Definitions and Scope	3
1.3 Challenges in Control Software Development	4
1.3.1 Managing Scale and Complexity	4
1.3.2 Incorporating Domain-Specific Knowledge	4
1.3.3 Ensuring Safety and Reliability	4
1.3.4 Long Development Cycles	5
1.4 Reconfiguration of Control Software	5
1.5 Problem Statement	6
1.6 Contributions and Scope of the Thesis	7
1.6.1 IDE Platform Architecture and Knowledge-driven Control Synthesis	7
1.6.2 Validation and Results	8
1.6.3 Contributions	9
1.6.4 Scope and Limitations	10
1.6.5 Assumptions	10
2 Systematic Mapping Study	12
2.1 Survey Procedure	12
2.2 Research Questions and Scope	13
2.2.1 Inclusion and Exclusion Criteria and Quality Assessment	13
2.2.2 Data Compilation and Analysis	14
2.3 Results	14
2.3.1 Data Extraction and Analysis	14
2.3.2 Insights and Trends	14
2.3.3 Engineering Phases and Paradigms	15
2.3.4 Application Domains and Concerns	15
2.3.5 Control Software Engineering Map	15
2.4 Open Opportunities	15
2.5 Conclusion	17

3	Related Work	18
3.1	Literature Study	18
3.1.1	Literature Survey Details	18
3.1.2	Qualitative Inclusion Criteria for Paper Selection	19
3.2	Literature Review	19
3.2.1	Knowledge-Driven Control Strategy Synthesis for Robotic Action Recipes	19
3.2.2	Automatic Composition of Modular Control Software	20
3.2.3	Compositional Design Creation using Knowledge of Mechatronic Modularity	20
3.2.4	Formal Design Synthesis Using Assume-Guarantee Contracts Knowledge	21
3.3	Knowledge-Driven Service Orchestration for Cyber-Physical Systems	22
3.3.1	Knowledge-Driven Self-Reconfiguration Design for Machine Control Systems	22
3.3.2	Knowledge Representation and Reasoning for Robotic Control	23
3.3.3	Knowledge-Based Platform for Decision Support in Engineering Design	23
3.3.4	Knowledge-Based Automated Artefact Generation and Consistency in System Design	24
3.3.5	Knowledge-Driven Framework for Digital Twin Manufacturing Cell	24
3.3.6	Semantic Extension of Nodejs library for Rapid IoT Application Development	25
3.4	Summary	26
3.5	Open Opportunities	26
3.6	Conclusion	27
4	Controller Design Tool Survey	28
4.1	Knowledge-Driven Control Software Development	28
4.1.1	Controller Synthesis Tool Support	28
4.1.2	Usage of Controller Synthesis Tools	30
4.1.3	Industrial Needs and Synthesis Tools	30
4.1.4	Industry 4.0 Scenario	31
4.1.5	Gaps in Existing Controller Synthesis Tools	32
4.2	Opportunities	34
4.3	Conclusion	34
5	Knowledge-Driven Approach for Control Software Development	35
5.1	Related Works	35
5.2	Knowledge-Driven Approach for Controller Synthesis	36
5.3	Capability Ontology for Control Components	38
5.3.1	System Interface	38
5.3.2	System Behavior	38
5.3.3	Interaction as Behavior Execution	38
5.3.4	Capabilities as Abstract Labels for Behavior	39
5.3.5	Relationships in the Capability Ontology	40
5.3.6	Example: Robotic Pick-and-Place Arm	41
5.3.7	Device - Capability Ontology	43
5.3.8	Classes	43

5.3.9	Properties	43
5.3.10	Rules	44
5.3.11	Responsibility-Oriented Capability Ontology	45
5.3.12	Comparison with Other Ontologies	45
5.3.13	Persistence in a Graph Database	46
5.4	Composition Architecture	47
5.5	K-IDE Platform for Knowledge-Driven Control Design and Dynamic Reconfiguration	48
5.5.1	Platform Overview	48
5.5.2	Automated Control Design	48
5.5.3	Semantic Code Generation Approach	49
5.6	Threats to Validity	51
5.6.1	Internal Validity	51
5.6.2	External Validity	52
5.7	Conclusion	52
6	Knowledge-aided Integrated Development Environment for Control Software	54
6.1	Knowledge enabled -Integrated Development Environment (K-IDE)	55
6.2	K-IDE: Architectural View	56
6.3	Domain-Specific Languages: The Core Vocabulary of K-IDE	57
6.3.1	M&CML as a Domain-Specific Language	58
6.3.2	Activity DSL: Linking Steps in Use Case Scenarios	59
6.3.3	Capability DSL: Cataloging Component Capabilities	59
6.3.4	Conclusion	60
6.4	Key Features	60
6.4.1	Automated Control Design	60
6.4.2	Semantic Code Generation	61
6.4.3	Design Validation	61
6.4.4	Design Visualization	61
6.4.5	Integration of Knowledge-Driven and Model-Driven Approaches	62
6.5	Learnings from K-IDE's Development	62
6.6	Challenges and Opportunities: Insights from K-IDE's Development	64
6.7	Conclusion and Future Prospects	65
7	Knowledge Representation Language	66
7.1	Importance of Knowledge Representation	66
7.1.1	Challenges and Needs	67
7.1.2	Connecting with Knowledge-Driven Composition and K-IDE	67
7.2	Related Work	67
7.2.1	Rule-Based Systems	68
7.2.2	Semantic Networks	68
7.2.3	Ontologies	68
7.2.4	Semantic Knowledge Graphs	68
7.2.5	Model-Driven Code Generators	68
7.2.6	Large Language Models (LLMs)	69
7.2.7	Comparison and Gaps	69

7.3	Our Approach	70
7.3.1	Architecture for Knowledge Representation and Querying	70
7.3.2	Design of the Template-Based Language	72
7.3.3	Integration with Semantic Knowledge Graphs	73
7.3.4	Knowledge-Driven Paradigm	74
7.4	KRL Grammar	75
7.4.1	Lexer and Parser	77
7.4.1.1	Lexer	77
7.4.1.2	Parser	78
7.4.2	Abstract Syntax Tree (AST)	78
7.4.3	Grammar Rules	78
7.4.3.1	Template Definition	79
7.4.3.2	Rich String Handling	79
7.4.3.3	Expressions and Operations	80
7.4.3.4	Loops and Conditional Statements	80
7.4.4	Language Features	80
7.5	Comparison with Related Approaches	81
7.5.1	Model-Driven Engineering (MDE)	82
7.5.2	Ontology-Based Data Access (OBDA)	82
7.5.3	Code Generation Tools	82
7.6	Examples: Representing Robotic Component Knowledge as an Executable Robot Control Script	83
7.6.1	Potential Applications	84
7.7	Learnings and Limitations	84
7.7.1	Learnings	85
7.7.2	Limitations	85
7.8	Conclusion	86
8	Use cases	87
8.1	Automated Code Generation for Boom Barrier Vehicle Entry System	87
8.1.1	Implementing a Boom-Barrier Vehicle Entry System	87
8.1.2	Composite Machine Configuration – Using Capability Composition	89
8.1.3	Implementation	91
8.2	Auto Generation of Robotic Control Software	91
8.3	Dynamic Reconfiguration of a Smart Meeting Room	93
8.3.1	Dynamic Reconfiguration Process	94
8.4	Comparison with Other Approaches	96
8.5	Approach Evaluation	100
8.5.1	User Evaluation Experiment Setup	101
8.5.2	User Interviews	102
8.5.3	Questionnaires	103
8.5.4	Quantitative Evaluation and Comparative Analysis	103
8.5.5	Performance Acceptance Criteria	104
8.6	Conclusion	106

<i>CONTENTS</i>	xi
9 Conclusion	108
9.1 Limitations	109
Appendices	113

List of Figures

Figure	Page
1.1 Hierarchical structure of control software within industrial plants	2
1.2 Examples of control software applications	3
1.3 Knowledge-aided Integrated Development Environment	8
2.1 Mapping Survey Procedure - A high-level view	13
2.2 Control Software Engineering Map	16
4.1 Industrial Controller Development	31
5.1 Controller Synthesis for Control Software Composition	37
5.2 Ontology illustrating abstraction from Interface through Behavior, Interactions, Capability, to Workflow.	42
5.3 Capability Ontology	43
5.4 Persistence of Device Knowledge in a Graph Database	46
5.5 SACE Architecture	47
5.6 K-IDE for Control Software Development	49
5.7 Semantic Code Generation	50
5.8 Mapping DSL for Robotic Control Software Generation using ROS	50
6.1 The Knowledge-Driven Approach (KDA) Stack	55
6.2 Solution Architecture Using Semantic Self-Description Knowledge	57
6.3 Representing a Controller with M&CML DSL	58
6.5 Automated Control System Design	60
6.6 Template for Semantic Code Generation	61
6.7 Domain-Specific Validation in K-IDE	62
6.8 Visual Representation of Control Design	63
7.1 Architecture of the KRL, illustrating the constructs for defining templates and their integration with semantic elements in the SKG.	72
7.2 Query Construct of KRL	73
7.3 KRL - Language Constructs to Represent an ML Pipeline in Octave[154]	74
7.4 The knowledge-driven paradigm illustrating the generation of multiple target representations and the reflection of changes in the knowledge base.	76
7.5 AST of the language	79
7.6 KRL Example: Representing Robotic Knowledge as a Robot Control Code	83

8.1	Describing the Boom Barrier Capabilities with Capability DSL	90
8.2	Describing the Business Workflow for Smart Vehicle Entry Automation Scenario	90
8.4	Simulation SDF Model	92
8.5	Execution of prototype in simulated environment	93
8.6	Composition of Controller for Smart Meeting Room Workflow	95
8.7	Generation of Executable Control Software using Zetta[165] framework	95
8.8	Execution of Smart Meeting Control Software: Using Simulation	96
8.9	User Evaluation Results: Box and Swarm Plots of Interview and Questionnaire Feedback	104
8.10	Performance and Scalability Across Use Cases	105
8.11	Latency, Memory, and Complexity Comparison	106
1	Classification Taxonomy	125
2	Trends in Control Software Engineering	126
3	Publications supporting a specific engineering phase by using a specific engineering paradigm	127
4	Publications supporting a specific engineering phase for a specific target application domain	128
5	Trends in Control Software Engineering	129
6	Research Objectives Distribution	130
7	Tool vs Concept for Engineering Phases	131
8	Identified concerns for control software	132
9	Concerns and relationships with target application domains and engineering paradigms	133

List of Tables

Table	Page
2.1 Sources Distribution	14
3.1 Comparison of Literature Approaches	26
4.1 Comparison of Open Source Control Development Tools	29
4.2 Controller Development Requirements vs Features Offered by State-of-the-Art Tools	33
5.1 Comparison of Capabilities, Services, Functionalities, and Features	37
5.2 Interpretation of the capability 5-tuple and how each element is operationalized in the ontology.	40
5.3 Classes in the Capability Ontology	44
5.4 Object Properties in the Capability Ontology	44
5.5 Comparison of Ontologies for Designing Control Software	46
8.1 Overview of Experimental Setup Across Use Cases	88
8.2 Software Development Parameters	91
8.3 IoT Devices in the Smart Meeting Room	94
8.4 Commands for the IoT Devices	94
8.5 Commands Sent During the Late-Night Brainstorming Session	94
8.6 Comparison with other approaches across various metrics	99
8.7 Observed and Estimated Metrics Across Real-World Use Cases	101
8.8 Average ratings from user interviews (on a scale of 1-5)	103
8.9 Average ratings from the questionnaire	103
1 Summary of research questions in literature survey	115
2 Search strings used for literature search	116
3 Engineering methods used in the approaches from articles in the Literature Survey	116
4 Engineering phases addressed by the approaches in the articles from literature survey	116
5 Engineering paradigms used by the approaches in articles considered in the literature survey	117
6 Research objectives of the articles in the literature survey	117
7 Target domains in articles from the literature survey	117
8 Concerns addressed in articles from the literature survey	118

9	Search results from the literature study	118
10	Selection of articles for literature review	119
11	Paper list for literature review	120
12	Questions related to respondent characteristics	121
13	General questions for demographic analysis of survey participants	121
14	Likert scale-based questions to understand the role of knowledge and time consumed	121
15	Likert scale-based questions to understand challenges in control design and software development	121
16	Likert scale-based questions related to control design, code translation, and testing	121
17	Age distribution of respondents	122
18	Gender distribution of respondents	122
19	Country distribution of respondents	122
20	Years of software engineering experience of survey respondents	122
21	Role of the respondents in control software development	122
22	Frequency, Percent, and Valid Percent Evaluation on time evaluation measurement for development using Testing component	123
23	Frequency, Percent, and Valid Percent Evaluation on the Limitation on Testing new Developed Software	123
24	Cumulative Percentage Evaluation on most consuming tasks that might result in more time for software development, enhancing the overall cost	124
25	Tooling and Environment Details for Reproducibility	152

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
AST	Abstract Syntax Tree
BDD	Binary Decision Diagram
CA	Context Awareness
CNC	Computer Numerical Control
CNL	Constrained Natural Language
CPS	Cyber-Physical System
DA	Dynamic Adaptation
DSL	Domain-Specific Language
DSML	Domain-Specific Modeling Language
EMF	Eclipse Modeling Framework
FH	Fault Handling
FD	Fault Detection
GR(1)	General Reactivity of Rank 1
HCI	Human-Computer Interaction
IDE	Integrated Development Environment
IMC	Intelligent Mechatronic Component
IoT	Internet of Things
K-IDE	Knowledge-aided Integrated Development Environment
KDA	Knowledge-Driven Approach
KDTMC	Knowledge-Driven Twin Manufacturing Cell
KRL	Knowledge Representation Language
LTL	Linear Temporal Logic
LLM	Large Language Model
MaRCO	Manufacturing Resource Capability Ontology
MCAS	Maneuvering Characteristics Augmentation System
M&CML	Monitoring and Control Modeling Language
MIC	Model-Integrated Computation

MDE	Model-Driven Engineering
OBDA	Ontology-Based Data Access
OWL	Web Ontology Language
PAC	Programmable Automation Controller
PF	Programming Framework
PLC	Programmable Logic Controller
RDD	Responsibility-Driven Design
ROS	Robot Operating System
RTOS	Real-Time Operating System
SACE	Sensor Actuator Control Element
SBC	Single-Board Computer
SKA	Square Kilometre Array
SKG	Semantic Knowledge Graph
SPARQL	SPARQL Protocol and RDF Query Language
SSN/SOSA	Semantic Sensor Network / Sensor, Observation, Sample, and Actuator
SWRL	Semantic Web Rule Language
TANGO/POGO	Tools for control software development (TANGO framework)
UML	Unified Modeling Language
XText	Eclipse-based language engineering framework

Chapter 1

Introduction: Control Systems in Modern Industrial Plants

Control software is the backbone of modern industrial systems, orchestrating real-time interactions between sensors, actuators, machines, and robots. It ensures the smooth, efficient, and safe operation of critical processes across industries such as manufacturing, healthcare, aerospace, and energy [1].

Control software has become increasingly critical with the rise of Industry 4.0[2]. The integration of cyber-physical systems (CPS), robotics, and the Internet of Things (IoT) has elevated its role, enabling automation, predictive maintenance, and operational optimization in modern factories, healthcare, and aerospace [3]. For example, control software manages hundreds of machines in real-time, ensures precision in robotic surgeries, and governs safety-critical systems such as autopilots and navigation in aircraft.

A tragic example that highlights the criticality of control software is the Ethiopian Airlines Flight 302 crash in 2019, where a malfunction in the aircraft's control system contributed to the loss of 157 lives [4]. This incident underscores how failures in control systems can have catastrophic consequences. Reliable and validated control software is thus crucial, especially in safety-critical sectors.

Figure 1.1 illustrates the hierarchical role of control software in industrial settings, showing its coordination across various operational layers. The supervisory layer above orchestrates every operation layer and performs the task of interacting with the subsystem below.

1.1 Key Characteristics of Control Software

Control software has distinct characteristics that set it apart from general-purpose software:

- **Real-Time Operations:** Control software operates under strict timing constraints, often in milliseconds, ensuring machine synchronization and smooth operation [1].
- **Safety and Reliability:** Failures can lead to severe consequences, including equipment damage and loss of life. Rigorous testing is essential to ensure reliability [5].

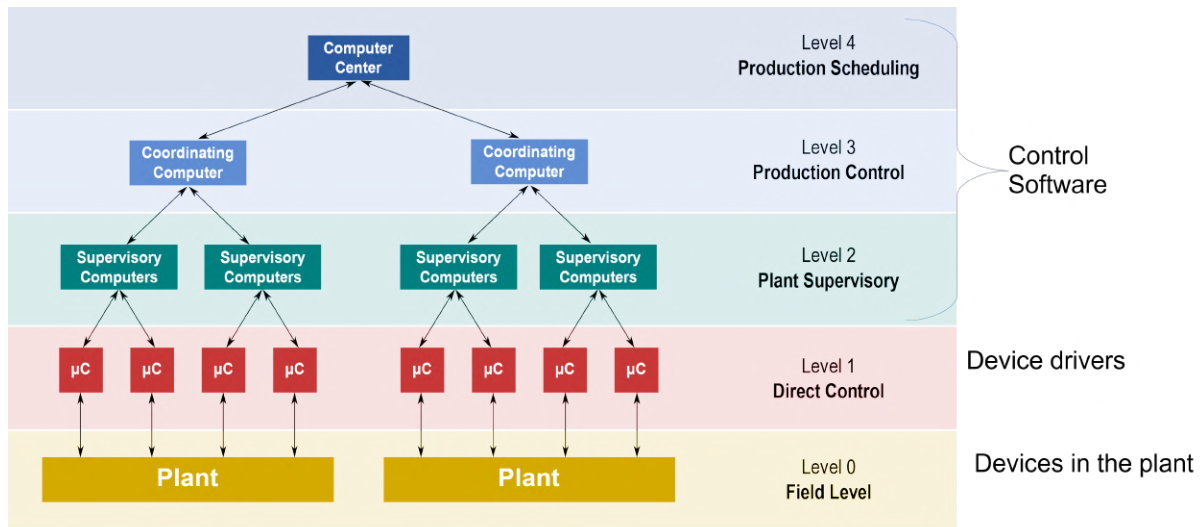


Figure 1.1: Hierarchical structure of control software within industrial plants

- **Domain-Specific Expertise:** Control software is highly tailored to its operational domain, incorporating specialized knowledge of physical processes and industry-specific safety protocols [6].
- **Continuous Operation:** Control software must run continuously without downtime, as disruptions can halt production or critical operations [6].
- **Interaction with Physical Systems:** It interfaces directly with physical devices, making software errors immediately impactful [7].

Figure 1.2 shows examples of control software applications in robotics and IoT-based systems, emphasizing their cross-industry significance.

1.2 Differences Between Control Software and General-Purpose Software

Control software differs significantly from general-purpose software in the following ways:

- **Physical Interaction:** Control software manages real-world devices with specific timing and reliability requirements [8].
- **Specialized Tools:** Developing control software requires specialized tools like real-time operating systems (RTOS) and hardware-in-the-loop testing, which are not typical in general-purpose software development [6].

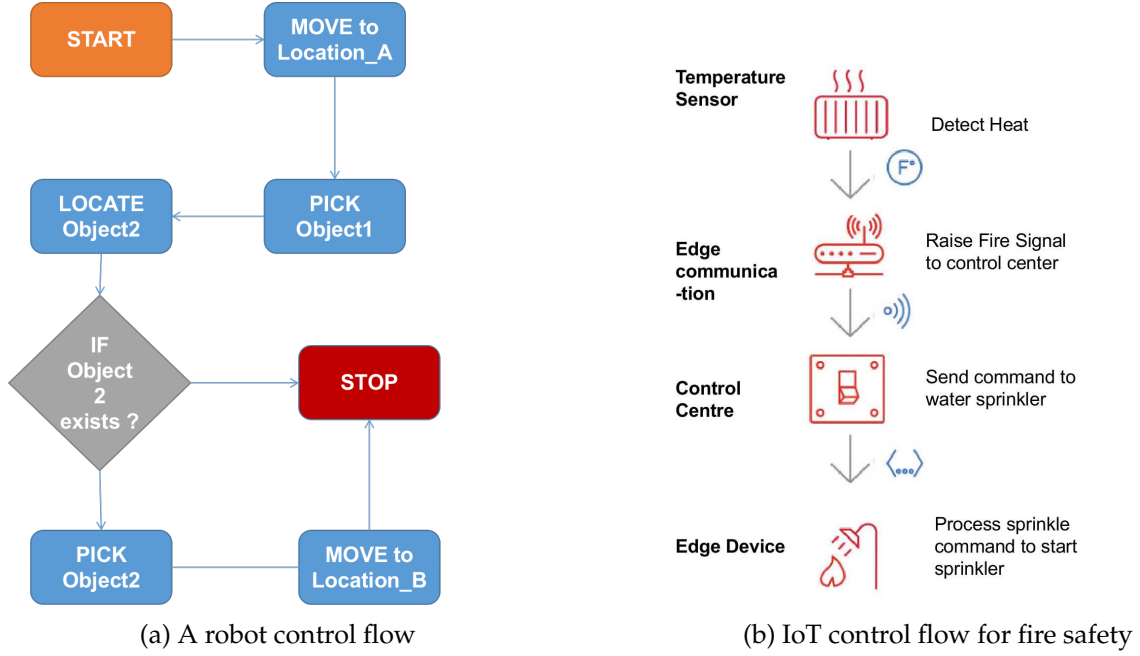


Figure 1.2: Examples of control software applications

- **Domain Knowledge:** Control software development demands in-depth domain-specific knowledge of physical systems and safety protocols [6].

1.2.1 Working Definitions and Scope

To avoid ambiguity and ensure clarity, this thesis adopts the following working definitions.

Control software refers to software that governs the execution, sequencing, coordination, and supervision of physical or cyber-physical components under timing, safety, and operational constraints.

Control is said to be achieved when system-level goals, expressed as ordered workflows or state-transition requirements, are satisfied through coordinated execution of device behaviors, rather than through low-level signal control. The focus of this thesis is therefore on goal satisfaction and behavioral correctness, not continuous feedback control.

Discrete control denotes control logic expressed as transitions between well-defined system states, activities, or tasks, typically event-driven and symbolic in nature. This thesis explicitly focuses on discrete control, where control logic is synthesized from abstract behavioral specifications and capability constraints. Continuous control and signal-level control are assumed to be handled by underlying device controllers or middleware.

Reconfiguration refers to modifications in control logic necessitated by changes in devices, workflows, or operational constraints.

Dynamic reconfiguration denotes such modifications occurring when the system is operational; without redesigning the system from scratch, by reusing and recomposing existing knowledge artifacts such as capabilities and workflows.

1.3 Challenges in Control Software Development

This section identifies the prevalent challenges in control software development based on available literature.

1.3.1 Managing Scale and Complexity

Large industrial systems, such as SKA, require control software to manage millions of devices across geographically dispersed locations. The SKA telescope, for instance, handles real-time data acquisition, device synchronization, and system failures [9]. Similarly, the ITER project involves real-time control of plasma confinement under extreme conditions, with significant safety implications[10].

For example, developing the Square Kilometre Array (SKA) telescope control software required 4 million lines of code [9]. This complexity scale is comparable to sizeable general-purpose software projects like the Linux kernel, which contains over 4 million lines of code [11]. Similarly, power plant control software may require several years to build and typically spans more than a million lines of code.

1.3.2 Incorporating Domain-Specific Knowledge

Control software must be tailored to specific industries, integrating domain knowledge to ensure safety and effectiveness. In healthcare, for example, control software must meet stringent safety standards for life-critical devices [6]. Automobile systems must account for vehicle dynamics and mechanical properties to ensure safe operation. Similarly, an automated warehouse [12] needs to know the procurement processes, the repository of items, and their properties.

1.3.3 Ensuring Safety and Reliability

Failures in control software can have severe consequences, as seen in the Ethiopian Airlines Flight 302 crash, where a malfunction in the Maneuvering Characteristics Augmentation System (MCAS) contributed to a tragic outcome [4]. Safety and reliability testing are critical to prevent such failures.

1.3.4 Long Development Cycles

The complexity of control software creates lengthy development cycles and resource requirements. For instance, the Square Kilometre Array (SKA) project took over five years of development involving 100 engineers [9]. Similarly, the control software for ITER has been under development for over a decade, reflecting the immense scale and complexity.

1.4 Reconfiguration of Control Software

Control software is frequently reconfigured to accommodate hardware upgrades, process changes, and device failures. Reconfiguration scenarios include:

- **Device Upgrade:** When a device is upgraded, the control software must adapt to the new device's communication protocol. For example, replacing a robotic arm in a manufacturing system with a newer model requires reconfiguring the software to interact with the updated protocol[13].
- **Process Change:** Process changes require the control software to reorder device interactions. In an assembly line, for instance, a process change may require parts to be inspected by a camera before a robotic arm assembles them. The control software is reconfigured to ensure the camera inspection happens first[13].
- **Device Failure:** When a device fails, the control software must bypass the faulty device to maintain system operations. For example, if a conveyor belt fails, the robotic arm may need to retrieve parts directly from a temporary holding area, bypassing the belt to keep the process running[13].

Frequent reconfigurations are a crucial challenge in multiple industries [14]. Below is a summary of the scale, frequency, and effort required for reconfigurations in each of these industries.

The automotive industry frequently undergoes reconfigurations to accommodate new vehicle models, components, and updates to the production process. Industry best practices typically guide automotive reconfigurations, which indicate an average time of 8 to 10 hours for reconfiguration [15]. Each reconfiguration normally requires 6 to 10 hours of engineering effort, which involves updating control software, testing, and validation to ensure seamless integration[16].

Power plants also undergo periodic reconfigurations, primarily to integrate new systems and improve operational efficiency. On average, these reconfigurations occur 5 to 8 times a month and take about 15 to 20 hours per reconfiguration, with substantial time spent on testing and validation [17, 18].

In aerospace, reconfigurations occur approximately every 3 to 4 weeks. New sensors, updates to flight safety protocols, or software upgrades often trigger these changes. The reconfiguration process can take several days to complete, depending on the complexity of the updates [19].

In robotic surgical systems, reconfigurations occur up to 10 times monthly in healthcare. Each reconfiguration requires 4 to 6 hours of testing to ensure compliance with operational and safety standards, which is critical for patient safety [20].

The semiconductor industry faces an exceptionally high frequency of reconfigurations, with up to 30 monthly reconfigurations. These changes are necessary to introduce new machinery or production techniques and require meticulous updates to control software. In some cases, reconfigurations can take several days, depending on the complexity of the changes, adding substantial engineering time and effort to ensure minimal production disruption [21].

The frequent reconfigurations across these industries increase operational costs due to the required time and engineering effort, also introducing the risk of delays and errors in production [14]. The need for efficient automation approaches to streamline reconfiguration processes is becoming critical across all these industries.

1.5 Problem Statement

The development and maintenance of control software in industrial systems present significant challenges. These systems coordinate real-time interactions among physical components, including sensors, actuators, and robots, and must operate under strict timing, safety, and reliability constraints. The complexity arises from the heterogeneity of devices and their frequent reconfiguration due to hardware upgrades, production changes, or device failures.

In practice, reconfiguration occurs frequently. For example, a manufacturing plant may need to reconfigure control software weekly to integrate new machines or optimize processes. Current methods rely heavily on manual engineering effort and domain expertise, leading to processes that are time-consuming, error-prone, and costly. Moreover, domain knowledge, such as device behaviors, interface specifications, and operational constraints, is rarely formalized, limiting reuse and automation.

Existing automation approaches, including generative AI and model-driven tools, fall short in several ways. They either lack formal semantics to ensure correctness or cannot capture and reuse detailed domain knowledge. Consequently, they are not suitable for safety-critical and dynamically evolving control environments.

In summary, these challenges motivate the need for a knowledge-driven solution that systematically captures domain knowledge, abstracts device behaviors, and enables automated synthesis and reconfiguration of control software with minimal manual intervention.

This thesis addresses the following research problem:

How can domain knowledge be systematically captured, represented, and used to automate the design and reconfiguration of control software in industrial systems?

To address this problem, the thesis proposes a knowledge-driven approach that:

- Encodes device and system-level knowledge using a capability ontology.
- Uses domain-specific languages (DSLs) to specify requirements and capabilities abstractly.
- Applies synthesis algorithms to generate and reconfigure control logic.
- Realizes these concepts in a platform (K-IDE) that provides concrete automation support.

The approach is validated across diverse industrial scenarios, demonstrating its feasibility, generality, and benefits in reducing engineering effort and improving software adaptability.

1.6 Contributions and Scope of the Thesis

This thesis presents a knowledge-driven approach for automating the design and reconfiguration of control software through the development of an information system that integrates knowledge representation, reasoning, synthesis, and code generation. The proposed Knowledge-aided Integrated Development Environment (K-IDE) operationalizes this approach by enabling capture, reuse, and transformation of domain knowledge into executable control software.

1.6.1 IDE Platform Architecture and Knowledge-driven Control Synthesis

This thesis presents a knowledge-driven approach to the development of control software through the design and implementation of a Knowledge-aided Integrated Development Environment (K-IDE) that automates the synthesis of control logic (design) from high-level task specifications. The K-IDE platform leverages structured device ontologies and semantic knowledge graphs to enable reusable, composable, and verifiable control components.

Figure 1.3 illustrates the overall architecture and workflow of the K-IDE platform. The system is composed of several high-level modules that reflect different stages of the knowledge-driven control synthesis process. The components and their roles are as follows:

1. Human Input: The pipeline begins with human-specified task requirements, representing system-level goals. These are translated into formal representations using the device ontology and knowledge graph, marking the start of the knowledge acquisition and formalization process.

2. Device Knowledge Graph / Ontology: The device ontology encodes physical or logical devices' capabilities, properties, roles, and expected behaviors. The implementation models

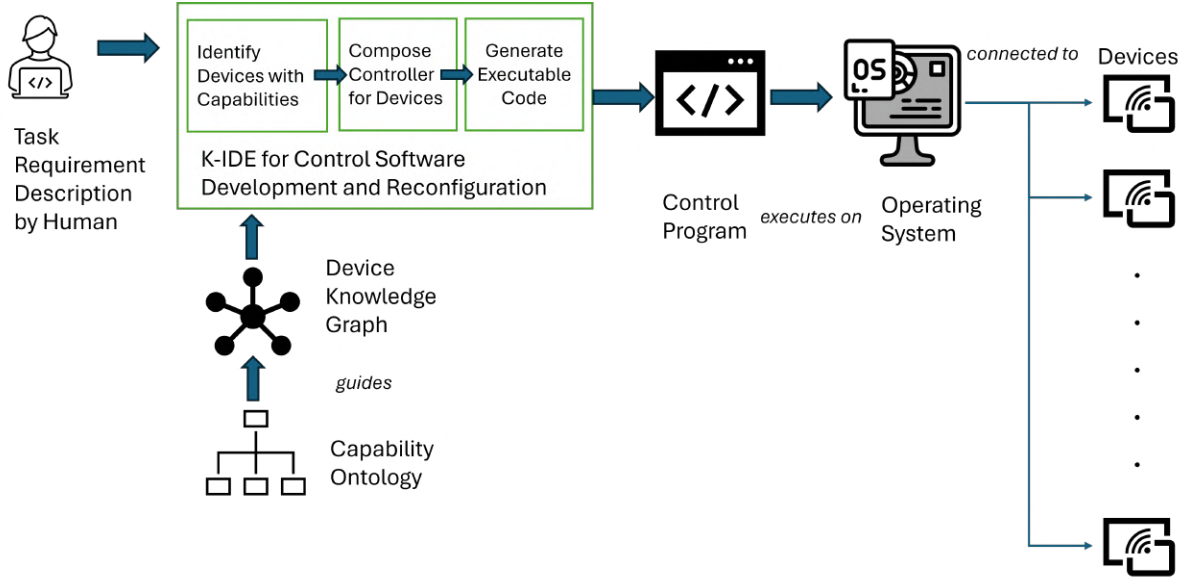


Figure 1.3: Knowledge-aided Integrated Development Environment

80 devices, each annotated with around 20 roles, 20 properties, and one skill per node. This structured representation enables automatic capability matching and reasoning.

3. K-IDE Platform: Implemented in approximately 3000 lines of code, the K-IDE parses task specifications, queries the knowledge base, and synthesizes deployable control logic (design). It matches user goals to device capabilities and generates programs suited for orchestration.

4. Orchestration System (OS): The synthesized programs are dispatched at runtime to devices (e.g., Dev1, Dev2, Dev3) by an orchestration system. This system dynamically manages execution based on availability and system goals, supporting runtime reconfiguration.

5. Dynamic Capability Composition: The K-IDE platform supports dynamic recomposition of control strategies, adapting to device failures, changes in requirements, or runtime constraints. This reflects one of the core contributions of the thesis: enabling adaptive, goal-driven control logic deployment.

1.6.2 Validation and Results

The proposed approach was validated through industrial use cases, vehicle entry automation, robotic warehousing, and smart environments, using quantitative metrics, user feedback, and scalability tests. A controlled study with ten developers was conducted, covering 5–10 control tasks per use case. Manual implementation required up to 100 lines of code per task, while the K-IDE platform generated an average of 1500 lines per developer. The system achieved a 60% reduction in development effort and a 50% reduction in rework.

Ontology-based modeling covered 80 devices, each annotated with 20 roles and 20 properties, and supported 1 capability per node. Automation coverage and artifact reuse reached up to 80%, and programming effort was reduced by up to 86%. The K-IDE platform maintained real-time performance with up to 1000 devices, achieving latency below 130 ms and CPU usage under 10%.

User interviews and surveys indicated improvements in development workflows, error reduction, and system adaptability. In comparison with benchmark approaches like Node-Red [22] and MaRCO [23], the K-IDE platform introduced a structured synthesis process with $O(n \log n)$ complexity, enabling dynamic reconfiguration not supported by the alternatives. The results demonstrate the approach’s applicability, scalability, and alignment with the requirements of control software engineering.

1.6.3 Contributions

1. **Capability Ontology:** We developed a structured ontology to capture and formalize the capabilities of devices in industrial systems. This ontology models device behavior, interaction protocols, and their relationships to workflows, enabling reuse, reasoning, and systematic composition of component functionalities across control design scenarios.
2. **Domain-Specific Languages (DSLs):** The thesis introduces a family of DSLs that represent capability-related knowledge at different abstraction levels, and clarifies their scope as follows:
 - *Domain-agnostic (reusable) DSLs:* Activity and Capability DSLs are provided to encode capability requirements, constraints (including preconditions and postconditions), and composition logic in a reusable manner across multiple control application domains.
 - *Domain-specific DSL:* A domain-specific modeling language is provided for the Monitoring & Control (M&C) domain (M&CML), capturing domain-specific concepts and constraints required for M&C-oriented control software development.

Together, these DSLs ensure consistent and reusable knowledge across control design scenarios, reducing manual effort and improving traceability from requirements to synthesized designs.

3. **Algorithm for Control Design Generation:** A synthesis algorithm automates the generation of control system designs using formalized device capabilities and encoded constraints (including interaction/session assumptions). This enables rapid reconfiguration with minimal manual input while maintaining correctness with respect to capability contracts and workflow requirements.

4. **DSL for Code Generation:** A DSL translates synthesized control designs into executable software code, bridging the gap between design and implementation and simplifying deployment. This language focuses on preserving the semantics of the synthesized design (interfaces, sequencing logic, and constraints) during implementation.
5. **Knowledge-aided Integrated Development Environment (K-IDE):** A platform is implemented to integrate these steps [24]. It combines knowledge technologies such as semantic ontologies and knowledge graphs. Reasoning constructs like SWRL, language frameworks like Xtext and Sirius, and model-to-model and model-to-text transformations implement the synthesis algorithm and code generation. The K-IDE thus operationalizes the full toolchain from capability knowledge capture (ontology + DSLs) to design synthesis and code generation, supporting scalable adoption in industrial settings.

1.6.4 Scope and Limitations

1. **Real-Time and Performance Considerations:** The thesis does not focus on real-time performance optimization. It assumes middleware handles such considerations.
2. **Middleware Dependency:** The proposed method relies on existing middleware for communication and real-time constraints. This thesis focuses on control design and code generation, not middleware development.
3. **Generalization to Other Domains:** We validate the approach in industrial automation scenarios. Extending it to other domains, such as healthcare or aerospace, will require further investigation.

1.6.5 Assumptions

The proposed approach operates under the following assumptions. First, real-time guarantees, communication latencies, and scheduling are handled by existing middleware, operating systems, or device controllers. This thesis does not propose new real-time scheduling mechanisms.

Second, devices expose well-defined interfaces and behavior descriptions that can be captured in the capability ontology. Device-level fault detection and low-level control loops are assumed to exist outside the scope of the proposed framework.

Third, reconfiguration scenarios are assumed to be detectable at the orchestration or system level, enabling the synthesis engine to trigger recomposition using existing knowledge. These assumptions allow the thesis to focus on knowledge-driven control design and software generation rather than infrastructure development.

The structure of this thesis is as follows.

- Chapter 2 explores opportunities in control software development through a systematic mapping study.
 - Chapter 3 reviews the existing work to identify research gaps.
 - Chapter 4 investigates the gaps in the supporting tools for control design.
 - Chapter 5 introduces a knowledge-driven methodology to capture device knowledge using a capability ontology and automatically compose control software from high-level requirements.
 - Chapter 6 presents the Knowledge-aided Integrated Development Environment (K-IDE) that implements the approach discussed in Chapter 5.
 - Chapter 7 introduces a knowledge representation language to convert knowledge into target representations or code.
 - Chapter 8 demonstrates solved use cases using the proposed approach and presents the results.
 - Chapter 9 provides insights and conclusions drawn from the thesis with directions for future work.
-

Chapter 2

Systematic Mapping Study

The foundation laid in Chapter 1 emphasized the importance of control software engineering within technological landscapes, with a specific focus on knowledge-driven approaches. Drawing from this context, this chapter uses a structured survey procedure to explore the domain. The objective is to systematically categorize and analyze the extensive literature, methodologies, and knowledge areas associated with control software engineering, particularly concerning knowledge-driven development.

2.1 Survey Procedure

The Systematic Mapping Study (SMS) implemented in this chapter aims to identify approaches across dimensions such as engineering phases, knowledge-driven strategies, concerns, and application domains. The method adheres to the guidelines proposed by Petersen et al. [25], augmented with an initial step to develop a base taxonomy. This taxonomy captures our preliminary understanding of the field, emphasizing knowledge-driven methodologies, and serves as a cornerstone for subsequent analysis.

A subset of research questions is selected from all possible questions derived from the base taxonomy. The SMS procedure includes article identification, data extraction, analysis, and reporting, focusing on knowledge-driven control software development. Figure 2.1 illustrates the survey procedure.

The initial *base taxonomy* synthesizes insights from prior knowledge of knowledge-driven strategies and control software engineering nuances. This taxonomy lays the groundwork for establishing the knowledge architecture and related aspects, as discussed in Chapter 3. As the survey progresses, this taxonomy will evolve to include a broader range of elements, as shown in Figure 1.

The *base taxonomy* guides the derivation of research questions, illuminating different facets of control software engineering. The comprehensive list of these questions is detailed in Appendix 1.

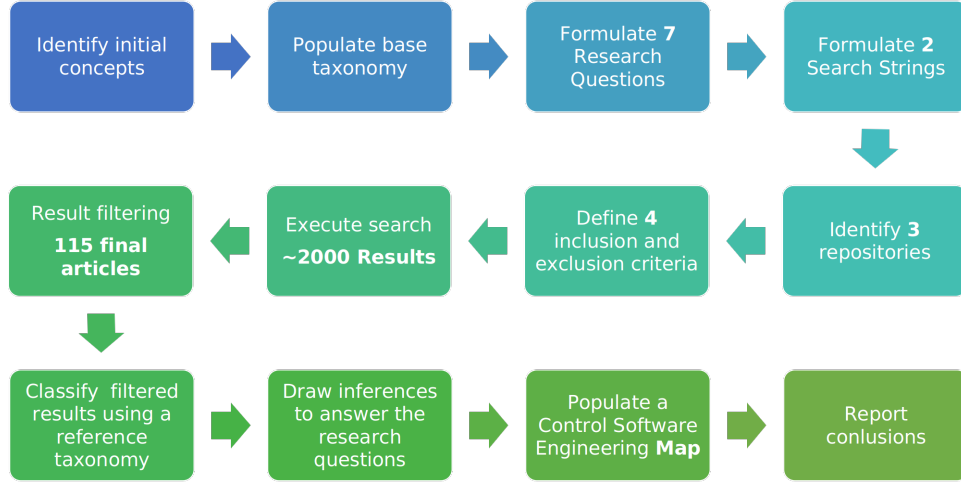


Figure 2.1: Mapping Survey Procedure - A high-level view

2.2 Research Questions and Scope

Exploring multifaceted aspects of control software engineering, including methodologies, paradigms, application domains, and concerns, necessitates the formation of specific research questions. These questions are derived from a base taxonomy that characterizes the primary elements in control software engineering.

The base taxonomy is a set with $n = 6$ elements. By considering all possible subsets of the base taxonomy, the total number of potential research questions, denoted by n_{rq} , is calculated as $n_{rq} = 2^6 = 64$. However, the scope of this study is narrowed to 11 specific questions, chosen to investigate facets of control software engineering, including trends, methodologies, multi-phase solutions, concerns, paradigms, and domains.

The detailed descriptions of the 11 research questions are systematically organized and presented in Appendix 1. The study aims to provide comprehensive insights into control software engineering by addressing these research questions, ensuring a coherent and focused narrative throughout the thesis.

This survey draws from prestigious digital libraries and is complemented by search engines like *Google Scholar* and *Semantic Scholar*. Structured keywords guide the search to ensure precision and comprehensiveness. The detailed keyword-based queries are presented in Table 2.

2.2.1 Inclusion and Exclusion Criteria and Quality Assessment

Explicit inclusion and exclusion criteria are set, focusing on knowledge-driven aspects. Quality assessment criteria ensure that only high-quality papers contribute to our findings. The papers are evaluated based on criteria such as peer review, validation of proposed ap-

proaches, and contributions to control software engineering. In total, 115 papers have been selected as primary studies, listed in the appendix (Tables 2–8).

2.2.2 Data Compilation and Analysis

The collected data is transformed and analyzed using the R Environment, with the `bib2df` R package ensuring seamless data compatibility. This meticulous process underscores the rigor in the literature assessment, paving the way for an in-depth exploration of knowledge-driven methodologies in control software engineering, as discussed in Chapter 3.

2.3 Results

The results of our research provide a comprehensive overview of control software development. Table 2.1 illustrates the distribution of the original 567 articles found, identifying 329 unique articles, 170 as relevant, and 115 as primary studies. The details of the preliminary studies are available in the appendix.

Table 2.1: Sources Distribution

Sources	Found	Unique	Relevant	Primary
Science Direct	216	77	62	39
ACM Digital Library	364	211	81	34
IEEE Digital Library	423	354	76	42

2.3.1 Data Extraction and Analysis

Utilizing the R Environment, the bibliography file is transformed into a data frame [26] and scored (0 or 1) for each article concerning the defined ontology categories. This structured information informs our analysis to address the research questions.

2.3.2 Insights and Trends

Figures 2a and 2b highlight trends and categories within control software engineering. Key insights include the surge in control software engineering articles, a marked research deficit in *Engineering Methods*, and specialization in literature. These findings reveal potential areas for future exploration and validate existing theories in the field [2, 27].

2.3.3 Engineering Phases and Paradigms

The analysis of 115 primary articles reveals preferences for specific engineering phases and methodologies, with an emerging interest in *Agile* and *knowledge-driven* approaches. Figure 3 captures the distribution of engineering paradigms across research articles.

2.3.4 Application Domains and Concerns

Our analysis uncovers various domains engaging with engineering phases, such as cyber-physical systems (CPS), robotics, and automotive systems. Top concerns include *Modularity and Interfacing*, *Real-time Control*, and *Fault Detection*. Figure 9 assists in deducing main trends and areas lacking attention.

2.3.5 Control Software Engineering Map

The systematic mapping approach of this study has identified 115 primary studies, categorizing and analyzing them to determine contributions, solution approaches, engineering methods, and target technology domains. Figure 2.2 presents a control software engineering map, visually representing the intersection of categories and the density of interventions across control software engineering, guiding future research in this expansive field.

2.4 Open Opportunities

Integrating pervasive computation, communication, and sensing in control systems has opened exciting opportunities across diverse fields such as military, commercial, and scientific domains [28]. This advancement sets the stage for addressing uncertainties while developing high-performing, resilient, and adaptable systems. However, these prospects come with challenges and unexplored areas that require further research.

The shift from traditional control mechanisms to advanced decision-making processes introduces new possibilities in areas like autonomous systems, unmanned operations, enterprise scheduling, and resource allocation. While these applications can improve efficiency and safety, they also present complex challenges:

- *Management of Systems with Symbolic and Continuous Dynamics*: Developing robust methods for systems that integrate logical and continuous operations.
- *Stable Control in Networked Environments*: Creating new formalisms to ensure stability and strong performance in distributed systems.

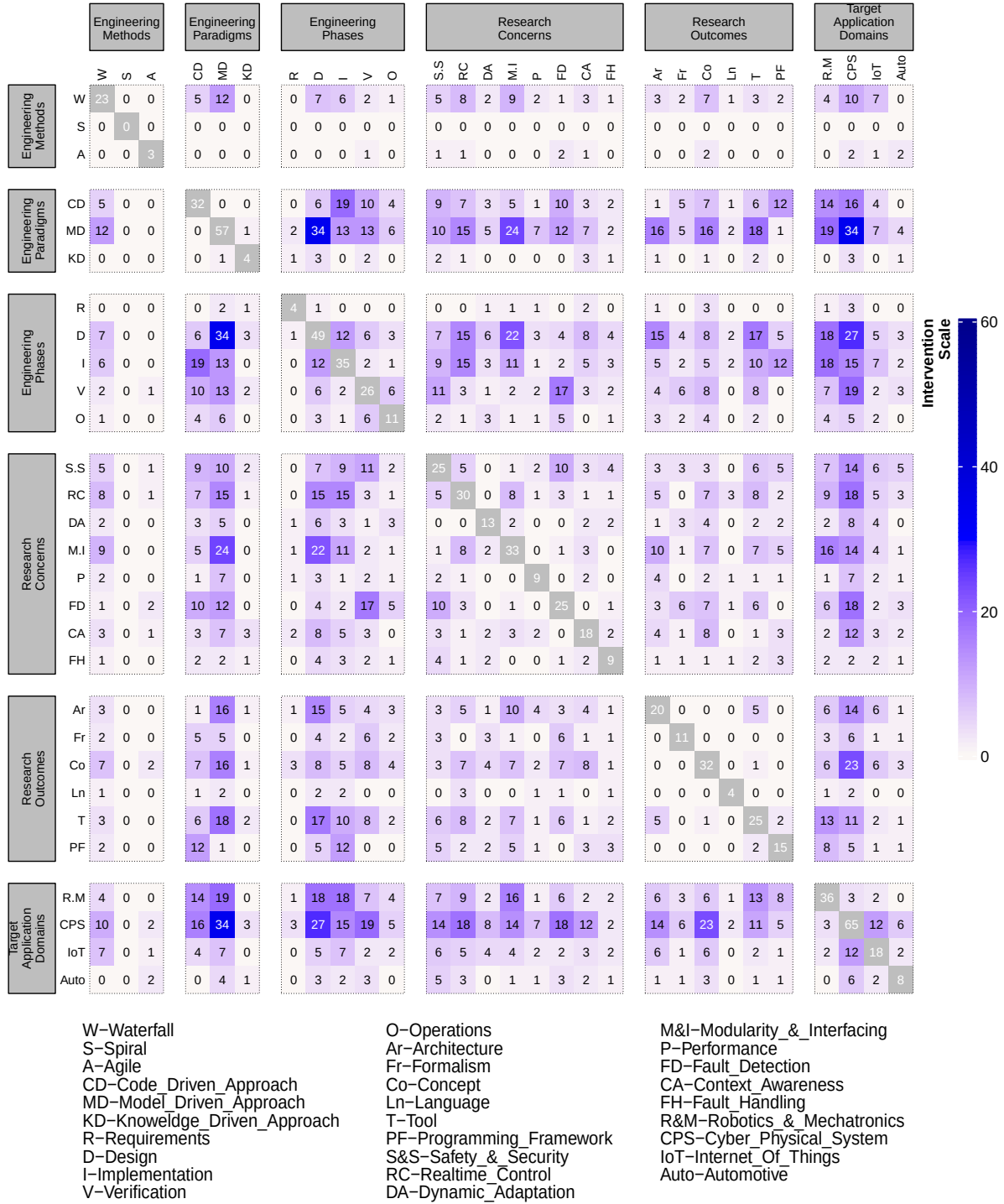


Figure 2.2: Control Software Engineering Map

- *Higher-Level Coordination and Autonomy*: Expanding control mechanisms into decision-making platforms that operate at a higher level requires further research to achieve uniform performance.
- *Integration of Automated Control Design with Validation*: Developing comprehensive tools to manage all stages of control design, from synthesis to verification, more efficiently.
- *Constructing Reliable Systems from Unreliable Components*: Designing adaptive systems that gradually adjust to faults, minimizing sudden failures.

Addressing these challenges requires coordinated efforts based on rigorous research and strategic investment. It involves identifying gaps and leveraging open opportunities, as discussed in Section 2.3. Overcoming these challenges is essential for future research in control systems. By actively engaging with these open opportunities, control software engineering can evolve to harness its full potential in various high-impact domains.

2.5 Conclusion

This chapter synthesizes critical insights from the control software engineering landscape, identifying existing gaps and outlining potential areas for future research. As discussed in Section 2.4, a unified vision of the field has emerged, focusing on both the current state and the future opportunities in control software development.

Control software engineering involves multiple stages, requirements, design, implementation, verification, and operations, each with challenges and opportunities for advancement. Emerging paradigms, particularly knowledge-driven approaches, offer promising directions for innovation [29, 30, 31, 32, 33, 34]. However, complexities such as integrating *fault-detection* and *fault-handling* with *performance* and *real-time control* highlight the need for more cohesive strategies that incorporate domain knowledge and adaptive control mechanisms.

The current landscape also reveals a significant gap: the absence of a comprehensive platform that integrates user-friendly interfaces, robust debugging capabilities, dynamic reconfiguration, and effective collaboration between engineers, domain experts, and developers. This points to the need for an Integrated Development Environment (IDE) to streamline the control system design process, enhance collaboration, and adapt to changing project needs.

In conclusion, this chapter lays the groundwork for the following chapter, where an analysis of 11 selected studies will provide further insights into knowledge-driven control software development. The findings presented here form a foundation for continued exploration and innovation, addressing the critical challenges in control software engineering and advancing the field toward more efficient and adaptable solutions.

Chapter 3

Related Work

This chapter discusses the current research on knowledge-driven methods in control software engineering. It builds on the trends and gaps identified in Chapter 2 and explores how combining knowledge-driven techniques with control software development can improve system intelligence, flexibility, and efficiency. We discuss 11 key approaches that represent important progress in knowledge-driven control software. This chapter highlights the key ideas, methods, challenges, and possible future directions.

3.1 Literature Study

This section delineates a multifaceted and rigorous literature survey procedure tailored to knowledge-driven control software development. The strategy includes a **Multi-Dimensional Literature Search**, comprehensively exploring various sources such as journals, conference papers, patents, etc. **Leveraging Advanced Information Retrieval Techniques** through utilising state-of-the-art tools enhances the search process. **Data-driven Bibliometric Analysis** applies advanced techniques to identify key themes, researchers, and networks, as discussed in Chapter 2. Insights are further enriched through **Expert Interviews and Surveys** from field experts, as will be elaborated in Chapter 4. The survey also includes **Novelty Assessment and Insights**, highlighting innovative and disruptive works, and **Synthesis and Knowledge Framework Construction** for integration into a cohesive framework. This innovative approach aims to uncover untapped potential and inspire knowledge-driven control software development advancements.

3.1.1 Literature Survey Details

The literature survey encompasses the articles' selection and inclusion criteria, ensuring relevance to the field. Targeted searches were conducted in IEEE Xplore, ACM Digital Library, Springer, and Elsevier, resulting in 115 primary articles. Eleven were selected based on specific

criteria, as illustrated in Table 9. The selection criteria included relevance, publication quality, recency, methodological soundness, originality, innovation, diversity, interdisciplinary relevance, clarity, and English comprehension.

3.1.2 Qualitative Inclusion Criteria for Paper Selection

The literature review applied rigorous qualitative criteria for a nuanced selection, including relevance, methodological rigour, novelty, impact, significance, theoretical and practical relevance, the credibility of references, language clarity, and publication venue preference. This approach ensured the selection of high-quality contributions that balance theoretical advancements and practical applicability. Table 10 summarises the final article selection.

3.2 Literature Review

This section overviews recent advancements in knowledge-driven approaches for various aspects of control engineering. It explores multiple methodologies and approaches to enhance the design process and promote consistency. Each subsection offers valuable insights into the challenges and opportunities of designing intelligent systems and developing IoT applications that leverage semantic technologies.

3.2.1 Knowledge-Driven Control Strategy Synthesis for Robotic Action Recipes

The RoboEarth framework, developed by Waibel et al.[35], employs a three-layered architecture that facilitates robots in sharing and utilizing semantic knowledge to execute action recipes.

Control strategies within RoboEarth translate high-level recipes into robotic commands. This intricate process identifies required skills, adheres to order constraints, links perceptions to tasks, and ensures precise execution. Q-Learning is employed for knowledge sharing among robotic units but is limited to individual entities [36].

Despite its innovative design, RoboEarth faces challenges such as:

- Manual knowledge gathering.
- Static environmental assumptions.
- Q-Learning constraints [37].
- Context-specific knowledge neglect.
- Robot-specific ontology focus.

These challenges reveal significant gaps in control strategies and underscore areas for further research and improvement.

3.2.2 Automatic Composition of Modular Control Software

AutoMoDe (automatic modular design) presents a novel approach for the automated assembly of modular control software, utilizing probabilistic finite-state machines (PFSMs) [38, 39]. PFSMs, known for their modularity and readability, comprise states and transitions. AutoMoDe employs this framework for control architecture, selecting relevant states from pre-existing behaviors and introducing modifications based on initial knowledge.

The system searches a knowledge base to find the optimal combination of modules for a specific task. Each behavior, such as 'moving from location A to B,' includes associated parameters, enabling adaptation to different scenarios. Conditions govern transitions between actions, and these may recur within a PFSM. AutoMoDe synthesizes specific module instances to create an optimized composition, exploring a search space defined by all possible PFSMs using the given modules.

However, AutoMoDe faces several limitations:

- A bias towards particular assigned modules constraining the system's symbolic power.
- Absence of knowledge ontologies and reliance on machine specification libraries, limiting reasoning abilities during control design synthesis.
- Lack of a clear language for capturing the complexities of requirements, machine specifications, and representations encoding PFSMs, leading to contextual limitations.

AutoMoDe offers a promising method for automating modular control software design using PFSMs. Despite challenges in representational power, knowledge ontologies, and specification language clarity, its potential applicability across various control systems warrants further exploration and refinement. The insights into capabilities and constraints can foster a broader understanding and development of control systems, underscoring the need for continued research and evolution.

3.2.3 Compositional Design Creation using Knowledge of Mechatronic Modularity

The advent of Intelligent Mechatronic Components (IMCs) using the IEC 61499 architecture marks a significant advancement in control software, fostering flexibility and reconfigurability [40, 41]. This approach systematically constructs modular software components with standardized IMC interfaces, providing a cohesive control architecture. The rule-based method

facilitates design, implementation, and reconfiguration, reducing time and human intervention.

Empirical evidence supports the efficiency of this approach, especially in distributed control systems. The Prolog reasoning engine, central to automation, utilizes a knowledge base that includes task specifications, processes, and IMCs, as depicted.

Despite its advantages, the approach encounters challenges such as:

- High developmental costs for IMCs.
- The requirement for specialized tools to cater to diverse stakeholders.

The exploration of mechatronic modularity aligns with the thesis's emphasis on knowledge-driven methods in control software development. It also anticipates future research directions, including extending the knowledge base to encompass context and environmental awareness.

In conclusion, this section aligns with the thesis's central theme and offers valuable insights into integrating design automation and control software. It illustrates the synergy between efficiency and adaptability, reflecting the potential trajectory of the field.

3.2.4 Formal Design Synthesis Using Assume-Guarantee Contracts Knowledge

This section presents a cutting-edge methodology that leverages assume-guarantee contracts to manage the design complexity of cyber-physical systems [42].

The methodology employs contracts to coordinate the design's horizontal and vertical dimensions, strengthening the architectural integrity of system structures and control algorithms.

The approach includes Reactive Synthesis-Based and Programming-Based methodologies, which aid in optimizing control mapping. Its adaptability is demonstrated through applications in aircraft electric power and air management systems, and the integration of a toolkit and domain-specific languages broadens the applicability of contract-based specifications.

However, the methodology encounters challenges in practical implementation, indicating areas for further refinement, such as:

- The development of abstraction layers.
- The incorporation of temporal logic.

In summary, this section extends the thesis's discussion of modular design by introducing the concept of formal contracts. It contributes a focused and academically rigorous analysis, aligning with the overarching theme of knowledge-driven control software development.

3.3 Knowledge-Driven Service Orchestration for Cyber-Physical Systems

This section delves into the knowledge-driven orchestration engine developed by Dai et al.[43], focusing on its innovative approach to service composition within industrial cyber-physical systems.

Dai et al.[43] propose an orchestration engine that leverages dynamic service composition through automated information collection and machine control commands.

Despite its proficiency in translating low-level controls into services, the engine encounters specific challenges:

- Context-dependent constraints on composition.
- Ambiguities in service descriptions.
- Absence of explicit mapping, leading to validation concerns.

In conclusion, Dai et al.'s orchestration engine, though not without limitations, represents a significant advancement in industrial agility. Introducing a novel methodology for service abstraction and control opens new avenues for exploration within the domain of cyber-physical systems.

3.3.1 Knowledge-Driven Self-Reconfiguration Design for Machine Control Systems

Zou et al.[44] introduced a methodology that leverages the Model-Integrated Computation (MIC) paradigm[45] to augment self-reconfiguration, flexibility, and robustness in machine control systems. By employing domain-specific models and languages (DSML)[46], the architecture integrates ontological models, including IEC 61499 elements[47].

The methodology emphasizes self-adaptive control modules for low-level motion control, utilizing SQWRL queries. Case studies validate its applicability across various scenarios.

However, the approach faces specific challenges:

- Lack of a standardized control model.
- Absence of formal methodologies for component interactions.
- Incomplete semantic ontology.

In conclusion, this subsection briefly overviews integrating knowledge-driven design with self-reconfiguration in machine control systems. It underscores both the achievements and areas for future research and development.

3.3.2 Knowledge Representation and Reasoning for Robotic Control

KNOWROB2, an extension of the original KNOWROB system [48], offers a significant approach to knowledge representation and reasoning in robotic control. Designed to complement the RoboEarth methodology (Section 3.2.1), KNOWROB2 integrates various forms of knowledge representation using a logic-based retrieval language. It fuses motor cognition with robot control, thereby enhancing reasoning capabilities and enabling complex tasks such as culinary preparation or scientific experimentation.

Unlike traditional query systems, KNOWROB2 actively participates in the robot's perception-action cycle. Task-specific queries often involve perceptions or motion execution, guiding object manipulation and motion parameterization. KNOWROB2's integration allows algorithm reuse and data structure sharing, enhancing efficiency.

Utilizing the KNOWROB ontology with added axioms and constraints, KNOWROB2 establishes a knowledge base for manipulation tasks. It encompasses various aspects, including robot components, objects, tasks, actions, behaviors, and the contextual environment, synchronizing these concepts across perception, reasoning, and control.

However, KNOWROB2 faces limitations, such as:

- Lack of an explicit control model, hindering validation.
- Dependence on Prolog poses practical issues as it is not a standard knowledge capture language.
- Inability to adapt automatically to variations in skills, environment, or objectives.

Addressing these challenges can lead to more advanced problem-solving and enable the creation of context-specific control strategies, highlighting areas for further development.

3.3.3 Knowledge-Based Platform for Decision Support in Engineering Design

Ming et al.[49] developed the Platform for Decision Support in the Design of Engineering Systems (PDSIDES), a structured, knowledge-based approach to engineering design decision-making.

The platform's distinctiveness lies in integrating decision-support processes (DSP) templates with ontologies, creating specialized roles for template creators, editors, and implementers. This supports diverse user requirements.

Despite its modular, ontology-driven structure, PDSIDES faces challenges:

- Neglect of distributed information control.
- Lack of formal semantic control models.
- Potential difficulties in translating complex stakeholder requirements.

In conclusion, PDSIDES represents a significant advancement in knowledge-driven engineering design, seamlessly integrating user assistance with design decision-making. While promising, the existing framework's limitations offer future research and development avenues, particularly in semantic control refinement and robust design methodology.

3.3.4 Knowledge-Based Automated Artefact Generation and Consistency in System Design

Santos et al.[50] propose an innovative approach to artefact derivation in system design, focusing on reducing inconsistencies and errors. The methodology synthesizes control designs from system requirements through contract expressions, utilizing an SAT solver for resolution.

This work integrates Contract-Based Design [50] with Computational Design Synthesis [51], enabling the automated generation of consistent artefacts. The approach leverages a SysML-inspired Domain-Specific Language (DSL) to compute compliant architectures, broadening the range of derivable artefacts.

While promising, the methodology faces limitations:

- Lack of a knowledge representation interface.
- Absence of a formalized description for the synthesized design.

Future research may focus on accommodating diverse requirements, such as temporal constraints, possibly requiring specialized solvers.

In conclusion, the work by Santos et al.[50] opens a new frontier in system design, emphasizing automated artefact generation and consistency. The fusion of Computational Design Synthesis with Contract-Based Design represents a significant advancement in control design synthesis, though areas like knowledge representation warrant further exploration and development.

3.3.5 Knowledge-Driven Framework for Digital Twin Manufacturing Cell

Zhou et al.[52] propose the Knowledge-Driven Digital Twin Manufacturing Cell (KDTMC), a groundbreaking advancement in intelligent manufacturing. This framework synergizes physical, digital, knowledge, and social domains, endowing KDTMC with cognitive abilities such as self-guided thinking, decision-making, execution, and refinement. The goal is to enhance quality and throughput while ensuring adaptability and cost reduction.

The architecture, dynamically captures knowledge from the plant system. It incorporates state-of-the-art technologies such as Digital Twin Models, Dynamic Knowledge Bases, and Knowledge-Based Intelligent Skills. This integration enables autonomous execution and facilitates self-reasoning and evolution through physical, digital, and social connections.

The primary contributions lie in the innovative application of knowledge-driven engineering, reasoning, learning mechanisms, simulations, and integrated control. However, the approach faces challenges:

- Lack of domain semantics.
- Absence of a clear method for control design composition.

The KDTMC framework represents a pivotal step in intelligent manufacturing, with significant implications for industrial control design composition. While aligning with Industry 4.0 paradigms, it highlights areas for further exploration and refinement. The introduction of KDTMC promotes transformative reductions in manual intervention, setting a precedent for self-execution in complex plant systems.

3.3.6 Semantic Extension of Nodejs library for Rapid IoT Application Development

Thuluva et al.[22] present the Semantic Node-RED methodology, a novel approach to facilitate semantically interoperable Internet of Things (IoT) applications. This methodology integrates semantic definitions, such as IoT semantic models, into Node-RED, assisting stakeholders in configuring device semantics.

The approach employs Semantic Web (SW) formalisms, including RDFS and OWL, to make knowledge machine-interpretable. It enables mechanical skill matching between application requirements and devices and introduces semantic application templates, or “Recipes,” to enhance reusability and simplify complex development tasks. The ability to interoperate across different platforms, ecosystems, or vendors further contributes to semantic interoperability.

Despite its innovative contributions, the methodology faces challenges:

- Ambiguities due to the lack of formal definitions for constructs like *capability*.
- Limitations in handling complex applications with numerous sensors and actuators.
- Performance and scaling issues related to Node-RED dependency.
- Absence of semantic validation within the device semantics definition interface, leading to potential human errors and hindering reusability.

The Semantic Node-RED methodology significantly advances IoT application development, promoting automated, consistent, and interoperable solutions. While accessible to those without semantic expertise, it highlights complexity, scalability, and validation challenges. Continued research to overcome these obstacles will enhance the adaptability and efficiency of this innovative approach across diverse technological contexts.

3.4 Summary

The previous sections explored several strategies in knowledge-driven control software design. These include approaches like RoboEarth, which focuses on sharing knowledge among robots; the KNOWROB method, which helps represent knowledge for robotic tasks; AutoMoDe, which uses probabilistic state machines for automatic control design; and various proposals that discuss component knowledge, contract-based knowledge, and capturing control logic. Table 3.1 provides a detailed summary of the reviewed articles and their key features.

Table 3.1: Comparison of Literature Approaches

System	Platform	Knowledge Reuse	Composition			Control Model	Knowledge Architecture
			Contracts	Synthesis	Service-Based		
RoboEarth	✓		✓				
AutoMoDe				✓			
A Compositional Approach to Control Software Design of Automation Systems Based on Mechatronic Modularity						✓	
Compositional design using contracts			✓				
Knowledge-driven service orchestration engine	✓				✓		
Model-based development of knowledge-driven self-reconfigurable machine control systems					✓		
KnowRob							✓
PDSIDES				✓			
CONDEnse			✓				
Knowledge-driven digital twin		✓					
Semantic Node-RED		✓			✓		

3.5 Open Opportunities

The current research in this field highlights several open opportunities for further development and innovation:

- **Generic Knowledge Ontology:** There is a need for a general knowledge ontology that captures processes, devices, and contextual information. This would allow for reusability across different technological domains.
- **Comprehensive Control Models:** Many studies use state machines for control models, but they often miss crucial aspects like control hierarchy, topology, and domain-specific vocabulary. Future work can focus on creating a model that includes these elements.

- **Knowledge Description and Validation:** Describing and validating knowledge across component specifications, technical and business domains, and contexts is still challenging, customized interfaces for representing knowledge need to be developed.
- **Understanding Component Interactions:** It is important to understand how control components interact, behave, and depend on their context. Methods such as Session Types[53] and Transition Systems[54] can be helpful in this area.
- **Knowledge-Driven Code Generation:** Knowledge-driven code generation can produce verified implementations specific to particular technologies, reducing development costs and time[55].
- **Dynamic Adaptation:** Current process descriptions are mostly static, limiting reasoning and adaptation. Learning mechanisms, like Recurrent Neural Networks[56], can improve control software design by making it more adaptive.

These open opportunities represent essential challenges that need further research. Addressing these challenges can lead to significant progress in developing knowledge-driven control software.

3.6 Conclusion

The related work in automating control software development highlights essential areas for future research. Ontology creation, control models, architectural patterns, and compositional strategies present a rich field for advancing intelligent control software development. While knowledge-driven control software is a growing field, it faces challenges such as the lack of integrated tools that adapt to environmental changes, enhance efficiency, and reduce manual errors. Building on this, the following chapter surveys controller synthesis tools, offering more profound insights into how these tools transform synthesized models into practical solutions, furthering the goal of knowledge-driven control software design.

Chapter 4

Controller Design Tool Survey

4.1 Knowledge-Driven Control Software Development

This chapter examines the tool support, design, and development needs of the industrial controller. The survey of controller synthesis tools builds on the previous chapter, identifying the gap between theoretical methods and industrial needs, particularly in tooling support. As systems become more complex, manual design and verification face limitations, making automated solutions necessary. While Linear Temporal Logic (LTL) based synthesis tools are a well-established method for controller design and synthesis, their industrial use has faced challenges [57, 58, 59]. This chapter discusses and identifies open-source tools and compares them against the industrial requirements for control software development.

4.1.1 Controller Synthesis Tool Support

Eight open-source tools were selected based on practical use, standardized LTL inputs, and independence from specialized third-party software. Table 4.1 compares these tools, outlining their features and limitations to inform future research.

The list below presents several prominent tools for synthesizing controllers from Linear Temporal Logic (LTL) specifications. Each tool has unique features and applications catering to control software development and system design aspects.

- Acacia+[60] uses symbolic incremental algorithms to synthesize winning strategies for Linear Temporal Logic (LTL) specifications. It provides a compositional approach for combining LTL formulas and generates a small, practical strategy. Acacia+ can synthesize control code, debug specifications, and create deterministic automata.
- AutoCode4[61] synthesizes reactive controllers from LTL specifications in the GXW subset. The output is in Synchronous Dataflow (SDF) format, which can be converted into

Tool	LTL Fragment	Synthesis Outcome	Strengths	Limitations
Acacia+[60]	SMV	1. DOT Diagrams as Moore Machines	1. Composition of large formulas from smaller formulae 2. Reduction of states to coBuchi automata	1. Repeatability of formulae for similar cases 2. Performance issues due to python
Autocode4[61]	GWX	1. Matlab 2. Ptolemy 3. Pseudo-Code	1. Reduction of the state space using effective GR1 approach 2. Generation of an executable code 3. API hook to extend code synthesis module 4. Ability to synthesise complex use-cases (>100 states)	1. Lack of explicit difference between assumption, and guarantee 2. Lengthy specifications become too verbose and difficult to read and understand. 3. Synthesised pseudo-code is incomplete with no transition conditions
G4LTL[62]	GR(1)	Executable PLC Code, Pseudo-Code	1. Synthesize controllers for PLC and embedded systems 2. A simple Java-based UI 3. Generated code can be used directly in PLC controllers.	1. GR(1) fragment usage reduces the readability of specs 2. Difficult to debug specifications in case of unrealizability
Lily [63] Tulip [64]	NuSMV	1. BDD and DOT Graphs	1. Python supported LTL specification 2. State-space and transition optimizations	1. Difficult to write long specs due to lack of modularity 2. Synthesised controllers are diagrams and affects reading x and understanding
STRIX [65]	SuSMV, PSP	1. BDD and DOT Graphs	1. A tool to write Requirements in constrained natural language 2. Conversion of CNL to LTL 3. Support for multiple LTL forms 4. Successfully solved most of the problems in SYNTCOMP 2019 [66] and winner of Synthesis Quantity & Quality Track	1. Needs a third party tool SpecPro/SPOT for LTL to Buchi conversion 2. APIs are not well documented
AbsSynth [67]	Python Based Temporal Properties	1. Aiger Circuit	1. A powerful tool, with optimized algorithm for synthesis 2. Winner of Realizability Track in SYNTCOMP 2019	1. Restricted usage of LTL syntax due to python based programming interface 2. The output is an Aiger circuit, which cannot be executed
ApectLTL [68]	SMV	1. Executable JAVA Code	1. Allows defining different assumptions and guarantees for different aspects of a system. E.g. Security, Safety, Optimization etc. 2. Every spec for an aspect is defined as a module.	1. Does not support defining array/list elements of a type 2. Every array element has to be accessed by a numeric constant e.g. a[2]
Spectra [69]	GWX, NuSMV, GR(1)	1. Binary Decision Diagram (BDD) 2. Pseudo-Code	1. Provides a strong language to specify modular specifications 2. Supports iterative statements for specification 3. Inbuilt model checking and synthesis tools	1. Needs JAVA environment 2. It depends on CUDD toolchain

Table 4.1: Comparison of Open Source Control Development Tools

SCADE/Lustre and Ptolemy II. It ensures traceability between requirements and code blocks and is embedded within the Ptolemy II platform for cyber-physical systems.

- G4LTL-ST[62] synthesizes control code for industrial Programmable Logic Controllers (PLCs) based on LTL specifications with arithmetic and timing constraints. It uses pseudo-boolean abstraction for data constraints and compiles timing constraints into LTL.
- Tulip Tulip[64] is a toolbox for designing controllers for hybrid systems using temporal logic specifications. It supports workflow design for systems with both discrete and continuous state spaces.
- Strix[65] is a tool for reactive LTL synthesis. It converts LTL formulas into deterministic parity automata (DPA) and solves parity games using a multi-threaded solver. The output can be a Mealy machine or an AIGER circuit.
- AbsSynthe[67] automatically constructs systems that are correct by design, using only an environment model and a property specification. It provides feedback when synthesis is not possible.
- Spectra[69] is a specification language for reactive systems. It includes features like patterns, monitors, and arithmetic operations. The tool offers a synthesizer for obtaining correct-by-construction implementations.

4.1.2 Usage of Controller Synthesis Tools

Despite the availability of several synthesis tools, none have become industry standards. Controller synthesis tools are not widely used in industrial processes, unlike modeling tools such as MATLAB and Simulink. Our experience with controllers for SKA[70], GMRT[71], and ITER[72] shows that current tools do not fully meet industrial expectations. Although some tools have been experimentally tested[73, 74, 75], they remain primarily research-focused rather than widely adopted in practice.

4.1.3 Industrial Needs and Synthesis Tools

This section reviews the selected tools in light of industrial requirements for control software development. The tools are evaluated based on open-source availability, real-world deployment, LTL support, and independence from third-party environments like MATLAB. Table 4.1 compares the tools' strengths and limitations, aligning them with industrial needs for future development.

4.1.4 Industry 4.0 Scenario

Industry 4.0 introduces new paradigms for industrial control systems. Three key enablers are the Internet of Things (IoT), Cyber-Physical Systems (CPS), and Smart Factories [76]. IoT connects physical objects, forming the basis for CPS, while CPS integrates digital and physical systems to create Smart Factories.

Industrial controllers typically include:

1. **Computer Numerical Control (CNC):** Controls manufacturing equipment, often using G-code.
2. **Programmable Logic Controllers (PLC):** Automates industrial processes with flexible programming.
3. **Programmable Automation Controllers (PAC):** Standardized PLCs with specific logic techniques.
4. **Single-board Computers (SBC):** High-end computation for advanced control tasks.
5. **PC-based Controllers:** Manage data-intensive systems with tight system integration.

Controller development remains manual, increasing costs due to time-consuming design and verification, even with tools like MATLAB and Simulink. Figure 4.1 illustrates the development process for industrial controllers.

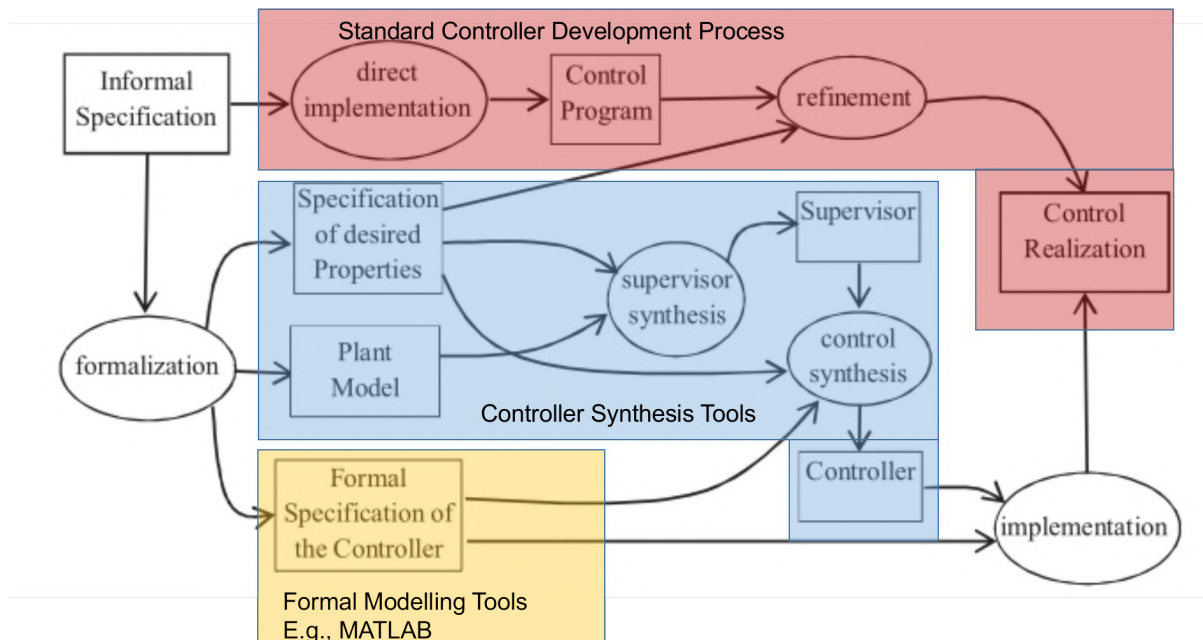


Figure 4.1: Industrial Controller Development

The shift towards IoT, CPS, and Smart Factories increases the need for advanced control software capable of managing complex subsystems. As a result, controller development becomes critical for effectively deploying Industry 4.0 technologies.

4.1.5 Gaps in Existing Controller Synthesis Tools

This section evaluates the current limitations of controller synthesis tools. Table 4.2 outlines the gaps, such as limited requirement specification interfaces, lack of domain-specific customizations, and insufficient support for integrated environments. These gaps must be addressed to make synthesis tools more practical for industrial applications.

As seen in the requirements-feature mapping, current controller synthesis tools fail to meet many industrial needs, limiting their adoption. The key gaps identified are:

1. **Limited requirement specification interface:** Writing LTL specifications is complex due to low-level syntax, and there is no support for high-level requirements or easy translation to LTL, reducing reusability.
2. **Lack of domain specificity:** These tools do not incorporate domain-specific details, making the generated controllers too generic for specialized applications like IoT or CPS.
3. **Absence of explicit models:** Model-driven approaches are successful in industry [77], but synthesis tools often hide important models. While tools like AspectLTL and SPECTRA address this, the models remain low-level and lack controller-specific semantics.
4. **Lack of design customization:** The generated controllers cannot be easily adjusted, limiting their use in different engineering contexts.
5. **Unusable implementations:** Most tools generate pseudo-code or state chart diagrams, requiring significant effort to convert into industry-standard code.
6. **Lack of trust:** The black-box nature of these tools makes it difficult for engineers to understand and verify the generated controllers, reducing confidence in their use.
7. **Lack of a uniform system model:** Different tools use varying models, leading to inconsistencies and challenges in converting between models.
8. **Lack of engineering support:** Most tools lack key features like debugging, visualization, simulation, auto-assistance, and error correction.
9. **Incompatibility between tools:** The variety of tools and their unique characteristics make integration difficult and limit the possibility of sharing components.

Sr No	Controller development Needs of Industry	Acacia+	autoCode4	G4LTL	Lily / Tulip	STRIX	AspectLTL	SPECTRA
1	Interface to capture informal requirements	✗	✗	✗	✗	✗	✗	✗
2	Interface to capture formal requirements	✓	✗	✓	✓	✗	✓	✓
3	Validate requirements feasibility, completeness, realizability	✗	✗	✓	✓	✓	✗	✓
4	Reuse of requirement descriptions (Store as macros, libraries, use across multiple descriptions, patterns)	✗	✗	✗	✗	✗	✗	✓
5	Capture requirements using higher level specification language	✓	✗	✗	✗	✗	✓	✓
6	Incorporate application domain concepts	✗	✗	✗	✗	✗	✓	✓
7	Re-usage of synthesized design	✗	✗	✗	✗	✗	✗	✓
8	Domain-specific design incorporating domain architecture	✗	✗	✗	✗	✗	✗	✗
9	Export design into domain specific executable code	✗	✓	✓	✗	✗	✓	✗
10	Design abstraction	✗	✗	✗	✗	✗	✗	✗
11	Customize implementation code while synthesizing code from design	✗	✓	✗	✗	✗	✗	✗
12	Trusted design synthesis, with no need to re-verify	✗	✗	✗	✗	✗	✗	✗
13	Operations support to model operational concerns	✗	✗	✗	✗	✗	✗	✓
14	Allow dynamic adaptation or customization of the synthesized design	✗	✗	✗	✗	✗	✗	✗
15	Engineering support (Debug, Edit Design, Trace Synthesized Controller, Requirement Editor Support)	✗	✗	✓	✓	✗	✓	✓

Table 4.2: Controller Development Requirements vs Features Offered by State-of-the-Art Tools

4.2 Opportunities

These gaps present opportunities for innovation and development in controller synthesis tools. Key opportunities include:

- **Integrated development environments for controller synthesis:** A domain-specific climate, similar to software engineering IDEs, can provide transparent, customizable, standardized solutions.
- **Domain-specific LTL languages:** Developing domain-specific languages (DSLs) for temporal requirements would simplify specification at a higher abstraction level, as demonstrated in M&cML [78] and RobotML [79].
- **Improved code generation:** Enhancing code generation strategies with configurable templates can produce more practical, multi-domain controllers [80].
- **Knowledge-driven approaches:** Storing requirements as temporal behavioral knowledge would enable reusability and compatibility in synthesizing supervisory controllers.

4.3 Conclusion

This chapter examined controller design automation tools, focusing on their applicability in the industry. The analysis shows gaps in current tools, particularly knowledge representation and usability, and highlights several opportunities for future development. The following chapter presents an empirical study to identify practical challenges in control software development projects, laying the groundwork for solutions that address these challenges comprehensively.

Chapter 5

Knowledge-Driven Approach for Control Software Development

Chapter 1 laid the foundation by highlighting the importance of knowledge-driven approaches in CSD. Chapters 2 and 3 provided a mapping study and literature review, identifying gaps in knowledge representation and the need for better domain knowledge integration. Chapter 4 discussed the available tooling support for CSD, while Chapter 5 explored the practical challenges engineers face in real-world implementations. This serves as the input for our solution to automate design composition in control systems, which is crucial for industrial automation and reconfiguration [81].

This chapter discusses our solution, a knowledge-driven approach to address the challenges in CSD by focusing on:

- Streamlining knowledge capture for efficient domain expertise collection.
- Developing frameworks for effective knowledge representation.
- Using knowledge for the auto-generation of control software.
- Presenting a Knowledge-aided Integrated Development Environment (IDE) for development, testing, and maintenance.

This chapter introduces the proposed knowledge-driven methodology that systematically integrates capability ontologies, control composition, and toolchain automation. The chapter is organized to cover the proposed methodology, the capability ontology, knowledge-driven composition architecture, K-IDE implementation, evaluation, limitations, and conclusion, continuing the progression of the thesis from previous chapters.

5.1 Related Works

The field of automated composition in control software for cyber-physical systems (CPS) has grown substantially, becoming a crucial area of research. Despite the progress, current

methodologies still face certain limitations. This section reviews key works in the field, highlighting their contributions and identifying the gaps they leave unaddressed.

The *Manufacturing Resource Capability Ontology (MaRCO)* by Järvenpää et al.[23] advanced resource capability standardization but struggled with real-time adaptability and detailed resource behavior representation. Similarly, *Capability for Industry (C4I)* by Weser et al.[82] uses semantic web technologies but does not fully account for the complexities and uncertainties of real-time production environments.

The *Model-Driven Engineering Approach* by Merdan et al.[83] offers a framework for development but lacks comprehensive solutions for dynamic reconfiguration and system synchronization. Fadhlillah et al.'s *Multidisciplinary Delta-Oriented Variability Management*[81] focuses on managing variability but does not address dynamic machine reconfiguration and domain-specific processes.

Alsafi's *Ontology-Based Reconfiguration Agent*[84] faces challenges in handling complex and rapidly changing reconfiguration scenarios, while Pentga et al.'s *Knowledge and Reasoning Framework*[85] overlooks machine variability and its impact on the system. The *Semantic Extension to Node-RED Tool* by Thuluva et al.[22] enables interoperable IoT applications but falls short on addressing controller synthesis and dynamic reconfiguration issues.

In summary, while these works contribute to knowledge-driven development, variability management, and ontology-based models, they leave gaps in fully synchronizing and dynamically reconfiguring CPS. Current approaches often lack comprehensive systems that can effectively capture, represent, and apply domain knowledge, interaction protocols, and tasks, critical for controller synthesis and reconfiguration in control software systems.

This chapter addresses these gaps using a knowledge-driven approach. The objective is to enable design composition and dynamic reconfiguration by standardizing machine knowledge's capture, representation, and use. This aligns with the thesis's focus on knowledge-driven approaches in control software development, as established in previous chapters.

5.2 Knowledge-Driven Approach for Controller Synthesis

Adapting to changes in the environment or system state is crucial for maintaining optimal performance and functionality[86]. A clear understanding of a device's capabilities is essential for designing effective control strategies at both the design and runtime stages. This knowledge enables dynamic reconfiguration, allowing the system to adapt as needed. The capability-driven approach for controller synthesis addresses these challenges in CPS by leveraging the functionalities of various devices for efficient controller design[87].

- Controller Synthesis Process

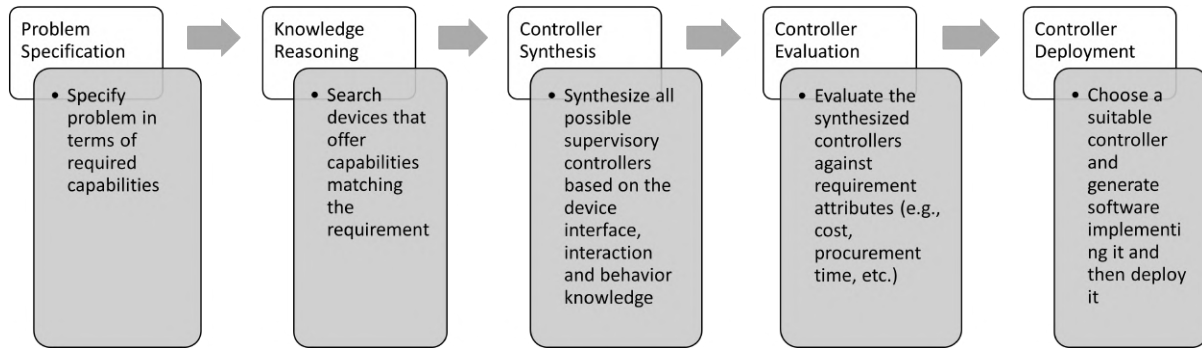


Figure 5.1: Controller Synthesis for Control Software Composition

The proposed controller synthesis process uses knowledge representation, reasoning, and synthesis techniques to automate controller design dynamically. Figure 5.1 shows a high-level view of this process.

The synthesis process includes knowledge acquisition, reasoning, controller design, and deployment. It integrates device characteristics, specifications, and operational details into the control system.

- Understanding Device Capabilities

Device capabilities refer to specific functions a component can perform. Capturing these capabilities is essential for configuration management, reasoning, and effective controller synthesis[88, 89].

- Components of Capability Self-Description

Describing capabilities, interfaces, and context is crucial for composition and reasoning. Key components include standardized functionality descriptions, interface specifications, interaction protocols, and contextual dependencies[88, 89, 90, 91].

- Differentiating Capabilities from Services, Functionalities, and Features

Capabilities	Functionalities	Features	Services
Specific functions a device can perform.	Range of behaviors of software.	Tools or functions that perform tasks.	Functions that meet a specific need.
Capture intrinsic abilities.	Include capabilities and behaviors.	Vary in complexity.	Range from simple to complex services.

Table 5.1: Comparison of Capabilities, Services, Functionalities, and Features

Understanding the distinctions between capabilities, services, functionalities, and features is important in controller synthesis. Focusing on capabilities as intrinsic abilities enables more effective design and dynamic composition[92, 93, 94].

- **Formal Representation of Capabilities**

The following section comprehensively represents the capability ontologies and their components, including system behavior, requirements, and interaction protocols. This allows for precise composition and reasoning about device capabilities.

5.3 Capability Ontology for Control Components

This section introduces a structured capability ontology that captures different types of knowledge required for designing and managing a control component. In addition to formally defining the core ontology layers, we explicitly connect each formal element to an intuitive meaning so that the main thread of the argument remains visible even when rigorous structures are introduced.

5.3.1 System Interface

The system interface forms the foundation of the ontology. It defines how a control system interacts with its environment. This includes the system's possible states, accepted inputs, produced outputs, initial configurations, and how it transitions from one state to another. The interface also includes a mechanism to relate internal states to externally observable outputs. This establishes a clear boundary between the system's internal logic and external behavior.

5.3.2 System Behavior

System behavior refers to the observable patterns generated by the system when it operates from its initial state. These patterns result from how the system responds to inputs over time. System behavior includes all possible sequences of outputs that the system can produce during valid executions. It is useful for analyzing the system from an external point of view without exposing internal details.

5.3.3 Interaction as Behavior Execution

To enable the execution of particular behaviors, the system interacts with external components, such as controllers or operators. These interactions follow well-defined protocols that

describe the valid order of messages or commands exchanged. Each interaction protocol specifies what inputs can be sent to the system and how it is expected to respond. Consequently, only a specific subset of the system’s behaviors will be realized during the interaction session.

5.3.4 Capabilities as Abstract Labels for Behavior

A capability is a semantic label assigned to a set of system behaviors that satisfy a set of requirements. If a system can exhibit all the behaviors necessary to meet a particular requirement, it is said to possess the capability associated with that requirement. Capabilities provide a simplified, high-level abstraction of component behaviors.

Preview of the formal definition (informal explanation). To avoid losing the conceptual thread when moving into formalism, we summarize the constituents of the capability definition upfront:

- **Interface:** the externally visible interaction boundary of the component (states, inputs/outputs, transitions), as introduced in the *System Interface* layer above.
- **Behavior:** the set of observable execution traces admitted by the interface, as introduced in the *System Behavior* layer above.
- **Context:** the environmental and session-related assumptions under which the capability is meaningful (e.g., session type, operating mode, workflow context). This is operationalized later using classes such as *SessionType* and *Workflow* and their relations.
- **Preconditions:** conditions that must hold before attempting the capability (e.g., safety interlocks, resource availability, readiness of devices). These are represented later as constraints associated with actions/activities and can also be captured by rule-based constraints.
- **Postconditions:** conditions that must hold after successful completion of the capability (e.g., object placed, actuator returned to idle). These are represented later as outcome constraints over states/outputs and can be linked to workflow completion conditions.

In simple representation, capability can be defined as a 5-tuple:

$$Capability = (Interface, Behavior, Context, Preconditions, Postconditions)$$

Constituent summary and traceability within this chapter. The following table provides a compact “legend” for the 5-tuple so that each element is immediately interpretable at the point of definition, while also indicating where the corresponding notions are operationalized in the ontology introduced later in this section.

Intuition: what a capability bundles.

Tuple Element	Meaning (brief)	Where realized here
Interface	Boundary definition: states, inputs/outputs, transitions, observables	“System Interface” + Interface class
Behavior	Observable traces admitted by the interface under valid executions	“System Behavior” + Behavior representation
Context	Assumptions that scope the capability (session, workflow, environment)	SessionType, Workflow, Activity (see “Classes” / “Properties”)
Preconditions	Required conditions before execution (safety, readiness, resource availability)	Action/Activity constraints + rule-based constraints (see “Rules”)
Postconditions	Required outcomes after execution (goal achieved, safe termination state)	Workflow/Activity completion semantics + rule-based constraints (see “Rules”)

Table 5.2: Interpretation of the capability 5-tuple and how each element is operationalized in the ontology.

A capability is not merely a label; it is a *contract* that bundles (i) what the component exposes (**Interface**), (ii) what it can produce under valid executions (**Behavior**), (iii) under what operating assumptions it is meaningful (**Context**), and (iv) the safety/goal constraints that must hold before and after invocation (**Preconditions**, **Postconditions**). This contract-like interpretation is crucial for systematic design, validation, and subsequent synthesis of control software.

A capability denotes what a device can do, independent of quantitative limits such as throughput or load, which are treated as capacity attributes. For example, multiple robotic arms may share the same pick-and-place capability, even though their speeds or payload limits differ. This abstraction enables reuse of control logic across devices and supports dynamic reconfiguration when devices are replaced or fail, without redesigning workflows.

5.3.5 Relationships in the Capability Ontology

The key concepts of the capability ontology are organized hierarchically, as follows:

1. The **system interface** defines the states, inputs, outputs, and state transitions.
2. The **system behavior** is derived from the interface as sequences of observable outputs.

3. The **interaction** protocols constrain which behaviors are executed in practice.
4. The **capabilities** summarize and label the behaviors that fulfill specific requirements.

This results in the following layered structure:

Interface $\rightarrow$ Behavior $\rightarrow$ Interaction $\rightarrow$ Capability

Each layer builds on the previous one, forming a comprehensive ontology to support system design, analysis, and control. In particular, the capability 5-tuple introduced earlier (Table 5.2) should be read as a compact encapsulation of this layering: **Interface** and **Behavior** are foundational, **Interaction** constrains realizable behavior during operation, and **Context/(Pre,Post)** conditions scope and constrain the validity of the capability contract.

5.3.6 Example: Robotic Pick-and-Place Arm

To clarify the ontology concepts, consider a robotic pick-and-place arm:

- **System Interface:** Defines the states (such as idle, moving), accepted commands (such as pick, place), and outputs (such as position).
- **Behavior:** Describes valid sequences of actions like picking an item, moving it, placing it, and resetting.
- **Interaction:** Defines the sequence in which commands are issued and acknowledgments are received during a task.
- **Capability:** Abstractly described as the “pick-and-place capability,” representing the system’s ability to complete pick-and-place operations reliably and safely.

Capability tuple instantiation For this example, the 5-tuple can be understood concretely as follows:

- **Interface:** commands {pick, place, moveTo, reset}, outputs {position, ack, error}, and states {idle, moving, holding, placing}.
- **Behavior:** traces such as idle $\rightarrow$ moving $\rightarrow$ holding $\rightarrow$ placing $\rightarrow$ idle with corresponding observable outputs.
- **Context:** the active session/workflow in which the arm is invoked (e.g., pick-and-place workflow), the current session type (manual/automatic), and environmental assumptions (object present, workspace clear).

- **Preconditions:** object detected in grasp region, gripper available, safety interlock satisfied, arm not in fault state.
- **Postconditions:** object placed at target location within tolerance, gripper released, arm returns to a safe/idle state or provides a well-defined error output.

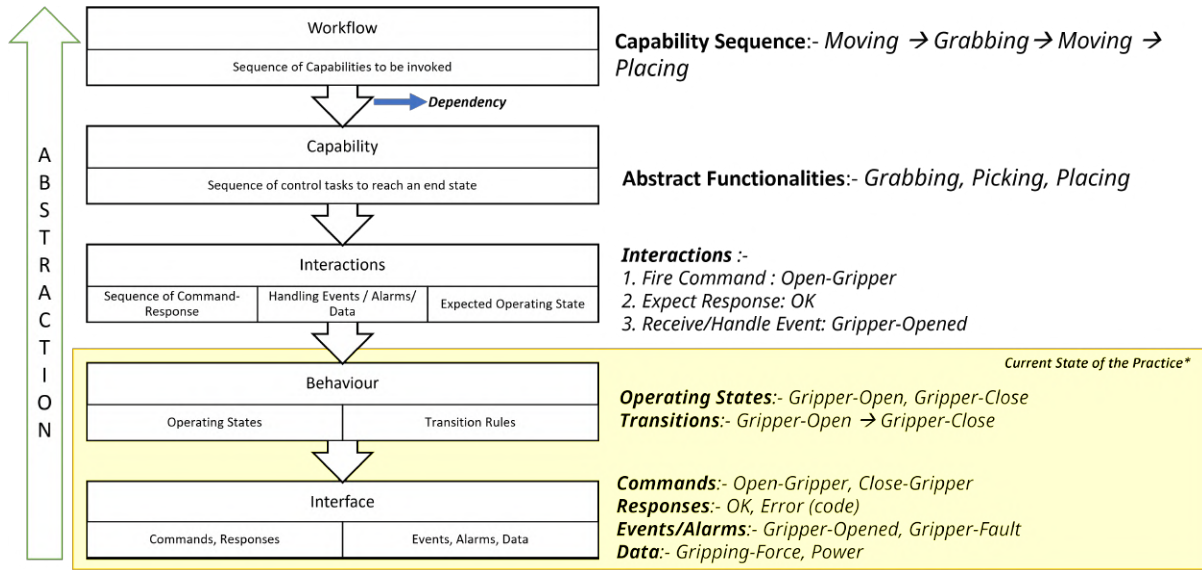


Figure 5.2: Ontology illustrating abstraction from Interface through Behavior, Interactions, Capability, to Workflow.

Figure 5.2 visualizes this ontology, emphasizing the progression from low-level system interfaces to high-level workflows, enabling traceability and systematization across abstraction layers. The notions of *context* (workflow/session), as well as *preconditions/postconditions* (constraints before/after invocation) are made explicit later in the ontology through the introduction of *Workflow*, *SessionType*, and rule-based inferencing mechanisms.

The robotic pick-and-place arm example above, is used as a *minimal, mechanical* running example to introduce and validate the *conceptual layering* of the capability ontology, from **Interface** to **Behavior** to **Interaction** to **Capability**, and to make the capability contract (including *Context*, *Preconditions*, and *Postconditions*) concrete at the point where the ontology is defined. Chapter 6, in contrast, focuses on the *language-level realization* of the same ideas: it demonstrates how the ontology concepts are captured and manipulated using the proposed DSLs for practical control design tasks. For this purpose, Chapter 6 uses a smart meeting-room / video conferencing scenario, because it naturally exhibits heterogeneous devices, session types, workflow orchestration, and dynamic reconfiguration, thereby stress-testing the DSL constructs in a realistic IoT setting. Although the two scenarios are different, they share the same methodological backbone: the above described example establishes *what must be represented* (capability contracts and their abstraction layers), while the Chapter 6 example demon-

Class	Description
Action	Models an action executable by a device
Activity	Encapsulates sequences of actions for dynamic reconfiguration
Device	Represents a physical device with defined capabilities
Capability	Abstracts functionality offered by a device
Interface	Models the interaction interface of a device
SessionType	Specifies session types governing device interactions
Workflow	Comprises activities orchestrating dynamic reconfiguration

Table 5.3: Classes in the Capability Ontology

ity is not asserted in isolation, but is tied to workflows, session types, and activity requirements that collectively scope when a capability is valid and how it is invoked.

Object Property	Description
fulfilledBy	Relates a capability to the device that fulfills it
hasActivities	Associates workflows with their constituent activities
hasCapability	Associates devices with their capabilities
hasInteractions	Associates session types with possible interactions
hasInterface	Associates devices with their interfaces
implementsSession	Associates devices with session types they implement
requiredCapability	Associates activities with required capabilities

Table 5.4: Object Properties in the Capability Ontology

5.3.10 Rules

The ontology integrates Semantic Web Rule Language (SWRL) rules [96] to enable inferencing. For instance, the following rule infers device capabilities based on workflow invocations:

```
Device(?d) ^ CapabilityInvocation(?ci) ^ Capability(?c) ^
invokesCapability(?d, ?ci) ^ offersCapability(?ci, ?c) ->
hasCapability(?d, ?c)
```

Such rules allow dynamic adaptation of capability knowledge in environments where devices may acquire or lose capabilities. Importantly, beyond inferring *which* capabilities hold, rule-based constraints can also be used to encode and validate elements of the capability contract, including **Preconditions** (e.g., disallowing invocation when a required session type is not active) and **Postconditions** (e.g., asserting that a workflow-level goal state holds after completion), thereby operationalizing the last two elements of the 5-tuple.

5.3.11 Responsibility-Oriented Capability Ontology

The concept of *capability* is central to the knowledge-driven methodology presented in this thesis. Conventionally, a capability refers to an abstract behavioral label assigned to a device or a system component that can perform a designated operational role. To ensure better alignment with object-oriented design principles and to promote clearer separation of concerns, this work adopts and extends the notion of *Responsibility-Driven Design (RDD)* [97].

In RDD, a **responsibility** is a contract that an entity commits to fulfill, usually defined by the actions it performs and the information it requires. This design paradigm encourages modular architecture, enhances component cohesion, and supports explicit specification of interfaces. Interpreting capabilities as responsibilities significantly improves the semantic clarity of roles assigned to system components. This “contract” interpretation also aligns directly with the capability 5-tuple introduced earlier: responsibilities can be understood as commitments over a well-defined **Interface** and allowable **Behavior**, scoped by **Context**, and constrained by explicit **Preconditions** and **Postconditions**.

Within this framework, the capabilities ontology is reformulated as an ontology of responsibilities. This results in a more granular and meaningful classification of component roles, as described below:

- **Sensors** generate observations of particular environmental or system variables.
- **Actuators** are responsible for performing state-altering actions, subject to well-defined preconditions.
- **Controllers** assume responsibilities for sequencing, composing, and coordinating the responsibilities of other components to achieve higher-level system goals.

This responsibility-oriented interpretation remains consistent with the underlying transition system model. Responsibilities correspond to allowable behaviors constrained by interaction protocols and contextual assumptions. By embedding responsibilities in the capability ontology, we enhance semantic traceability, support automated synthesis of control strategies, and facilitate rigorous system validation and verification.

5.3.12 Comparison with Other Ontologies

Table 5.5 contrasts the proposed ontology with related works. While prior ontologies focus on static device descriptions, the proposed model supports workflows, dynamic reconfiguration, and inferencing.

Ontology	Year	Device Interface	Device Behavior	Device Interaction	Workflow Representation	Inference Rules	Persistence in Graph Database	Supports Dynamic Reconfiguration
Our Capability Ontology	2023	Yes	Yes	Yes	Yes	Yes	Yes	Yes
IotSchema [95]	2017	Partial	No	Partial	No	No	Yes	No
SAREF [98]	2015	Yes	No	No	No	No	Yes	No
WoT [99]	2020	Yes	Partial	No	No	No	Yes	No
SSN/SOSA [100]	2017	Yes	No	No	No	No	Yes	No

Table 5.5: Comparison of Ontologies for Designing Control Software

5.3.13 Persistence in a Graph Database

Figure 5.4 illustrates the storage of capability knowledge in a graph database, facilitating efficient querying and reasoning over the ontology.

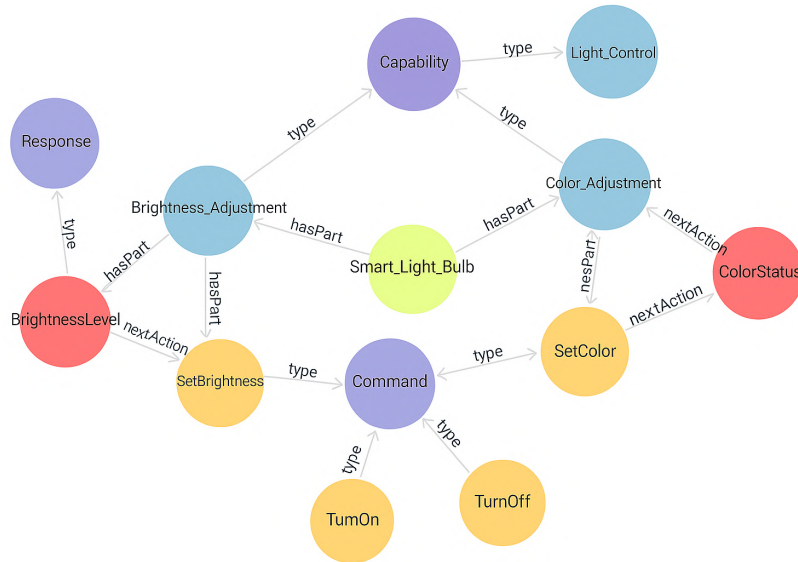


Figure 5.4: Persistence of Device Knowledge in a Graph Database

Graph databases (e.g., GraphDB, Stardog) allow ontology instances to be modeled as nodes and relationships as edges, supporting SPARQL queries and rule-based reasoning. This representation offers a robust and scalable solution for managing dynamic control software in complex IoT environments. In particular, graph persistence ensures that the complete capability contract (Interface/Behavior plus contextual and pre/post constraints) can be queried, audited, and validated systematically at runtime and during evolution of the control software.

5.4 Composition Architecture

The Sensor Actuator Control Element (SACE) architecture [101, 102], illustrated in Figure 5.5, is a model-driven framework designed for control and monitoring software in large real-time systems, such as radio telescopes. The architecture assumes that control software consistently performs key functions: communication, coordination, command issuance to subsystems (actuators), and data collection, processing, transmission, and archiving from subsystems (sensors).

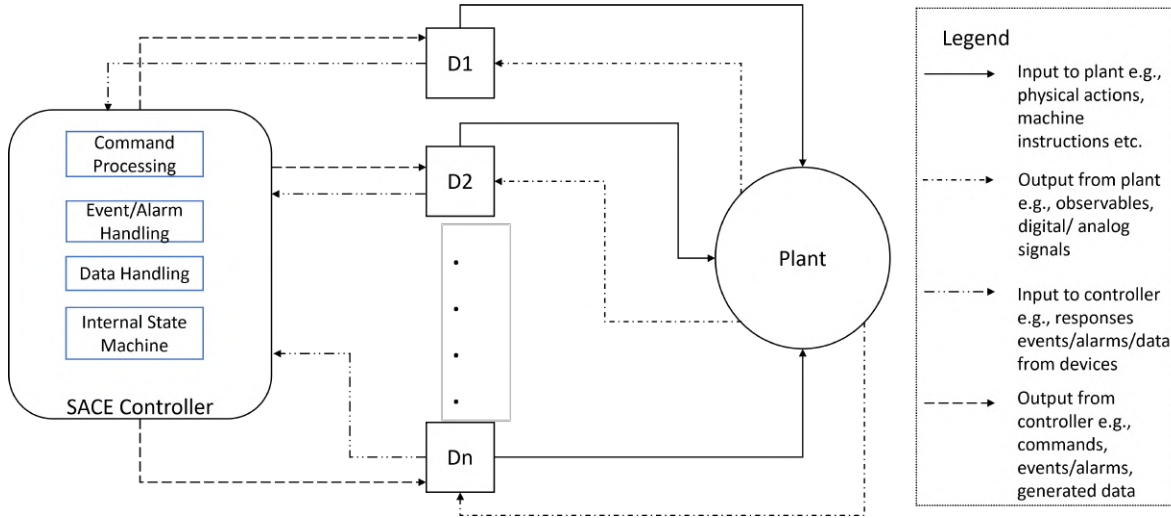


Figure 5.5: SACE Architecture

The SACE architecture organizes control nodes in a recursive, hierarchical manner, similar to how the proposed capability ontology structures device capabilities. Each control node coordinates its subsystems, analogous to how the `Device` class coordinates its `Capabilities`. The control software for each node is generated from specifications related to control logic, data processing, and event handling, just as the ontology defines device rules and properties.

A key feature of the SACE Node, the fundamental building block of this architecture, is that it provides three functional interfaces: a command interface, a data interface, and an event interface. These interfaces facilitate the hierarchical composition of control nodes, reflecting the recursive nature of the ontology.

In control software reconfiguration, the SACE architecture offers a flexible framework for adapting to changing control requirements. This adaptability aligns with the ontology's emphasis on dynamic reconfiguration, reducing development time and making software evolution easier. The reconfiguration process for control software changes follows the same approach in handling subsystem changes.

The proposed ontology builds on the SACE architecture by defining device capabilities and control interactions in a structured hierarchy, facilitating decision-making and control across

the system. The architecture provides a central supervisory controller that gathers information from lower levels, analyzes it, and issues control commands. This hierarchical design enables better coordination, fault tolerance, and scalability.

The next section introduces an algorithm for system composition, further enhancing the adaptability of the SACE architecture for control software reconfiguration.

5.5 K-IDE Platform for Knowledge-Driven Control Design and Dynamic Reconfiguration

This section describes an Integrated Development Environment (IDE) platform that leverages capability knowledge to automate control design and facilitate dynamic reconfiguration in various applications. K-IDE integrates the Capability Ontology, the SACE architecture, and the algorithm for composing higher-order machines to streamline control software development.

5.5.1 Platform Overview

The K-IDE platform is a comprehensive tool for control system engineers, providing a user-friendly interface and features to support control design, analysis, and reconfiguration. K-IDE utilises capability knowledge to automate key aspects of control software development, including device selection, control strategy formulation, and system reconfiguration. Figure 5.6 illustrates K-IDE's structure, showing the various modules that enable knowledge-driven controller synthesis and reconfiguration.

K-IDE is based on the knowledge-driven approach (KDA) [89] and follows the KDA stack. The modules, shown in Figure 5.6, are implemented using the technology stack indicated on the left. These modules are integrated into the Eclipse framework [103] to create a cohesive development environment.

5.5.2 Automated Control Design

K-IDE automates the control design process using the Capability Ontology and the algorithm for composing higher-order machines. Users can specify control objectives with a domain-specific language (DSL) [104] and define desired workflows within the K-IDE [88]. K-IDE uses ontology to identify devices with the necessary capabilities and session types. The algorithm then automatically composes controllers based on the identified devices, session types, and the SACE architecture. The generated control design is represented in a DSL specific to control design [88].

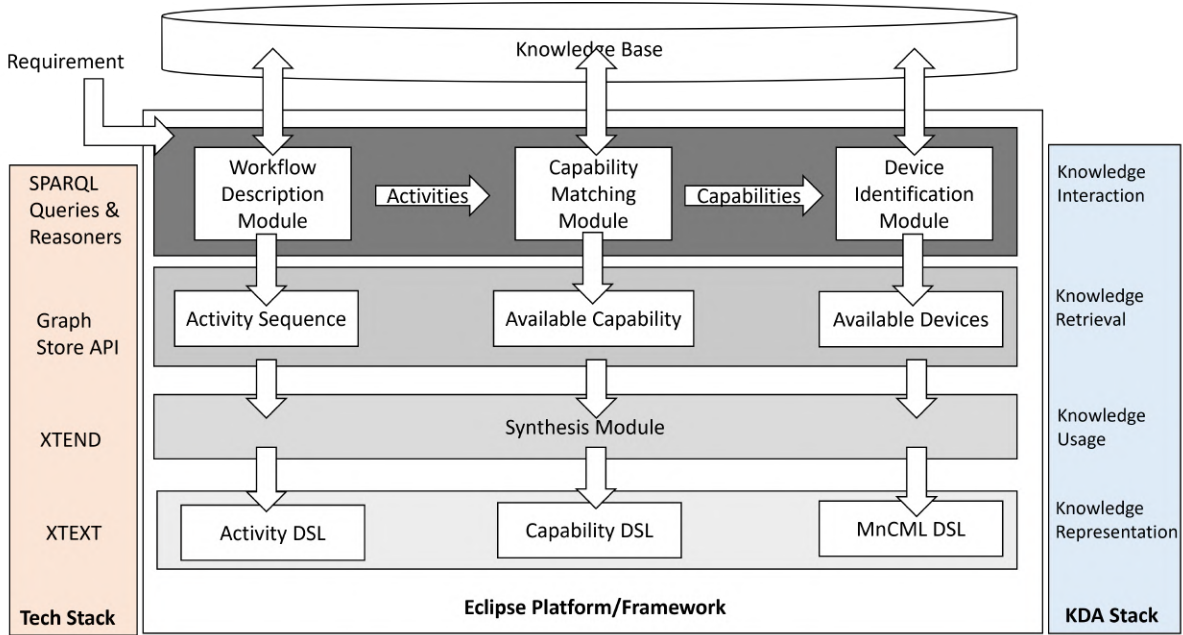


Figure 5.6: K-IDE for Control Software Development

This automation significantly reduces the manual effort needed for device selection, control strategy design, and component integration. K-IDE ensures design validation through ontology and built-in DSL validation mechanisms that detect unwanted behaviors. This allows control system engineers to focus on high-level objectives while K-IDE manages the details of device selection and controller composition.

5.5.3 Semantic Code Generation Approach

The K-IDE uses a semantic code generation approach, which uses a custom DSL to map semantic entities to code templates. This approach promotes knowledge reuse and flexibility, streamlining the code development process.

The code generation process starts by querying domain knowledge to identify relevant entities and mappings. These mappings are then used to merge domain knowledge with pre-defined code templates, producing the final code. Figure 5.7 illustrates the architecture of this code generation mechanism, which leverages domain knowledge. Figure 5.8 shows an example of the DSL used to generate robotic control software code using the Robot Operating System (ROS) [34].

This approach offers two key advantages:

- **Knowledge Reuse:** By reusing existing domain knowledge, the generated code reflects the most accurate and up-to-date information, essential in complex, evolving fields.

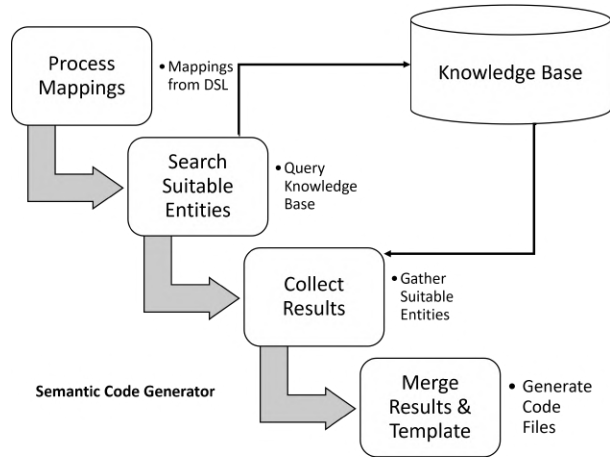


Figure 5.7: Semantic Code Generation

```

# Create Action Server for Async Command «command»
def action_server_for_command_«command»(self):
    self.as_«command» = actionlib.SimpleActionServer
    ('~/«cn»/«command»', «command»Action,
     execute_cb=self.handle_action_command_«command»,
     auto_start = False
    )
    self.as_«command».start()

# Handle the Action Request Received for Command «command»
def handle_action_command_«command»(self, goal):
    r = rospy.Rate(1)
    success = True
    rospy.loginfo('Executing Goal')
    # Create Business Logic
    «getStateTransition(cb)»
    # start executing the action below
    # Execute Action
    if success:
        rospy.loginfo('Succeeded')
        # Do the pprocessing of the goal
        self.as_«command»._feedback = MOVE_TO_LOCATIONFeedback()
        self.as_«command»._result = MOVE_TO_LOCATIONResult()
        «IF(cb!=null && cb.responseBlock!=null && cb.responseBlock!=null)»
        «FOR resB1 : cb.responseBlock»
        «IF resB1.response!=null && resB1.response.parameters!=null»
        «FOR par:resB1.response.parameters»
        #self.as_«command»._result.«GeneratorUtils.getParameterName(par)»
        = Initialize Value
        «ENDFOR»
        «ENDIF»
    """

```

The diagram shows the mapping of DSL to ROS code. 'Semantic entities' points to the «command» placeholders in the code. 'Custom Queries' points to the «getStateTransition(cb)» line. 'Code template' points to the «FOR resB1 : cb.responseBlock» and «FOR par:resB1.response.parameters» lines.

Figure 5.8: Mapping DSL for Robotic Control Software Generation using ROS

- **Flexibility:** The custom DSL allows for adaptable mappings that can evolve as the domain changes, ensuring that K-IDE can accommodate new knowledge and requirements.

The semantic code generation approach has been successfully applied in various scenarios, including white/grey-box digital twins [91, 105] and automated control software development [88, 89]. The benefits of this approach are demonstrated through evaluations of these use cases.

In summary, the semantic code generation approach provides a flexible and efficient framework for generating accurate, domain-specific code, leveraging knowledge to improve the quality and relevance of the final software.

5.6 Threats to Validity

This section outlines the potential threats to the internal and external validity of the proposed knowledge-driven control software development approach. These threats are important to acknowledge for understanding the applicability limits and potential biases in the evaluation.

5.6.1 Internal Validity

Ontology Completeness: The effectiveness of the synthesis and reconfiguration mechanism depends on the completeness and correctness of the device capability ontology. Missing or inconsistent ontology entries may lead to synthesis failures or incorrect control logic. While domain experts contributed to the ontology modeling, it is not exhaustive across all control device classes. Potential threats arising from semantic drift or ontology versioning over long-term maintenance remain a topic for future exploration.

Ontology Evolution Challenges: An additional source of potential threat arises from the long-term maintenance and evolution of the capability ontology. As new devices, hardware platforms, protocols, and control interfaces are introduced into industrial systems, the ontology must be updated to reflect these changes. Incomplete or inconsistent updates may lead to misalignments between the synthesized control logic and the physical system capabilities. While ontology management tools and version control strategies can partially mitigate this issue, the problem remains an open area for future research in sustainable knowledge evolution for control software systems.

Synthesis Algorithm Generality: The composition and synthesis algorithms assume deterministic transition systems and valid interface specifications. Errors in encoding device responsibilities or incorrect interface mappings can lead to non-terminating or invalid configurations.

Tooling Integration: The implementation in K-IDE integrates multiple components (ontology editor, DSL editor, code generator). Failures in any one module, especially code genera-

tion templates or query resolution logic, can affect overall behavior. While tested in selected scenarios, no formal guarantees exist for tool-wide correctness.

5.6.2 External Validity

Domain-Specific Generalizability: The evaluation is conducted primarily in industrial automation domains (e.g., robotic warehouse, vehicle entry, smart rooms). While the abstraction mechanisms are general, domain-specific extensions to the ontology and DSLs are required when applying the approach to healthcare, aerospace, or other CPS domains.

Scalability: The synthesis algorithm and semantic reasoning are tractable at the scale of 200 devices (largest evaluated scenario). However, it is not empirically validated for systems with thousands of devices or highly concurrent compositions. Graph query performance and reasoning latency may become limiting factors.

Middleware and Runtime Assumptions: The generated control code assumes an existing reliable middleware to handle communication, concurrency, and real-time execution. The approach does not address middleware design, which may affect the runtime behavior of the system if misaligned with design-time assumptions.

User Knowledge Dependency: Successful use of K-IDE depends on domain engineers accurately modeling the devices and workflows using DSLs. While K-IDE assists through abstraction, a learning curve remains for effective modeling and validation.

In summary, while the approach shows promising results, further validation is necessary to establish its robustness across other domains, tooling stacks, and deployment environments.

5.7 Conclusion

This chapter presented the application of capability knowledge ontology in control software composition, focusing on a Knowledge-aided Integrated Development Environment (K-IDE) platform. Key advancements in control software development have been made by integrating capability knowledge ontology with the Sensor and Actuator Control Element (SACE) architecture[101] and an algorithm for composing higher-order machines. The contributions include:

- **Streamlining Development:** Integrating the K-IDE platform with capability knowledge ontology has made the development process more efficient, reducing manual effort.
- **Automating Control Design:** Using Capability Ontology for automatic device selection, control strategy formulation, and system reconfiguration has improved efficiency and reduced errors.

- **Adapting to Changes:** The dynamic reconfiguration capabilities of the K-IDE allow adaptation to operational changes, equipment failures, or new device additions without extensive manual intervention.
- **Leveraging Ontology for Automation:** The capability ontology supports accurate representation of device capabilities, enabling precise selection, integration, and operation of control systems.

Integrating capability knowledge ontology into control software composition provides a structured approach to improving efficiency and adaptability in control software development.

The following chapters present translating this knowledge into control software code. They will discuss the development of the Knowledge-aided Integrated Development Environment (K-IDE) and a domain-specific language (DSL) to represent and utilize the knowledge within the environment. This will cover how knowledge is captured, described, and transformed into domain-specific views to generate actionable code.

The following chapters will demonstrate how knowledge-driven approaches can be implemented through specific tools and methodologies, continuing the development of the thesis's focus on knowledge-driven strategies in Control Software Development.

Chapter 6

Knowledge-aided Integrated Development Environment for Control Software

Building on the previous chapter, which detailed the capability knowledge ontology for control software, this chapter focuses on practically implementing these concepts. The previous chapter laid the foundation for exploring how the discussed knowledge-driven approach can be applied, and this chapter introduces the Knowledge-aided Integrated Development Environment (K-IDE).

K-IDE is designed to handle the development of control software development, using the knowledge-driven approach and the use of capability knowledge ontology. It integrates domain-specific knowledge into the development process, providing more accurate and efficient solutions. K-IDE uses domain-specific language instead of generic programming syntax, enabling context-aware knowledge usage based on the domain knowledge.

This chapter describes the design, implementation, and validation of K-IDE, showing how domain knowledge can enhance the development process and align software with specific industry needs. It builds on the theoretical work discussed earlier and demonstrates its practical application, continuing the progression from theory to implementation.

The landscape of software development has been significantly enhanced by Intelligent Integrated Development Environments (IDEs) such as Microsoft's Visual Studio Code, JetBrains' IntelliJ IDEA, and Eclipse. These IDEs have augmented the speed and precision of software creation. However, while they excel in providing syntactic assistance, they often fall short in delivering semantic aid, knowledge reuse mechanisms, context-aware recommendations, and collaboration support [106].

The limitations of these IDEs become particularly evident in the realm of control software development, characterized by its complex systems and the severe consequences associated with failure. Current IDEs, with their focus on individual project scopes, fail to harness the broader knowledge of the software development community fully. Moreover, they inadequately address the growing need for collaboration tools in an era dominated by team-based and open-source projects.

6.1 Knowledge enabled -Integrated Development Environment (K-IDE)

While Chapter 5 introduced the conceptual framework and theoretical foundations of the knowledge-driven control software development methodology, this chapter focuses on the practical realization of these ideas through the implementation of the Knowledge-aided Integrated Development Environment (K-IDE). In particular, this chapter presents the system architecture, internal modules, domain-specific languages, and the integrated toolchain that operationalizes the knowledge representation and synthesis algorithms discussed earlier. The goal is to translate the abstract knowledge representations into a fully functioning development platform for control software engineering.

The Knowledge-aided Integrated Development Environment (K-IDE) is a tool that uses the Knowledge-Driven Approach (KDA) [90, 91]. KDA is a four-layer framework comprising knowledge storage, acquisition, retrieval, usage, and representation. This framework enables K-IDE to automate control design and software code generation, as shown in Figure 6.1.

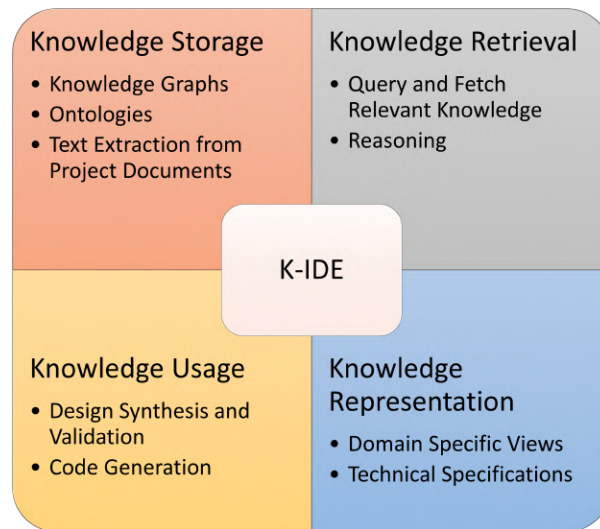


Figure 6.1: The Knowledge-Driven Approach (KDA) Stack

- **Knowledge Storage and Acquisition** The first layer of the KDA stack focuses on knowledge storage and acquisition. This involves gathering information from various sources, such as domain-specific literature, expert knowledge, and practical observations. This collected knowledge, which includes device data and control strategies, helps K-IDE make intelligent decisions and create customized solutions.
- **Knowledge Retrieval** The second layer, knowledge retrieval, enables K-IDE to access and use the stored knowledge as needed. This is done through data extraction and querying

mechanisms, ensuring that the right information is available at the right time to support decisions during development.

- **Knowledge Usage** The third layer, knowledge usage, applies the retrieved knowledge to automate different aspects of control software development. Using this knowledge, K-IDE reduces manual intervention, making the development process more efficient and accurate.
- **Knowledge Representation** The fourth layer, knowledge representation, involves organizing the knowledge in a way that K-IDE can easily process. This uses a Capability Ontology and a Domain-Specific Language (DSL) [104]. These tools allow K-IDE to understand the control system and generate suitable control designs.

Eclipse-based language engineering frameworks were selected due to their mature support for domain-specific language development, graphical modeling, and integration with model transformation pipelines. These tools enable reproducibility, extensibility, and maintainability of the proposed platform, which are essential for long-term industrial adoption.

In summary, K-IDE uses the principles of the knowledge-driven approach to automate complex tasks, adapt to changing requirements, and provide practical solutions for control software development. This creates a strong foundation for future tools using knowledge-based and intelligent software engineering approaches.

6.2 K-IDE: Architectural View

The Knowledge-aided Integrated Development Environment (K-IDE) is designed to streamline control software development by integrating a collaborative knowledge database. Its architecture focuses on capturing and utilizing knowledge from developers and domain experts to enhance software development processes. This section provides an overview of K-IDE's architecture and its approach to knowledge collaboration.

K-IDE's primary function is to generate customized software code, identify necessary resources, and establish connections between the controller and the identified resources. The architecture centres on the concept of "capability," representing the operational knowledge of components. K-IDE uses Domain-Specific Modeling Languages (DSML/DSLs) to describe component interfaces and capabilities.

The architecture combines two knowledge bases: RDF/OWL-based ontologies [107], which represent component and capability knowledge in a knowledge graph, and a machine learning model that predicts process flows and their related capabilities [88]. When engineers provide a high-level description of an objective, the machine learning model suggests appropriate process flows and identifies the necessary capabilities. The synthesizer then selects the resources

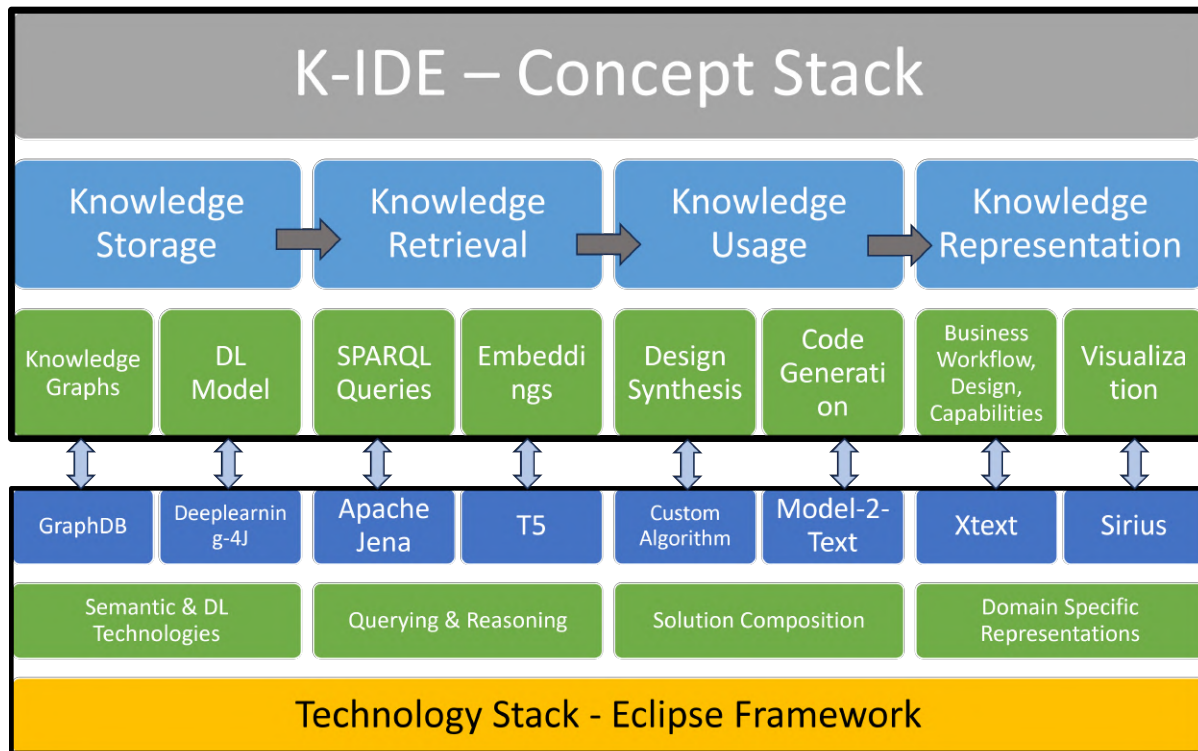


Figure 6.2: Solution Architecture Using Semantic Self-Description Knowledge

with the required capabilities and integrates them into the process flow, creating a hierarchical controller.

K-IDE continuously learns from interactions with developers and domain experts. This collaborative knowledge-sharing process allows K-IDE to provide context-sensitive support, adapting its suggestions based on evolving insights and challenges. As the shared knowledge base grows, K-IDE remains flexible and up-to-date with the latest information, improving its effectiveness in software development.

In summary, K-IDE's architecture promotes ongoing learning and collaboration, where users benefit from shared knowledge and experience. It serves as a platform for leveraging combined knowledge and artificial intelligence to advance control software development.

6.3 Domain-Specific Languages: The Core Vocabulary of K-IDE

K-IDE relies on domain-specific languages (DSLs) to capture and manage the knowledge needed for effective software development. This section introduces three key DSLs that form the core of K-IDE's knowledge management: M&CML, Activity DSL, and Capability DSL. These languages are tailored to the specific needs of control software development and streamline the process by providing structured, standardized ways to represent information.

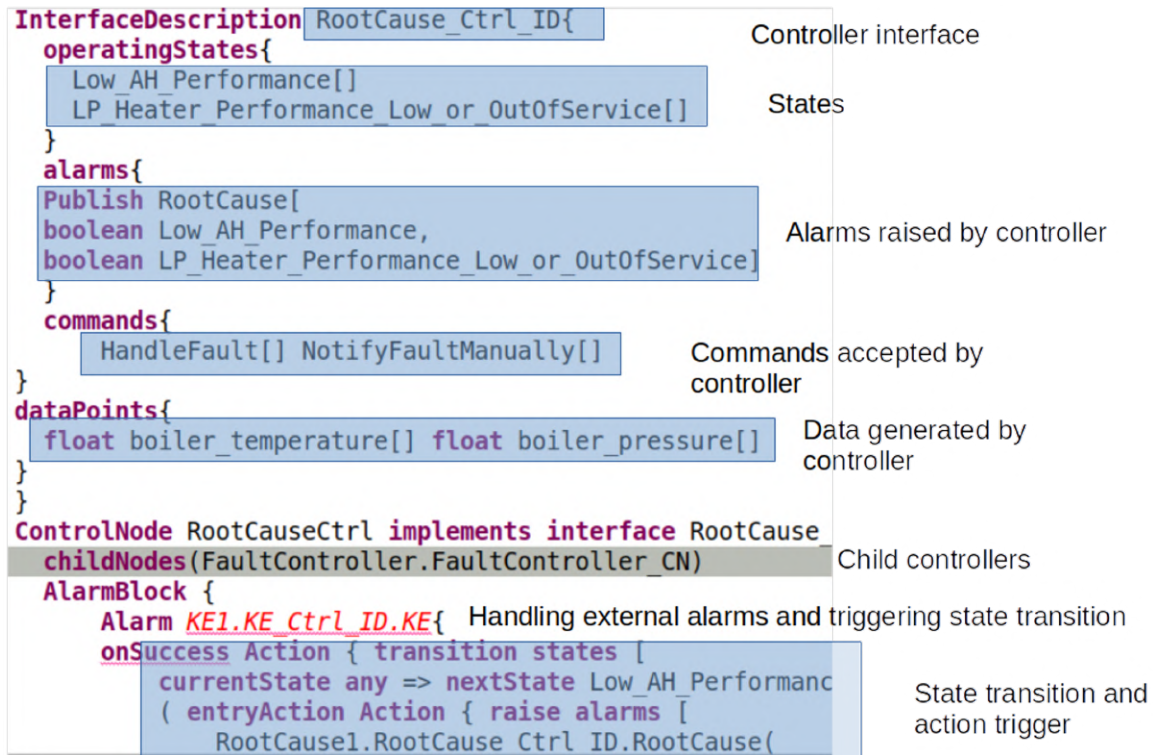


Figure 6.3: Representing a Controller with M&CML DSL

Several frameworks support the creation of DSLs, such as XText [108] and JetBrains' Metaprogramming [109], but the actual syntax and semantics are defined manually by DSL creators.

6.3.1 M&CML as a Domain-Specific Language

M&CML (Monitoring and Control Modeling Language) [110] is designed for the monitoring and control (M&C) domain. It captures the essential concepts and methodologies in real-time control systems, drawing from practical experience in large projects like ITER[111] and GMRT[71].

M&CML addresses the complexity of defining controllers in real-time systems by offering a focused language that models control systems' core functions. It includes components such as commands, events, data streams, and message processing, making it easier to design and analyze controllers for monitoring and control.

Figure 6.3 illustrates how M&CML represents a controller. It provides a standardized format that encapsulates control system attributes, helping automate the design process within K-IDE.

```

ActivityDiagram
Smart_Meeting_Task_Flow
has activities
{
  Activity Set_Up_Video{
    requireCapability : Video_Con
    conditions {
from Video_Con.Video_ID.success
if outcome
Video_Con.Video_ID.success.flag
is ( = (true))
=> nextActivity: Set_Up_Audio
}
},
  Activity Set_Up_Audio{
    requireCapability : Audio_Con
    conditions {
from Audio_Con.Audio_ID.success
if outcome
Audio_Con.Audio_ID.success.flag
is ( = (true))
=> nextActivity: Set_Up_Audio
}}}

```

(a) Smart Meeting Room Workflow with Activity DSL

```

Capability Video_Con compatible
component interface Video_Com.Video_ID {
  Init {
    fire Commands [
      Video_Com.Video_ID.start()
      responses=>{Video_Com.Video_ID.success},
      Video_Com.Video_ID.connect()
      responses=> {Video_Com.Video_ID.success}
    ]
    providesControlCapabilities {
      fireable commands :
      Video_Com.Video_ID.connect ,
      Video_Com.Video_ID.pause
    }
    providesOutcomes {
      receivable responses
      Video_Com.Video_ID.success,
      Video_Com.Video_ID.fail }}
}

```

Response Handling Knowledge ←

→ **Capability Implementation**

(b) Video Conferencing Device Capabilities with Capability DSL

6.3.2 Activity DSL: Linking Steps in Use Case Scenarios

Activity DSL is used to define workflows in control systems. It breaks down complex activities into tasks that can be aligned with the capabilities of different components. This DSL is designed to be accessible to business users, allowing them to specify workflows using a structured format.

Figure 6.4a shows how Activity DSL models a workflow in a smart meeting room. The workflow is mapped to the capabilities of various devices, making it easier to manage and automate control systems.

6.3.3 Capability DSL: Cataloging Component Capabilities

Capability DSL [88] provides a structured way to describe device capabilities in detail. It covers functionality, interfaces, interactions, and context assumptions, allowing for dynamic composition across devices from different providers. This DSL captures everything from physical usage to configuration parameters, making it a comprehensive tool for defining device behavior.

Figure 6.4b illustrates how Capability DSL represents the knowledge of a video conferencing device. This structured approach supports dynamic reconfiguration and enables precise control over system composition.

6.3.4 Conclusion

These three DSLs, M&CML, Activity DSL, and Capability DSL, are central to K-IDE's ability to automate control software development. Standardizing the representation of control designs, workflows, and device capabilities enables K-IDE to streamline the development process, reducing complexity and improving accuracy in software engineering.

6.4 Key Features

The Knowledge-aided Integrated Development Environment (K-IDE) offers a range of features designed to simplify and enhance control software development. These capabilities help software developers and domain experts address complex challenges efficiently.

6.4.1 Automated Control Design

K-IDE automates the process of selecting and integrating devices based on user-defined objectives. Using domain-specific languages (DSLs) and Capability Ontology [88], K-IDE reduces manual effort and ensures accurate design (Figure 6.5).

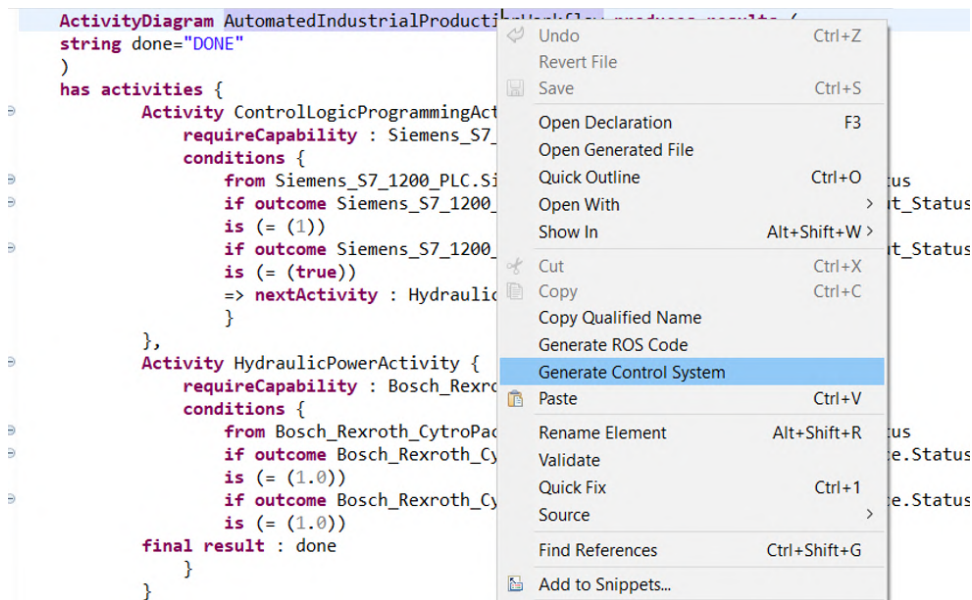


Figure 6.5: Automated Control System Design

6.4.2 Semantic Code Generation

K-IDE introduces a new approach to code generation by using a custom DSL that maps semantic entities to specific code templates [112]. This approach promotes knowledge reuse and adaptability, ensuring that domain knowledge is embedded in the generated code (Figure 6.6). The following chapter presents the details of the language used for code generation.

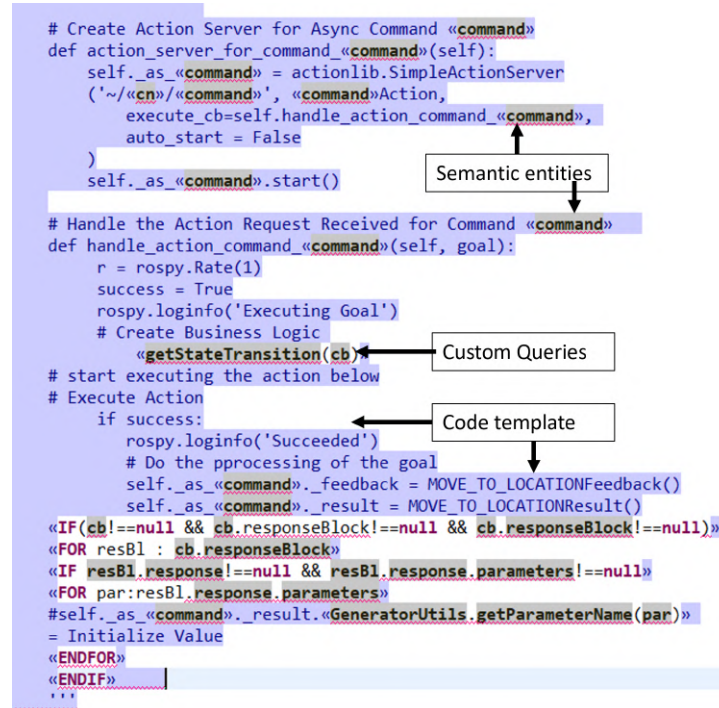


Figure 6.6: Template for Semantic Code Generation

6.4.3 Design Validation

K-IDE includes automated design validation that uses semantic checks to ensure the integrity of the control designs. This feature provides real-time feedback, helping to identify and resolve errors at design-time in the process. An example of the domain-specific validation in K-IDE is shown in Figure 6.7.

6.4.4 Design Visualization

K-IDE also offers the ability to create visual representations of control designs. These diagrams help users better understand complex systems and communicate their structure effectively (Figure 6.8).

```

4 Command AutomatedIndustrialProductionWorkflow.AutomatedIndustrialProductionWorkflow.INIT
5 {
6 Action {
7 }
8 }
9 Command AutomatedIndustrialProductionWorkflow.AutomatedIndustrialProductionWorkflow.Set_Pass
0 {
1 Action {
2 fire com
3 [
4 Bosch_Rexroth_CytroPac.Bosch_Rexroth_CytroPac_Interface.Set_Flow ( ) -> expected expectedResponse Bosch
5 Validate
6 { parameter Bosch_Rexroth_CytroPac.Bosch_Rexroth_CytroPac_Interface.Status.flow
7 [ Possible Values = ( 1 ) ] parameter Bosch_Rexroth_CytroPac.Bosch_Rexroth_CytroPac_Interface.Statu
8 [ Possible Values = ( 1 ) ]
9 onSuccess Action {
0 transition states [
1 currentState AutomatedIndustrialProductionWorkflow.AutomatedIndustrialProductionWorkflow.Hy

```

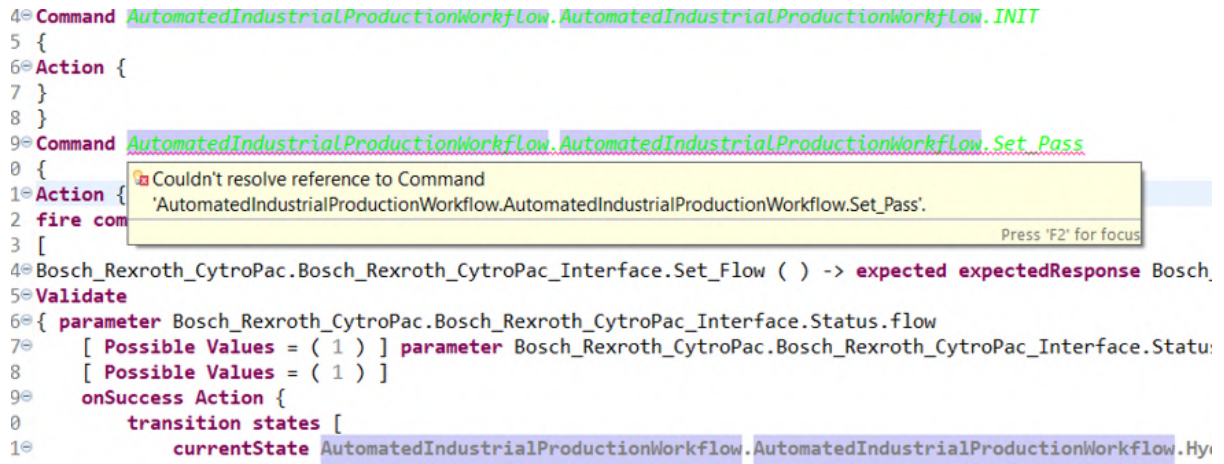


Figure 6.7: Domain-Specific Validation in K-IDE

6.4.5 Integration of Knowledge-Driven and Model-Driven Approaches

K-IDE combines knowledge-driven and model-driven strategies to provide a robust platform for software development. Domain-Specific Languages (DSLs) created using Xtext [108] import syntax and semantics directly from the knowledge base, ensuring that the most current domain knowledge is reflected. Visual models are generated using the Sirius framework [113], which translates DSL models into diagrams.

This integration of knowledge-driven and model-driven approaches enhances K-IDE's flexibility and adaptability, helping overcome the limitations of traditional methods. It allows developers to build software more efficiently and with greater precision, making K-IDE a powerful tool for control system development.

6.5 Learnings from K-IDE's Development

The development of the Knowledge-aided Integrated Development Environment (K-IDE) has provided valuable insights that shaped its evolution and success. Key lessons learned during this process include:

1. **Importance of Knowledge:** Structured domain knowledge is critical in efficient software development and high-quality code generation. However, the dynamic nature of software development demands ongoing efforts to keep this knowledge accurate and relevant.
2. **Value in Integration:** Combining knowledge-driven and model-driven approaches has improved software development. However, managing differences between these methodologies, such as conflicts in approach or assumptions, is a persistent challenge.



Figure 6.8: Visual Representation of Control Design

3. **Limits of Automation:** While automation reduces time and effort, defining high-level objectives and requirements still requires human input. The effectiveness of the generated code depends on the quality of the initial inputs provided by developers.
4. **Need for Validation:** Robust validation mechanisms are essential to ensure the quality of auto-generated code, especially given the potential risks associated with automation.
5. **Adaptability is Key:** The variety of use cases solved during K-IDE's evaluation highlighted the importance of adaptability. The system's ability to meet diverse requirements emphasizes the need for flexible tools in software development.

These lessons have shaped the development of K-IDE and will guide future improvements. They also offer valuable insights for other projects that aim to bridge the gap between academic research and practical software engineering applications.

6.6 Challenges and Opportunities: Insights from K-IDE's Development

The development of the Knowledge-aided Integrated Development Environment (K-IDE) presented several challenges and revealed potential opportunities. The main difficulties and their associated possibilities are outlined below:

1. **Methodology Transition:** Adopting knowledge-based systems like K-IDE can be difficult, as it requires shifting from established workflows. However, demonstrating K-IDE's practical benefits can encourage more efficient development practices.
2. **Market Entry:** Introducing K-IDE into a market with existing tools is challenging. However, K-IDE's specific capabilities for streamlining processes can offer advantages for organizations seeking improvements.
3. **Learning Curve:** K-IDE's advanced features may seem complex to engineers unfamiliar with knowledge-driven tools. This creates the need for effective training and well-structured documentation to simplify adoption.
4. **Scalability:** Ensuring K-IDE performs well as project complexity increases is a challenge that requires continuous refinement. Feedback from real-world applications will be key to improving K-IDE's ability to handle large-scale projects.
5. **Customization and Usability:** Balancing the need for project-specific customization with ease of use is essential. Developing user-friendly customization options can help make K-IDE more adaptable to various project needs.

K-IDE's integration of knowledge-driven and model-driven approaches offers opportunities for improving software development across different domains. These challenges and opportunities provide valuable direction for further development and refinement of the K-IDE platform.

6.7 Conclusion and Future Prospects

The Knowledge-aided Integrated Development Environment (K-IDE) has demonstrated its capability to enhance software development by integrating knowledge-driven and model-driven methodologies. As discussed in previous chapters, this approach has proven effective across various applications, particularly with the use of domain-specific languages (DSLs) to guide automated code generation.

Throughout the development of K-IDE, the importance of maintaining precise domain knowledge and balancing different methodologies has become clear. These elements are central to K-IDE's design and functionality. As K-IDE continues to evolve, its adaptability to changing software development trends will be crucial.

Future work will focus on refining K-IDE based on insights gained from this research, expanding its scope, and strengthening validation mechanisms to ensure reliability. These efforts will build on the foundations laid in this thesis, further supporting the theme of knowledge-driven approaches in control software development.

The ultimate aim is to establish K-IDE as a reliable tool that improves efficiency in modern software development and supports the creation of intelligent, adaptable systems.

The following chapter presents a specialized knowledge representation language that serves as a bridge between domain knowledge and software development. This language is essential for translating abstract concepts into concrete software components and forms a key part of K-IDE's framework. It will provide a deeper understanding of how K-IDE connects theoretical knowledge with practical development, continuing the exploration of knowledge-driven approaches presented in this thesis.

Chapter 7

Knowledge Representation Language

The preceding chapter highlighted the necessity for a specialized knowledge representation language, acting as an intermediate step between abstract domain knowledge and practical software development within the Knowledge-aided Integrated Development Environment (K-IDE) framework. This imperative lays the foundation for the current chapter's exploration into the broader context of knowledge representation for software development.

The digital age has led to an unprecedented data generation and consumption explosion. According to [114], the global datasphere is expected to grow to 175 zettabytes by 2025, a tenfold increase from 2010. This exponential growth necessitates effective management and representation of information, particularly in the context of knowledge representation across diverse formats.

7.1 Importance of Knowledge Representation

Knowledge representation is fundamental to intelligent systems, enabling the encoding, processing, and utilization of complex information [115]. In software engineering and data management, representing knowledge in a manner that various systems and stakeholders can readily consume presents a significant challenge. Traditional methods often involve manual translation and data transformation, a labor-intensive process that is prone to inconsistencies and inefficiencies [115].

The challenge manifests across various industries, each with unique complexities and requirements, such as IT/software, healthcare, finance, and manufacturing. Inconsistent and inefficient knowledge representation leads to potential system failures, hindering the delivery of personalized services, complicating regulatory compliance, and creating inefficiencies in supply chain operations [116, 117, 118, 119].

7.1.1 Challenges and Needs

The problem can be articulated through key challenges like fragmentation, custom solutions, knowledge utilization, scalability, and flexibility [116, 120, 121, 122]. Moving towards an increasingly data-driven future, the importance of efficient and flexible knowledge representation will only grow. The urgency and relevance, economic impact, technological advancements, scalability and adaptability, social and ethical considerations, and innovation potential are elaborated on [123, 124, 125, 126, 127, 128].

7.1.2 Connecting with Knowledge-Driven Composition and K-IDE

Our approach offers a viable solution to these multifaceted challenges, providing a scalable and adaptable framework that aligns with the evolution of software systems and data management. A core part of this solution is the integration of knowledge-driven composition techniques and developing K-IDE, building upon the need for an intermediate knowledge representation language discussed earlier.

The subsequent chapters will detail how our novel domain-specific language (DSL) leverages Semantic Knowledge Graphs (SKGs) to address the challenges outlined above, specifically focusing on the composition techniques and the role of K-IDE in bridging the gap between data and actionable knowledge. This connection between knowledge representation and knowledge-driven composition will be instrumental in unlocking new potentials and driving progress across domains.

While model-driven approaches provide structural abstractions, they often lack explicit semantic representations required for reuse, reasoning, and dynamic reconfiguration. In contrast, this thesis treats knowledge representation as a first-class concern, where device capabilities, behavioral constraints, and composition semantics are explicitly modeled and queried.

The proposed knowledge representation enables explainable synthesis and controlled reuse, which are essential in safety-critical control software contexts where purely data-driven or black-box approaches are unsuitable.

7.2 Related Work

Knowledge representation has evolved through various paradigms, each attempting to address the inherent challenges of encoding, processing, and utilizing complex information. This section reviews key approaches and identifies gaps that motivate our work.

7.2.1 Rule-Based Systems

Rule-based systems, such as expert systems, were among the earliest attempts to formalize human-like reasoning in machines [129]. Representing knowledge in the form of IF-THEN rules was a significant step towards automating decision-making processes. However, their rigidity made them unsuitable for handling complex or ambiguous situations, reflecting the philosophical challenge of capturing the fluidity of human thought in static rules [130].

7.2.2 Semantic Networks

The limitations of rule-based systems led to the development of semantic networks, representing knowledge as a graph of concepts connected by relationships [131]. This approach acknowledged the interconnected nature of knowledge but struggled with scalability and consistency, illustrating the difficulty of maintaining coherence in a complex web of relationships [132].

7.2.3 Ontologies

Ontologies emerged as a response to the need for a more formal and structured approach to knowledge representation [133]. By explicitly specifying a shared conceptualization, ontologies bridged the gap between human understanding and machine processing [134]. However, their focus on defining concepts and relationships left a gap in generating diverse target representations, reflecting the broader challenge of translating abstract concepts into concrete forms [135].

7.2.4 Semantic Knowledge Graphs

Semantic Knowledge Graphs (SKGs) extended ontologies by incorporating inference rules and constraints, offering a more expressive framework [136]. Despite their potential, the complexity of building and maintaining SKGs and the lack of standardized tools for translation into various target representations remain barriers, highlighting the ongoing struggle to balance complexity and usability in knowledge representation [137].

7.2.5 Model-Driven Code Generators

Model-driven code generators enable automatic code generation from high-level models, offering both efficiency and standardization. Key tools and approaches include:

- **Eclipse Modeling Framework (EMF):** A framework for building applications based on structured data models, with challenges in handling complex requirements [103, 138].

- **MATLAB Simulink:** Used for modeling, simulating, and analyzing dynamic systems, with potential issues in translating graphical models into efficient code [139, 140].
- **Model-Driven Architecture (MDA):** A design approach using UML, promoting platform independence but possibly leading to loss of control over generated code's performance [141, 142].
- **Domain-Specific Modeling (DSM):** Tools like MetaEdit+ allow for domain-specific languages and code generation but may have limited applicability outside targeted domains [143, 144].

While these approaches have advanced automation in software development, they often struggle with flexibility, adaptability, and efficient translation of high-level abstractions. The tension between automation and customization continues to be a challenge in model-driven code generation [145].

7.2.6 Large Language Models (LLMs)

Large Language Models (LLMs), such as transformer-based architectures, have emerged as a powerful tool in natural language processing, with applications in knowledge representation [146]. By leveraging large-scale data and deep learning techniques, LLMs offer a data-driven approach to capture intricate patterns and relationships within text. However, their challenges include a lack of interpretability, dependence on quality training data, computational complexity, and limitations in translating knowledge into diverse target representations [147, 148, 149, 150].

7.2.7 Comparison and Gaps

The following comparison highlights the philosophical underpinnings of the challenges in knowledge representation and identifies major gaps:

- **Flexibility:** The tension between structure and flexibility recurs across paradigms, with rule-based systems and model-driven code generators emphasizing structure and semantic networks, ontologies, and SKGs prioritizing flexibility.
- **Scalability:** Managing complexity and scaling from simple to complex systems remains an overarching challenge.
- **Consistency:** Ensuring consistency across different representations manifests the philosophical challenge of maintaining coherence.

- **Manual Effort:** The reliance on manual effort underscores the ongoing struggle to automate processes requiring human-like understanding.

In summary, the evolution of knowledge representation reflects a series of attempts to grapple with fundamental philosophical challenges, such as the tension between structure and flexibility, the management of complexity, the maintenance of coherence, and the automation of human-like understanding. The current state-of-the-art approaches, while significant, fall short of addressing these challenges. Introducing a novel, template-based language that leverages the inherent structure and semantic richness of SKGs aims to provide a scalable and adaptable solution to this longstanding challenge, bridging the gap between the philosophical ideals of knowledge representation and the practical needs of modern software systems.

7.3 Our Approach

Our approach to the complex problem of generating diverse target representations from a single Semantic Knowledge Graph (SKG) is rooted in the development of a novel, domain-specific language (DSL)[104] called Knowledge Representation Language (KRL). This is an external DSL[151], designed to leverage the inherent structure and semantic richness of SKGs, enabling the creation of templates that can convert semantic elements into various target representations, including visual, textual, or code formats. A high-level architecture of the approach is shown below in Figure 7.1

7.3.1 Architecture for Knowledge Representation and Querying

The diagram illustrates a comprehensive architecture that facilitates integrating, transforming, and querying data from diverse sources into a unified knowledge base. This architecture leverages various technologies and methodologies to achieve a seamless flow of information from raw data to a serialized final representation. Below are the key steps and components depicted in the diagram:

Steps in the Process:

1. **Data Fetching:** Data is gathered from multiple sources, from relational databases to NoSQL databases to search engines. This step ensures that a wide variety of information is accessible for processing.

2. **Graph Transformation:** The collected data is transformed into a neutral form of graph and stored in GraphDB¹. This graph structure allows for a flexible and efficient representation of relationships and entities.
3. **Knowledge Base Formation:** The graph in GraphDB serves as a knowledge base, encapsulating the structured information in a form that facilitates complex querying and analysis.
4. **KRL Description:** Users can describe the desired representation of the knowledge using KRL (Knowledge Representation Language). KRL provides a domain-specific syntax for defining the structure and semantics of the knowledge base.
5. **SPARQL Query Conversion:** The KRL description is systematically converted into SPARQL[152] queries. SPARQL, a standard query language for RDF (Resource Description Framework) data, enables precise querying of the graph-based knowledge base.
6. **Query Execution:** The SPARQL queries are executed on GraphDB, retrieving the relevant information based on the user's KRL description.
7. **Result Assembly:** An assembler component gathers the results of the SPARQL queries, organizing them into a coherent structure.
8. **Final Representation and Serialization:** The assembled results are built into a final representation and serialized. This step produces a well-structured output that can be utilized for various applications.

Key Components:

- **Data Sources:** These are the origin points of the raw data, including various types of databases and search engines.
- **GraphDB:** GraphDB is a specialized semantic data store that handles graph-based data structures.
- **KRL:** Knowledge Representation Language (KRL) is a domain-specific language used to articulate the structure of the knowledge base.
- **SPARQL:** SPARQL is the query language used to interrogate the RDF data within GraphDB.
- **Assembler:** The assembler is a vital component that stitches together the query results into a unified final representation.

¹GraphDB is a highly efficient and scalable RDF database developed by Ontotext. It supports SPARQL querying and allows for seamless integration with various data sources. More information can be found at <https://www.ontotext.com/products/graphdb/>

The diagram encapsulates a robust, flexible architecture that marries traditional data sources with modern graph-based technologies. Employing KRL and SPARQL ensures that users can easily and precisely define and query complex knowledge structures. The entire process underscores the potential of semantic technologies in handling diverse and intricate data landscapes.

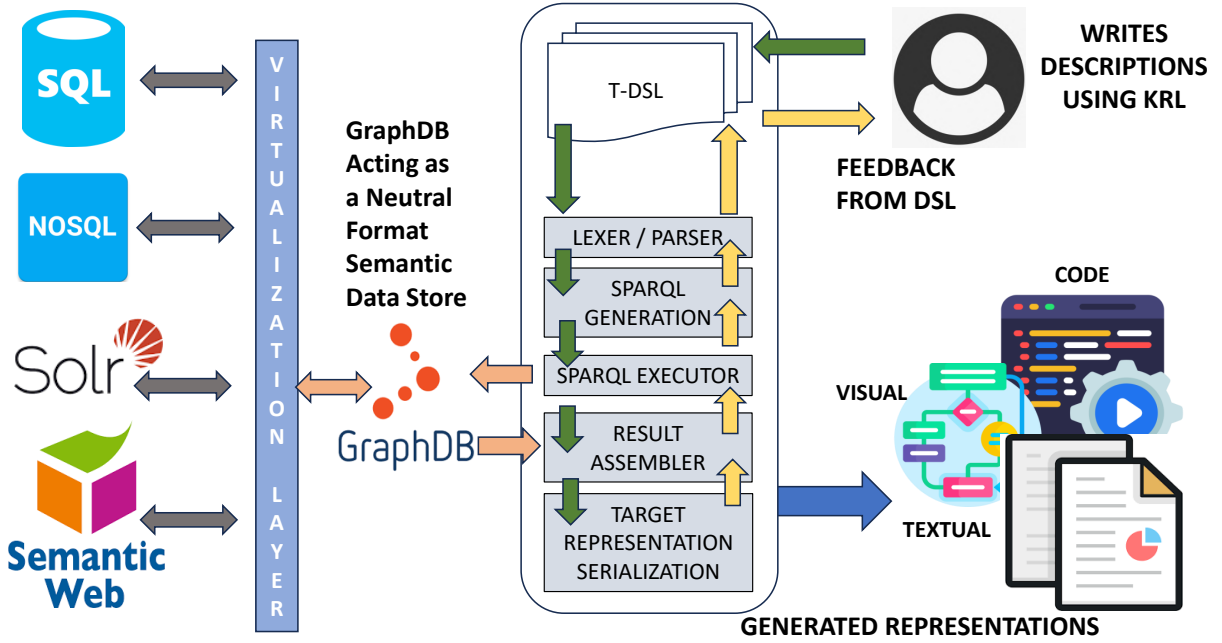


Figure 7.1: Architecture of the KRL, illustrating the constructs for defining templates and their integration with semantic elements in the SKG.

The architecture encapsulates a robust, flexible framework that marries traditional data sources with modern graph-based technologies. Employing KRL and SPARQL ensures that users can quickly and precisely define and query complex knowledge structures. The entire process underscores the potential of semantic technologies in handling diverse and intricate data landscapes, offering a scalable and adaptable solution for managing complex data transformation tasks. Integrating semantic technologies SPARQL further enhances the system's ability to represent and manipulate knowledge, paving the way for more intelligent and coherent data-driven applications.

7.3.2 Design of the Template-Based Language

The design of the KRL is guided by principles essential to DSLs[104, 143, 151]: flexibility, scalability, and consistency. It is crafted to accommodate various target representations, handle complex SKGs, and ensure accurate representation across different targets. The language consists of constructs that allow users to define templates for different types of semantic elements in the SKG, such as entities, relationships, and attributes. Each template specifies how a

particular type of semantic element should be represented in the target representation, demonstrating the language’s adaptability and precision.

7.3.3 Integration with Semantic Knowledge Graphs

The KRL integrates seamlessly with Semantic Knowledge Graphs[153] (SKGs), employing constructs for querying and retrieving semantic elements. This dynamic integration allows the templates to generate target representations based on the SKG’s current state, reflecting the language’s ability to interact with complex data structures and provide real-time solutions.

KRL provides a rich set of query constructs that enable users to interact with SKGs. These constructs allow retrieving specific entities, relationships, and attributes within the SKG. For example, a user can query for all products within a particular category or retrieve the relationships between different authors in a bibliographic database. Consider a medical SKG that contains information about diseases, symptoms, treatments, and medical professionals. A KRL query can be constructed to retrieve all treatments related to a specific disease, such as diabetes. This query would interact with the SKG to obtain the latest information, ensuring that the resulting representation includes the most recent medical advancements. An example of a query construct storing queries as function extensions is shown below in Figure 7.2

```
Ask getSolution: select m where m is a SolutionSpecification.
Ask getDomainModel : "select ?m where {?1 <for_domain_model> ?m}".
Ask getLearningModel : "select ?m where {?1 <create_solution_model> ?m}".
Ask getDomainDataSource : "select ?d where {?1 <data_source> ?d}".
Ask getCSVDataPath : "select ?p where {?1 <csv_path> ?p}".
Ask getCSVDataFileName : "select ?n where {?1 <csv_file_name> ?n}".
Ask getDomainFeatures: "select ?f where {?1 <features> ?f}".
Ask getSolutionMLModel : "select ?sm where {?1 <create_solution_model> ?sm}".
Ask getHypothesis : "select ?h where {?1 <uses_hypothesis> ?h}".
Ask getInputFeatures : "select ?f where {?1 <input_variables> ?f}".
Ask getFeatureColumnMapping: "select ?c where {?1 <maps_to> ?c}".
Ask getOutFeatures: "select ?c where {?1 <output_variables> ?c}".
Ask getExperience: "select ?e where {?1 <gains_experience> ?e}".
Ask getFeatureScalingAlgo: "select ?a where {?1 <perform_feature_scaling> ?a}".
Ask getScript : "select ?s where {?1 <algo_script> ?s}".
Ask getFeatureLearningAlgo: "select ?a where {?1 <implement_feature_learning_algorithm> ?a}".
Ask getAlpha : "select ?a where {?1 <alpha> ?a}".
Ask getIters : "select ?i where {?1 <iterations> ?i}".
Ask getParams : "select ?p where {?1 <parameters> ?p}".
Ask getInputFeatureCount : "select count(?p) where {?1 <input_variables> ?p}".
Ask getFeatureValue : "select ?v where {?1 <feature_value> ?v}".
Ask getAllScripts: select s where m has algo_script s.
Ask getScriptLocation: "select ?l where {?1 <location> ?l}".
```

Figure 7.2: Query Construct of KRL

Integrating KRL with Semantic Knowledge Graphs (SKGs) enables real-time, dynamic generation of target representations such as visualizations, textual descriptions, or code snippets. Templates in KRL can adapt to changes in the SKG, reflecting the current state of the knowledge graph. For example, a KRL template can automatically update a product catalog in a re-

tail SKG without manual intervention when a new product is added. Similarly, a KRL-based dashboard can instantly reflect significant market changes in a financial SKG tracking stock market data, providing investors with up-to-the-minute insights. Figure 7.3 shows a snippet using KRL to represent an ML data pipeline.

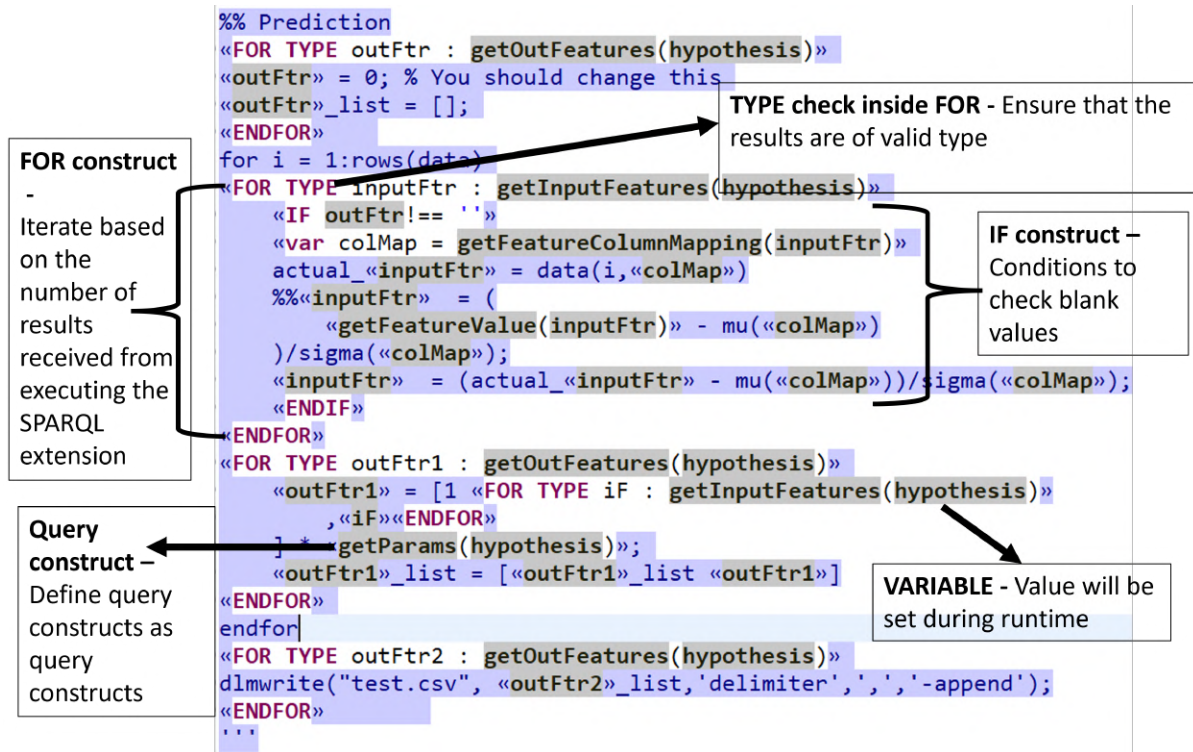


Figure 7.3: KRL - Language Constructs to Represent an ML Pipeline in Octave[154]

The ability to interact with SKGs also highlights KRL's capability to handle complex data structures. SKGs often contain intricate relationships and hierarchies, and KRL's constructs are designed to navigate these complexities efficiently. In a research SKG containing information about scientific papers, authors, institutions, and citations, a KRL query can be used to analyze collaboration networks between researchers. This would involve traversing multiple layers of relationships and aggregating data from different parts of the SKG.

Finally, integrating KRL with SKGs offers a powerful and flexible approach to knowledge representation. By enabling dynamic querying and generation of target representations, KRL provides a robust solution for interacting with complex data structures and delivering real-time, adaptable solutions.

7.3.4 Knowledge-Driven Paradigm

Our approach introduces a knowledge-driven paradigm[88, 90, 155] that emphasizes the central role of a unified knowledge base in generating diverse target representations. This

paradigm is not merely a theoretical construct but a practical framework that enhances the reusability of knowledge and minimizes redundancy.

- **Unified Knowledge Base:** The KRL operates on multiple connected Semantic Knowledge Graphs (SKG), ensuring the same underlying knowledge base is utilized across various target representations. This unification promotes consistency and coherence in the generated outputs.
- **Dynamic Generation:** The knowledge-driven paradigm allows for the dynamic generation of target representations based on real-time changes in the SKG. The generated representations immediately reflect any modifications to the knowledge base, ensuring up-to-date and accurate outputs.
- **Interoperability:** The paradigm facilitates interoperability between target representations, enabling seamless translation and transformation between visual, textual, or code formats. This interoperability is achieved through the flexible design of the KRL and its ability to handle complex semantic structures.
- **Scalability:** The knowledge-driven approach is scalable and can handle large and intricate SKGs. It can adapt to the growing complexity of knowledge representation, making it suitable for various domains and applications.
- **Integration with Existing Systems:** The paradigm allows integration with existing knowledge representation systems, bridging traditional rule-based and modern data-driven approaches. This integration enhances the overall robustness and flexibility of the knowledge representation process.
- **Sustainability:** Knowledge reuse contributes to the long-term sustainability of knowledge bases. By building on existing knowledge rather than continually recreating it, KRL promotes a more sustainable development process less prone to redundancy and obsolescence.

The knowledge-driven paradigm represents a significant advancement in domain-specific languages, offering a comprehensive and adaptable framework for knowledge representation. By leveraging the structure and semantic richness of SKGs, the paradigm enables efficient and consistent generation of diverse target representations, paving the way for more intelligent and coherent data-driven applications.

7.4 KRL Grammar

The language grammar includes constructs for defining templates, loops, conditional statements, expressions, and rich strings[156], among other features. This section provides a de-

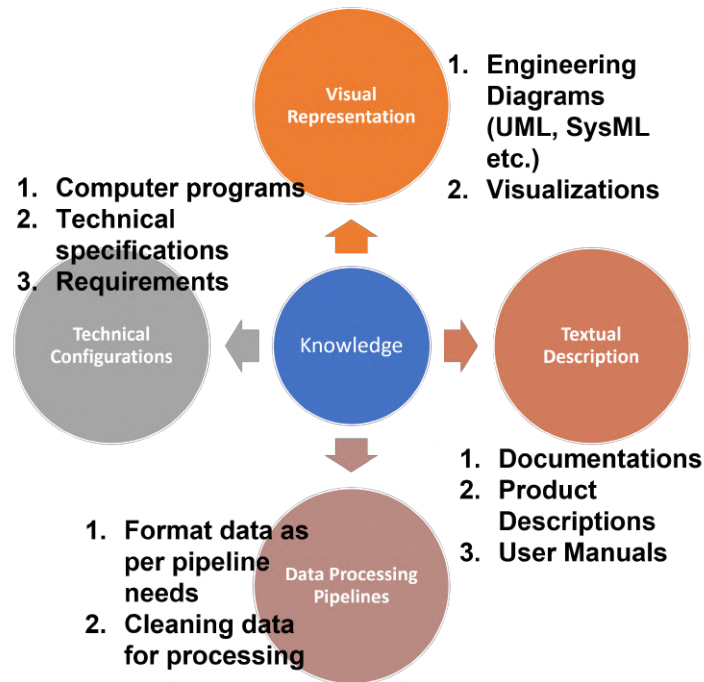


Figure 7.4: The knowledge-driven paradigm illustrating the generation of multiple target representations and the reflection of changes in the knowledge base.

tailed overview of the language's key constructs. The grammar is organized into several key components, each serving a specific role in defining the language's structure and functionality.

- **Template Definitions:** The `Template` rule defines the structure of templates in the language, allowing for optional `Forall` constructs and multiple `Body` elements.
- **Forall Construct:** The `Forall` rule provides a way to declare variables and expressions for universal quantification, followed by a generation statement.
- **Body Elements:** The `Body` rule defines the content of a template body, including optional expression formatters and extensions.
- **Rich Strings:** A series of rules define the structure of rich strings, allowing for literals, loops, translation extensions, append expressions, and copy file expressions.
- **Translation Types:** The `TranslationType` rule defines the types of translations supported, including rules, equations, external equations, and expressions.
- **Expressions:** A comprehensive set of rules defines the expression language, including variable expressions, if statements, unary and binary operations, literals, closures, and feature calls.

- **Type References:** The grammar includes rules for defining type references, including parameterized types, array brackets, and argument type references.

7.4.1 Lexer and Parser

The lexer[157] and parser[158] are essential components in the translation of source code into an Abstract Syntax Tree (AST)[159]. They work together to analyze the syntax of the code, with the lexer breaking down the input into tokens and the parser organizing these tokens into the hierarchical structure of the AST. The grammar includes several terminal rules that define the linguistic elements of the language, such as hexadecimal digits, rich text delimiters, numbers, and strings. These terminals provide the foundational building blocks for parsing the language's syntax.

7.4.1.1 Lexer

The lexer is responsible for scanning the source code and breaking it into tokens. Each token represents a basic building block of the language, such as a keyword, identifier, literal, or operator.

- **Keywords:** Keywords are reserved words with special language meanings. Examples include 'FORALL', 'Body', and 'Extension'.
- **Identifiers:** Identifiers are used to name variables, functions, and other entities. The grammar defines them by the ID rule.
- **Literals:** Literals represent constant values, such as strings, numbers, and booleans. The grammar includes rules for different types of literals, such as STRING, Number, and XBooleanLiteral.
- **Operators:** Operators are symbols that represent operations, such as addition, subtraction, and comparison. The grammar defines various operators, including '+=', '-=', and '*='.
- **Rich Text Handling:** The lexer includes special rules for handling rich text, with different tokens for the start, end, and in-between parts of a rich string.

The lexer's rules are defined using regular expressions and specify the patterns that match each token type.

7.4.1.2 Parser

The parser takes the sequence of tokens produced by the lexer and organizes them into the hierarchical structure of the AST. It does this by applying the grammar rules defining the language's syntax.

- **Template Parsing:** The parser constructs a `Template` node for each template definition, including its name, optional `Forall` construct, and bodies.
- **Expression Parsing:** The parser constructs nodes for various types of expressions, including additive, multiplicative, unary, and relational expressions, based on the corresponding rules in the grammar.
- **Rich String Parsing:** The parser constructs nodes for rich strings, including literals and expressions, based on the rich string rules in the grammar.
- **Control Flow Parsing:** The parser constructs nodes for loops and conditional statements, allowing for complex control flow within templates.

The parser ensures that the tokens conform to the language's syntax and constructs the AST to reflect the hierarchical nature of the language's constructs.

The lexer and parser play a crucial role in translating the descriptions into the structured representation of the AST. By understanding how they work and how they apply the grammar rules, one can gain insights into how the language is parsed and how its syntax is translated into the internal structure of the AST.

7.4.2 Abstract Syntax Tree (AST)

The AST is a hierarchical representation of the syntactic structure of a program. The parser constructs it and forms the basis for interpreting or compiling the program. Each node in the AST represents a construct in the language, such as a template definition, a loop, or an expression. Figure 7.5 below shows the AST using the Eclipse Modeling Framework (EMF)[103].

This diagram represents the AST for a template definition, including the name of the template, an optional `Forall` construct, and one or more `Body` nodes. The `RichString` node represents a rich string within a body, including literals and expressions.

7.4.3 Grammar Rules

The grammar of KRL is represented following EBNF[160] and implemented using Eclipse EMF[103], Xtext[161] and XBase[162] frameworks.

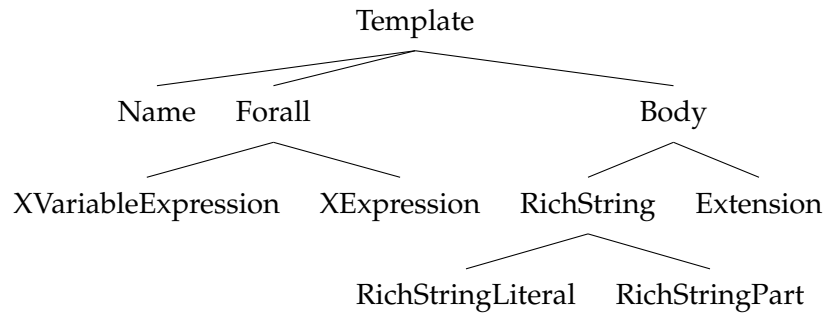


Figure 7.5: AST of the language

7.4.3.1 Template Definition

Templates are central to the language, allowing users to define reusable patterns for generating target representations.

```

Template:
'Name' name=ID '.'
(forAll = Forall)?
bodies+=Body*
;
  
```

A template includes a name, an optional `Forall` construct, and one or more bodies. The `Forall` construct allows for iteration over a collection, and the bodies define the template's content.

7.4.3.2 Rich String Handling

Rich strings are a powerful language feature, allowing for embedding expressions within strings.

```

RichString:
{RichString} (
  expressions+=RichStringLiteral |
  expressions+=RichStringLiteralStart expressions+=RichStringPart?
  (expressions+=RichStringLiteralInbetween expressions+=RichStringPart?)*
  expressions+=RichStringLiteralEnd
)
;
  
```

This rule defines a rich string as a sequence of literals and expressions, with different rules for the string's start, end, and in-between parts. Rich strings enable dynamic content generation within strings.

7.4.3.3 Expressions and Operations

The language includes a comprehensive set of expressions and operations, including arithmetic, relational, logical, and unary operations.

```
XAdditiveExpression:  
XMultiplicativeExpression (=>({XBinaryOperation.leftOperand=current} feature=OpAdd)  
rightOperand=XMultiplicativeExpression)*;
```

This rule defines an additive expression as a sequence of multiplicative expressions combined with addition or subtraction operators. Similar rules define other types of expressions and operations.

7.4.3.4 Loops and Conditional Statements

The language includes constructs for loops and conditional statements, allowing for complex control flow.

```
RichStringForLoop:  
{RichStringForLoop}  
"FOR" (types?="TYPE")?  
(declaredParam+=XVariableExpression(',' declaredParam+=XVariableExpression)?)  
' :' forExpression=XExpression  
eachExpression=InternalRichString  
"ENDFOR"  
;
```

This rule defines a FOR loop that iterates over a collection, executing the eachExpression for each element.

7.4.4 Language Features

The KRL grammar, defined using the S2M DSL, allows modeling the following features:

- **Type Safety Inside a FOR Loop:** The `RichStringForLoop` construct ensures type safety within the FOR loop, allowing optional type declarations for variables. This enhances code robustness by preventing type-related errors.
- **Conditional Logic with IF Statements:** The `RichStringIf` construct enables conditional execution based on expressions, allowing dynamic generation of content based on the current state of the Semantic Knowledge Graph (SKG).
- **Template-Based Code Generation:** The grammar supports rich string handling, allowing the creation of templates with placeholders and expressions. This facilitates dynamic code generation and the creation of reusable semantic structures.
- **Integration with External Libraries:** Though not explicitly defined in the grammar, the language's extensibility can be leveraged to integrate with external libraries, enhancing its applicability in various domains.
- **Rich Translation Extensions:** The `RichTranslationExtension` construct allows integration with translation types such as Rule, Equation, ExternEquation, and Expression, enabling the language to support translations between different formats.
- **Modularity and Extensibility:** The grammar is designed with separate constructs for different parts of the language, such as loops, conditionals, and expressions. This modular design allows easy extension or modification to suit specific needs.
- **Rich String Handling:** The grammar includes constructs for handling rich strings with various literals and parts, supporting the creation of human-readable representations.
- **File Operations:** The `AppendExpression` and `CopyFileExpression` constructs allow file manipulation, such as appending strings to expressions and copying files from one expression to another.
- **Expression Handling:** The grammar supports various expressions, including binary operations, unary operations, assignments, and literals. This provides flexibility in defining complex logic and mathematical operations.
- **Variable Handling:** The `XVariableExpression` construct allows variable declarations and expressions, supporting dynamic content generation based on variable values.

7.5 Comparison with Related Approaches

The proposed language presents a distinctive method for knowledge representation and translation, seamlessly blending semantic knowledge bases with the capacity to formulate diverse target representations. Although devoid of direct analogs for this particular integration,

examinations of comparisons with related fields and technologies underscore the uniqueness and benefits of this approach.

7.5.1 Model-Driven Engineering (MDE)

MDE uses models as primary artifacts, often transforming them into code or other representations. While sharing similarities in the use of models and transformations, our approach differs in several key aspects:

- **Semantic Integration:** Our language integrates with semantic knowledge bases, allowing for richer contextualization and dynamic updates.
- **Versatility:** Unlike MDE, which often targets specific platforms or languages, our approach enables the generation of diverse representations, including code, UML, documentation, and more.

7.5.2 Ontology-Based Data Access (OBDA)

OBDA aims to unify access to heterogeneous data sources through ontology mapping. While our approach also deals with knowledge representation, it extends beyond data access:

- **Knowledge Translation:** Our language represents knowledge and translates it into various consumable forms, enhancing applicability and value.
- **Customizable Templates:** The ability to define custom templates for different target representations offers flexibility and reusability.

7.5.3 Code Generation Tools

Various code generation tools exist, but our approach offers additional benefits:

- **Knowledge-Driven:** Unlike traditional code generators, our language leverages semantic knowledge, ensuring alignment with business logic and consistency across representations.
- **Cross-Domain Applicability:** The language's adaptability to various domains and its ability to generate diverse artifacts set it apart from specialized code-generation tools.

While related fields and technologies offer valuable insights, our approach presents a novel combination of semantic integration, versatility, knowledge translation, and customization. It addresses specific challenges related to data fragmentation, custom solutions, knowledge utilization, and scalability, positioning it as a viable solution for knowledge-driven development across diverse industrial contexts.

7.6 Examples: Representing Robotic Component Knowledge as an Executable Robot Control Script

The Knowledge Representation Language (KRL) is used in the given example to model a robotic control program using the S2M DSL. The code snippet provided is part of a larger system that leverages the Semantic Knowledge Graph (SKG) to store comprehensive knowledge about various robotic components, including their commands, event handlers, parameters, and response mechanisms.

The SKG serves as a rich repository of information, capturing the structure, semantics, and relationships between different elements of the robotic system. This knowledge is then accessed and manipulated using the S2M DSL, a domain-specific language designed to interact with the SKG, as defined by the grammar.

The snippet showcases how the device knowledge from the SKG is dynamically pulled and arranged into a template. This template is then used to auto-generate the ROS (Robot Operating System)[163] controllers for the robot. The generated code includes service servers for handling specific commands, subscribers for events, and conditional logic to process requests and responses.

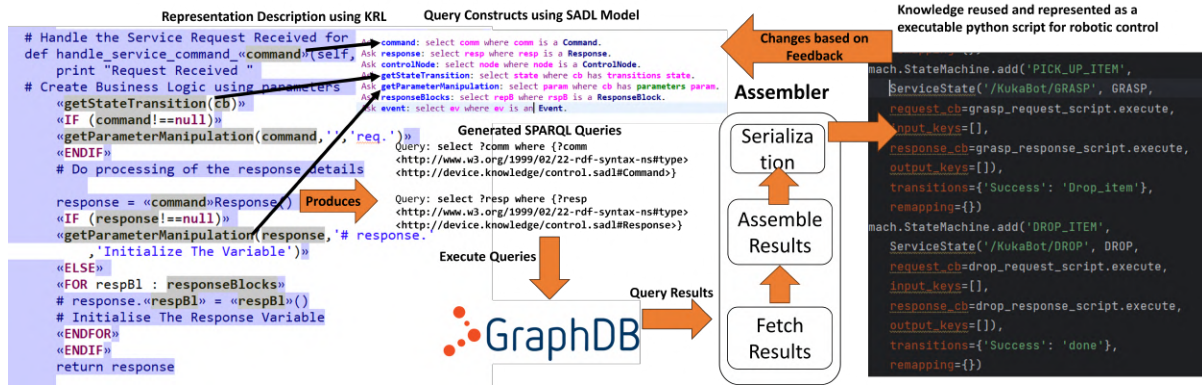


Figure 7.6: KRL Example: Representing Robotic Knowledge as a Robot Control Code

The integration of KRL with the SKG enables the creation of intelligent robotic control programs that are both flexible and adaptable. By utilizing the semantic richness of the SKG, the system can generate diverse target representations, ensuring consistent and efficient control of robotic components.

The given KRL snippet highlights the language's capabilities in ensuring type safety through constructs like the `RichStringForLoop`, enabling conditional logic with `RichStringIf`, facilitating template-based code generation using placeholders, and integrating with external libraries such as ROS. These features collectively contribute to the language's flexibility, robustness, and efficiency in domain-specific modeling and code generation tasks.

Using the KRL in this context demonstrates its power and flexibility in domain-specific software engineering[164] and code generation tasks, as depicted in Figure 7.6. This approach auto-generated computer code for multiple use cases that showed an efficiency of 60% in reducing the manual effort to write code and almost 50% reduction in the development time of control software[88, 89]. The following chapter presents the evaluation of the knowledge-driven approach.

7.6.1 Potential Applications

The KRL can be extended to various domains beyond robotic control. Some potential examples include:

- **Medical Knowledge Representation:** Creating visualizations and textual descriptions of medical data, such as patient records, treatment plans, and medical procedures.
- **Financial Modeling:** Generating real-time dashboards and reports for tracking financial metrics, stock trends, and market analysis.
- **Educational Content Generation:** Designing interactive educational materials, quizzes, and visual aids for various subjects.
- **Geographical Information Systems:** Representing geographical data through maps, charts, and spatial analysis.

These examples demonstrate the versatility and applicability of the DSL in representing knowledge through both textual and visual means, catering to diverse domains and requirements.

7.7 Learnings and Limitations

The development and application of the proposed language in various industrial contexts, including the representation of robotic component knowledge, have yielded valuable insights into its capabilities, strengths, and areas for improvement. As a comprehensive tool designed to integrate with semantic knowledge bases and enable the generation of multiple target representations, the language has demonstrated its potential to address complex challenges in configuration management, documentation automation, and cross-platform development. However, like any innovative solution, it also presents certain limitations and challenges that must be acknowledged and addressed. This section provides an in-depth analysis of the key learnings and limitations, exploring the language's usability, flexibility, efficiency, integration challenges, and more. The insights derived from this analysis contribute to a nuanced understanding of the language's applicability and guide future enhancements and adaptations.

7.7.1 Learnings

Usability and Flexibility: The language's rich syntax and semantic capabilities enable concise representation of complex structures and relationships. The template-driven approach enhances reusability and consistency in robotic control code generation. Furthermore, integration with knowledge bases facilitates centralized management and dynamic updates, enhancing real-time solutions.

Efficiency and Automation: The language contributes to reducing development time by up to 50% and ensures alignment with business logic through its code generation features. It also streamlines maintenance and enhances robustness through documentation automation, contributes to a 60% reduction in manual effort, and enhances maintainability through configuration management.

Cross-Domain Applicability: The versatility of the language demonstrates a wide-ranging impact across domains like medical knowledge representation and financial modeling. Its ability to be customized allows tailored solutions to specific industry needs, enhancing modularity and extensibility.

7.7.2 Limitations

Learning Curve: The complexity of the language may present a steep learning curve for new users, especially those unfamiliar with semantic modeling. Comprehensive documentation is required to facilitate adoption and mitigate learning challenges.

Performance and Scalability: Performance overhead may affect real-time responsiveness in large-scale systems, and scalability concerns require careful design and optimization to maintain consistency across representations.

Integration and Compatibility: The language poses challenges in adaptability when integrating with existing systems like ROS and necessitates thorough testing to ensure flexibility across platforms, standards, and technologies.

Maintenance and Support: Long-term maintenance is essential for long-term success and sustainability, and sustained community support is required to foster innovation and gather feedback.

The learnings from applying the language highlight its potential to enhance productivity, consistency, and automation across various domains. However, the limitations underscore the importance of addressing complexity, performance, integration, and long-term sustainability challenges. Continuous improvement, comprehensive documentation, community engagement, and alignment with industry standards are key to maximizing the language's impact and adoption.

7.8 Conclusion

This chapter encapsulates the conceptual basis, principles, and challenges of the proposed language; the subsequent chapter will delve into the practical applications of our approach. Building on the theoretical foundations and insights gained in this research, the next chapter will explore the real-world use cases mentioned earlier, demonstrating our approach's efficacy, versatility, and relevance in solving concrete industrial challenges. This transition sets the stage for an in-depth analysis, showcasing how theory translates into practice, and will unravel the true potential of our method in varied and complex environments.

Chapter 8

Use cases

This section presents a structured experimental study to evaluate the applicability and effectiveness of the K-IDE platform for automated control software development. The experimental setup is described below in Table 8.1.

The detailed use cases are described below.

8.1 Automated Code Generation for Boom Barrier Vehicle Entry System

The scenario includes a boom barrier that opens only for authorized vehicles. Each vehicle carries an RFID tag, which is detected and validated before allowing access.

The system must detect whether a vehicle is present and respond appropriately. If the vehicle is authorized, the system allows entry; access is denied. The system must also ensure it is ready before processing incoming vehicles. The control system begins in a ready state and only processes vehicles when ready. Once a vehicle is detected, the system checks whether the vehicle is authorized. If authorized, the boom barrier opens. If not, access is denied. After each event, the system returns to the ready state.

A coordination component ensures vehicles approach only when the system is ready. The system and its environment interact in a structured way to ensure valid behavior. This setup is broken into modular components, each with a defined responsibility. These include an RFID reader, a tag validator, and a gate actuator. Each component operates independently but is integrated to support the required system behavior.

8.1.1 Implementing a Boom-Barrier Vehicle Entry System

The boom-barrier system is composed of the following components:

- **RFID Sensor:** Detects RFID tags and outputs the vehicle identity.

Table 8.1: Overview of Experimental Setup Across Use Cases

Aspect	Use Case 1: Boom Barrier Vehicle Entry	Use Case 2: Robotic Pick-and-Place System	Use Case 3: Smart Meeting Room Reconfiguration
Objective	Automate vehicle access based on RFID validation	Automate robotic pick-and-place task execution	Dynamically reconfigure IoT devices based on user commands
Domain	Access Control / Embedded Systems	Industrial Robotics / Automation	IoT / Smart Environment
Modeled Devices	RFID Reader, Tag Validator, Gate Actuator	KUKA Robot Arm, Bins, Racks	Smart Lights, Thermostat, Projector
Control Capabilities	Tag detection, Authorization check, Gate actuation	Grasp, Move, Pick, Place	On/Off control, Set temperature
Knowledge Artifacts	OWL ontology (access rules), M&C ML for devices, Capability DSL	OWL ontology (task and geometry), M&C ML, SMACH task recipes	OWL capability ontology (devices and actions), dynamic intent interpretation
Modeling Steps	• Define goals • Model device states • Compose system behavior		
Execution Environment	Abstract simulation with capability traces	GAZEBO simulator with SDF models	K-IDE platform integrated with Zetta framework
Tools Used	K-IDE, OWL, M&C ML, Capability DSL	K-IDE, M&C ML, SMACH, GAZEBO	K-IDE, OWL, Zetta Framework
User Interaction	No interactive inputs, event-driven operation	Task-driven simulation and reconfiguration	Runtime command injection from user interface
Validation Strategy	Trace-based simulation and state analysis	Simulated execution and activity flow trace validation	Simulated dynamic behavior in response to runtime commands

- **Validator:** Verifies whether the tag corresponds to an authorized vehicle.
- **Actuator:** Controls the physical movement of the gate to allow or restrict entry.

Each component is modeled as a state-based system with well-defined inputs, outputs, and states:

- The RFID Sensor operates in idle, detecting, and detected states. It detects tags when powered on and transitions to a detected state upon signal receipt.
- The Validator operates in active or inactive states. It processes incoming validation requests and returns a result indicating whether the tag is authorized.
- The Actuator supports states such as off, ready, and opened. It receives commands to open or close the gate and performs the corresponding physical action.

Each component's behavior is described through a sequence of actions that reflect its role in the system. The observed behaviors collectively ensure that the system can fulfill the vehicle entry requirement.

8.1.2 Composite Machine Configuration – Using Capability Composition

By integrating the RFID reader, validator, and actuator, the composed system exhibits the correct behavior. Each component contributes part of the required behavior, and their combined operation replicates the expected logic of vehicle entry automation.

To interpret the system behavior correctly, it is necessary to understand how component outputs relate to application-level events. For example, tag detection by the RFID reader maps to vehicle recognition, and actuator actions correspond to allowing or denying access.

By analyzing the behavior of all three components in context, it is evident that the composed system performs as intended. The desired behavior is successfully achieved through the combination of device capabilities.

The major advantages of this approach include:

1. It improves transparency by emphasizing trace-based self-descriptions of behavior.
2. It provides a general framework for reasoning about correctness, completeness, and integration strategies.
3. It supports automation of control logic synthesis and validation in tools like K-IDE.

Although not intended for manual proofs, the formalism helps frame problems, clarify semantics, and guide the design of knowledge-driven control systems.

```

Capability Gate_Manipulation compatible component interface BoomBarrier.BoomBarrier Interface
{
  Init {
    fire Commands [
      BoomBarrier.BoomBarrier_Interface.POWER_ON()
    ]

    subscribe alarms [BoomBarrier.BoomBarrier_Interface.DEVICE_ERROR()]
  }
  providesControlCapabilities {
    fireable commands : BoomBarrier.BoomBarrier_Interface.OPEN_GATE,
      BoomBarrier.BoomBarrier_Interface.CLOSE_GATE
  }
  providesOutcomes {
    receivable responses BoomBarrier.BoomBarrier_Interface.GATE_OPERATION_RESPONSE
    receivable alarms BoomBarrier.BoomBarrier_Interface.DEVICE_ERROR
  }
}

```

Figure 8.1: Describing the Boom Barrier Capabilities with Capability DSL

```

ActivityDiagram Weighing_Bridge_Automation produces results
(string fail="fail",string success="success")
has activities
{
  Activity Detect_Vehicle {
    requireCapability : RFID Reader

    conditions {
      from RFID_Reader.RFID_Reader_Interface.RFID_TAG_DETECTED

      if outcome RFID_Reader.RFID_Reader_Interface.RFID_TAG_DETECTED.id :
        =>nextActivity : Validate_Vehicle ,

      from RFID_Reader.RFID_Reader_Interface.DEVICE_ERROR

      if outcome RFID_Reader.RFID_Reader_Interface.DEVICE_ERROR.msg is :
        final result : fail
    }
  },
  Activity Validate_Vehicle{
    inputData {
      RFID_Reader.RFID_Reader_Interface.RFID_TAG_DETECTED.id
    }
  }
}

```

Figure 8.2: Describing the Business Workflow for Smart Vehicle Entry Automation Scenario

8.1.3 Implementation

8.2 Auto Generation of Robotic Control Software

A real-life case study was conducted to validate the approach and principles explored previously. In collaboration with TCS Research, groups involved in a robotics challenge developed software for a pick-and-place robotic system. Relevant data about the manually implemented robot controller software were collected and are shown in Table 8.2.

Table 8.2: Software Development Parameters

Parameters	Value
Lines of code	1500
No. of packages	1
No. of files	21
Reused libraries	9
Coding structure	Non-modular
Time taken to develop the software (days)	30

The objective was to ascertain what fraction of the solution can be modeled as capability knowledge and to what extent the proposed approach can generate the solution automatically. The challenges encountered included:

- The code lacked proper modularization and structure.
- Reusable libraries and specific code intertwined in the same package.

As the first step, the code was reorganized over a couple of weeks. The main script containing the entry point function was identified, refactored, and restructured to align with the principles laid out in this thesis. This restructuring enabled the modeling of capabilities, resources, and software libraries accordingly. Notably, 66% of the code was automatically generated as it can be modeled as robotics capabilities knowledge, with the remaining 34% requiring manual writing due to specific business logic.

Following the proposed architecture, this use case was addressed using the following steps: (1) Capturing context knowledge as application domain knowledge using OWL ontology, which involved identifying all items in the application context, including knowledge related to the physical shape and geometry and integrating SDF based simulation models of objects; (2) Capturing device knowledge in terms of the resource interface and behavior using M&C ML DSL, including modeling the resource interface, defining resource behavior, creating the resource capabilities knowledge ontology, and detailing the resource simulation as models and configuration; (3) Designing a task recipe for specific actions using domain and robotic

capability knowledge; (4) Auto-generating activity flow and prototype implementation; (5) Enabling simulation in the GAZEBO simulator. This sequential approach ensures a structured and comprehensive way of building a context-aware system that can respond dynamically to various requirements and scenarios.

Simulation details were captured for robotic resources like the Kuka Arm and context objects like the Rack, Bin, etc. These models were reused from the GAZEBO models repository, further enriching the knowledge of robotic capability. Several figures were used to illustrate various aspects of the simulation, mechanical components, and execution.

This approach enables exploring multiple solution design options by altering resource-capability mapping. Quick prototype generation and minimal manual effort for specific logic specifications lead to a streamlined development process. The effort reduction in this use case amounted to approximately 60% (2.5X productivity), and the engineering cycle time was cut by around half (2X). With a base of reusable knowledge and components, the cycle time and productivity gains can be even higher, enhancing the quality of the final solution.

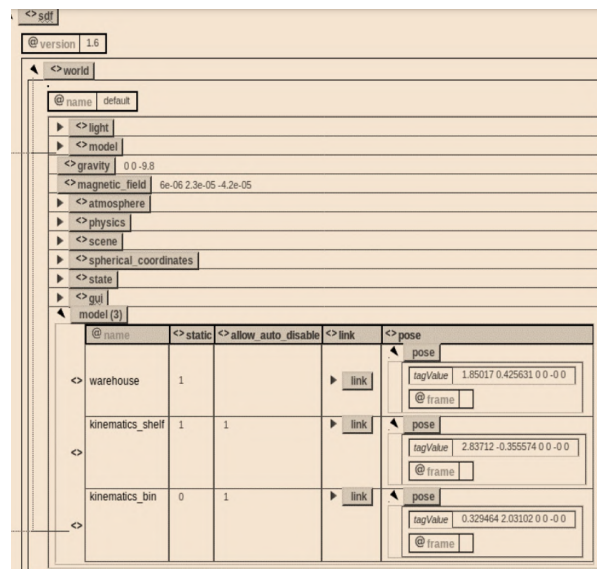


Figure 8.4: Simulation SDF Model



Figure 8.5: Execution of prototype in simulated environment

The integration of the case study into the broader context of this thesis contributes to validating the theoretical foundations laid out in previous sections and further supports the practical applicability of the approach in control software development.

8.3 Dynamic Reconfiguration of a Smart Meeting Room

The K-IDE platform demonstrates broad applicability across a range of industrial domains and industries. It can be utilized in robotics, automation, industrial control, energy management, and many other fields where control systems play a crucial role. This subsection describes a *Smart Meeting Room* scenario, solved with our approach and platform.

A smart meeting room has IoT devices such as smart lights, a smart projector, and a smart thermostat. The smart meeting room is initially configured to operate the devices based on a fixed schedule. For instance, the lights and projector are programmed to turn on during typical meeting hours, and the thermostat is set to maintain a comfortable temperature. A team decides to have an impromptu late-night brainstorming session. They want the lights to stay on, the projector to be ready, and the room temperature to be comfortable for their meeting. They use a mobile app to send this command to the smart meeting room system.

Device	Capability	Interface
Smart Light	On/Off	Zigbee
Smart Projector	On/Off	Wi-Fi
Smart Thermostat	Temperature Control	Wi-Fi

Table 8.3: IoT Devices in the Smart Meeting Room

Command	Action
Turn On	Activates the device
Turn Off	Deactivates the device
Set Temperature	Adjust the thermostat to the desired temperature

Table 8.4: Commands for the IoT Devices

Time	Device	Command
22:00	Smart Light	Turn On
22:00	Smart Projector	Turn On
22:00	Smart Thermostat	Set Temperature (22°C)

Table 8.5: Commands Sent During the Late-Night Brainstorming Session

8.3.1 Dynamic Reconfiguration Process

1. The smart meeting room system receives the team's command and identifies it as a request to change the operation of the lights, projector, and thermostat.
2. The system refers to the capability ontology to understand the capabilities of these devices and how they can be reconfigured.
3. The system identifies the "on/off" capability of the lights and projector, the "temperature control" capability of the thermostat, and the interfaces through which these capabilities can be accessed.
4. The system sends commands to the lights, projector, and thermostat through the identified interfaces, instructing them to stay on and maintain a comfortable temperature until further notice.
5. The devices respond to the commands and their operation is dynamically reconfigured according to the team's request.

Figures 8.6 and 8.8 show the use-case solved using the K-IDE Platform implementing our approach.

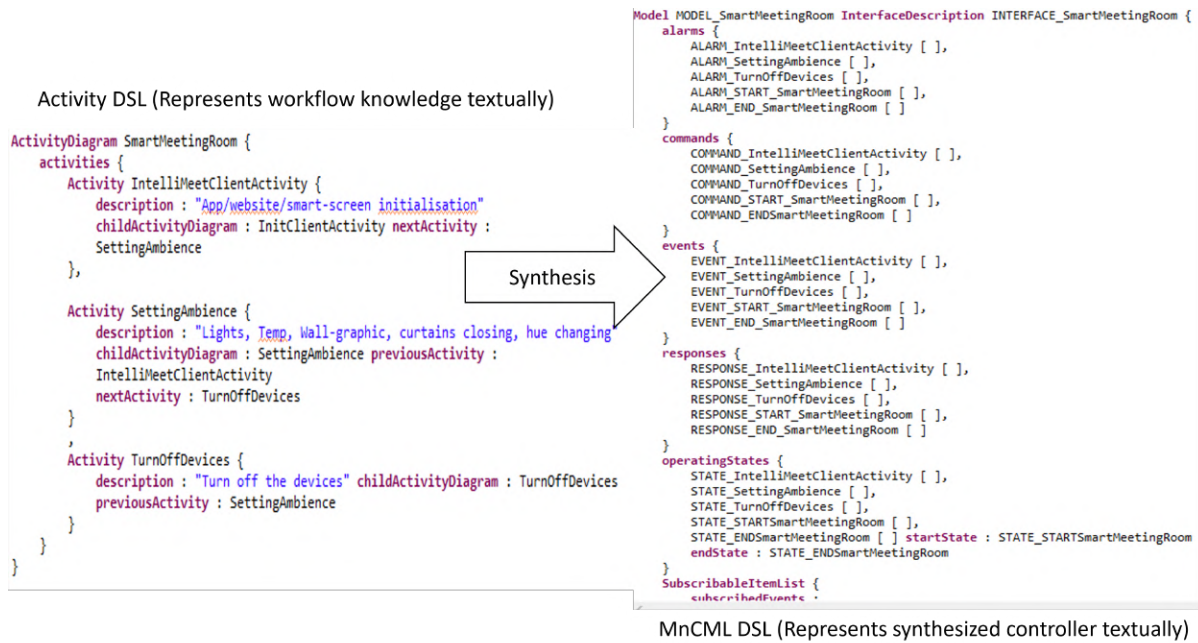


Figure 8.6: Composition of Controller for Smart Meeting Room Workflow

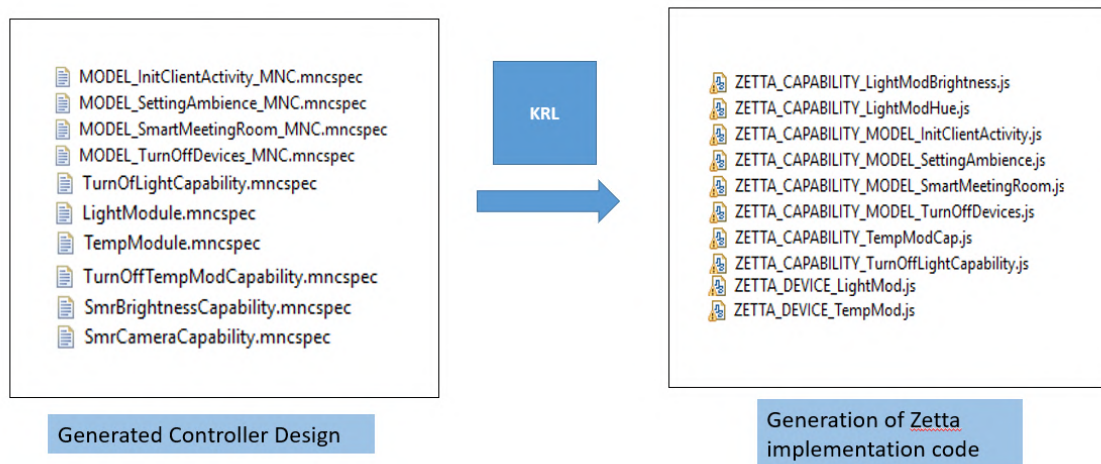


Figure 8.7: Generation of Executable Control Software using Zetta[165] framework

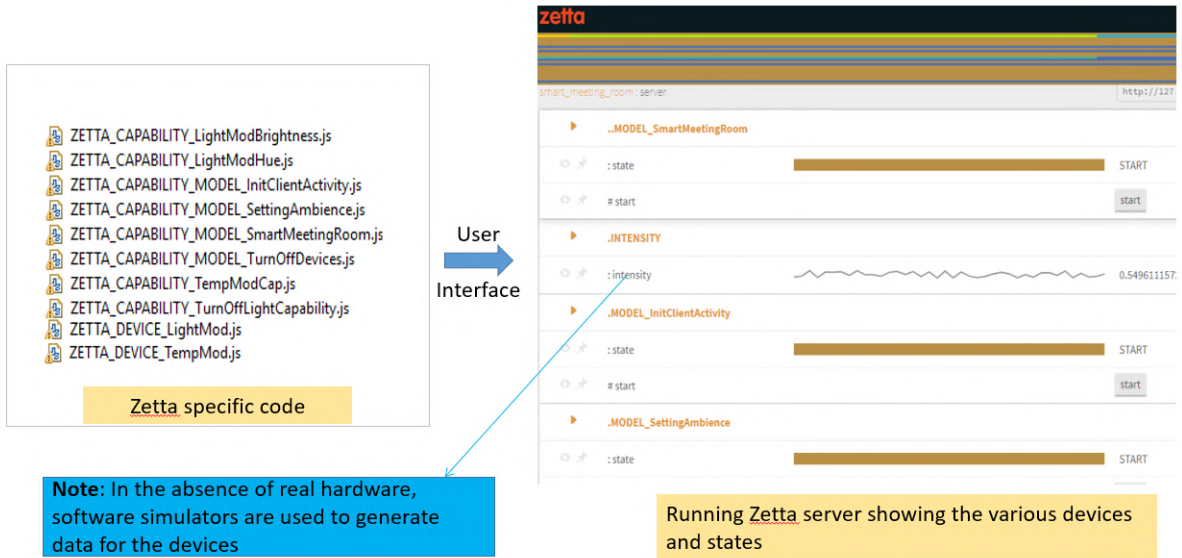


Figure 8.8: Execution of Smart Meeting Control Software: Using Simulation

This use case demonstrates how an intelligent meeting room system can use a capability ontology to understand the capabilities of its IoT devices and dynamically reconfigure their operation in response to changing requirements. This allows the system to provide a flexible and responsive environment that adapts to the needs of its users.

Similarly, in a manufacturing plant, the K-IDE platform can automate the design and reconfiguration of control systems for production lines. It can dynamically adjust the control strategies based on changing production requirements, equipment availability, or quality control parameters. K-IDE's capabilities enable efficient adaptation to varying production demands and optimize overall system performance.

The K-IDE platform can automate the control design and reconfiguration of smart grids or renewable energy systems in the energy management sector. It can dynamically optimize energy generation, storage, and distribution based on fluctuating energy demands, availability of renewable resources, or grid conditions. K-IDE's automation capabilities ensure efficient utilization of energy resources and enable the integration of new energy sources into the grid.

8.4 Comparison with Other Approaches

To benchmark K-IDE against existing automation paradigms, a structured comparative evaluation was conducted across four representative categories: rule-based systems [166], model-driven engineering tools [167], template-based code generators [161], and large language models [146]. Representative tools (DROOLS [166], TANGO/POGO [168], Yeoman [22],

and LLM-based assistants [169]) were applied to the three industrial use cases (8.1) described earlier. Each approach was evaluated using the below defined metrics:

- **Control Logic (Design) Automation** : The percentage of generated control logic (design).
- **Code Generation** : The percentage of deployable code generated without manual authoring.
- **Semantic Knowledge Representation** : The ability to capture, preserve, and apply domain semantics. “High” indicates explicit ontology-driven reasoning; “Moderate” indicates partial semantics inferred or encoded; and “Low” indicates no semantic modeling capability.
- **Custom Code Generation** : The system’s ability to produce tailored control code across heterogeneous device languages. “High” indicates multiple programming language and device-specific output; “Moderate” indicates partial customisation; “Low” indicates template-based or fixed-output code generation only.
- **Ease of Use** : The engineering effort required to configure, operate, and adapt the tool. “High” reflects minimal configuration effort; “Moderate” reflects some manual steps; and “Low” indicates steep learning effort or extensive configuration. This metric was further validated through a user study involving practitioners who assessed the usability of each approach. The details are shared in section 8.5.1.
- **Reconfiguration Support** : The extent to which each approach adapts to evolving device capabilities or domain knowledge. “Automated” denotes dynamic reconfiguration support without human intervention; “Limited” indicates partial or inference-based adaptation, where human trigger is needed; “Manual” denotes manual updates; and “None” indicates no reconfiguration support.

The same use cases were then implemented using K-IDE, enabling a controlled comparison of how ontology-driven semantic modelling influences automation performance, code quality, custom code generation, and adaptability. This produced a consistent, end-to-end comparative analysis of K-IDE and the four existing paradigms.

- **Rule-Based Systems (DROOLS)**: DROOLS relies on explicit rules that lack inherent semantic meaning and therefore cannot capture the contextual dependencies present in industrial automation scenarios. For example, in the *Vehicle Entry Automation* use case, rules such as “if RFID detected, open gate” cannot reason about additional constraints such as time-dependent access permissions or multi-sensor validation. The rules remain

rigid and require manual updates when domain knowledge changes. As a result, design automation remains limited to 30%, and semantic relationships between devices, policies, and events are not represented.

- **Model-Driven Engineering (TANGO, POGO):** TANGO and POGO generate code from UML diagrams but lose substantial semantic detail because UML models capture structural behaviour but not domain ontologies or contextual knowledge. In the *Robotic Pick-and-Place* use case, constraints such as workspace boundaries, allowed object types, or dynamic safety zones cannot be fully encoded in UML state machines alone. This limits the automation achievable during synthesis (approximately 55%) and complicates cross-language generation for PLCs, motion controllers, or ROS-based robots.
- **Code Generators (Yeoman):** Yeoman template-based generators automate boilerplate code creation but lack the semantic grounding required for knowledge-rich control tasks. In the *Smart Meeting Room Reconfiguration* use case, Yeoman can generate template scripts for sensor initialization or actuator control but cannot infer relationships such as “projector on” requiring a certain lighting condition, or “room occupied” implying HVAC adjustments. About 65% of the code is generated, but the control logic and semantic dependencies require manual effort.
- **Large Language Models (LLMs):** LLMs can infer patterns from data and generate control logic textually, but they struggle with strict domain constraints, safety rules, and deterministic system behaviour. In the *Robotic Pick-and-Place* scenario, LLMs generated certain unsafe or incomplete motion sequences, requiring engineers to manually verify correctness. Continuous tuning and prompt engineering are needed, limiting design automation to approximately 40%. While some semantic relationships are inferred, domain-specific constraints are inconsistently generated.
- **Our Approach (K-IDE):** K-IDE maintains semantic integrity through the use of ontologies, knowledge graphs, and capability models. In the *Smart Meeting Room Reconfiguration* use case, specifications such as “if a meeting is scheduled after 6:00 PM, maintain 22°C and dim lights to 40%” are represented semantically across devices, temporal conditions, and environmental preferences. This enables automated synthesis of valid control configurations, achieving 70% design automation and 75% code generation while supporting multi-language custom code generation and dynamic reconfiguration.

While alternative approaches exhibit partial success in capturing semantic knowledge, K-IDE allows design automation, and cross-language custom code generation. A tabular comparison of the evaluated approaches is provided in Table 8.6.

Table 8.6: Comparison with other approaches across various metrics

Approach		Control Logic (Design Automation (%))	Code Generation (%)	Semantic Knowledge Representation	Custom Code Generation	Ease of Use	Re-configuration Support
Rule-Based Systems		30%	40%	Low	Low	Low	None
Model-Driven Engineering (TANGO, POGO)		55%	50%	Moderate	Low	Moderate	Limited (Static)
Code Generators		60%	65%	Low	Moderate	Moderate	Manual
Large Language Models		40%	45%	Moderate	Moderate	Moderate	Limited (Inference-based)
Our approach (K-IDE)	Ap- (K-	70%	75%	High	High	Moderate	Automated

8.5 Approach Evaluation

This section evaluates the Knowledge-aided Integrated Development Environment (K-IDE) for control software development. The evaluation uses objective metrics, practical testing, user interviews, and surveys to assess performance. The methodology is adapted from Wiesmayr et al. [170].

Benchmark Dataset Availability: The use cases presented in this work are derived from real-world industrial scenarios in collaboration with research and industry partners. While no public benchmark datasets currently exist for knowledge-driven control software synthesis, portions of the domain-specific ontologies, sample device capability models, and representative control workflows used in this study may be made available upon request to support reproducibility and further research. The creation of standardized open benchmark suites for knowledge-driven control synthesis remains an important direction for the research community.

The system was tested in three real industrial scenarios described in the earlier usecases: 1) Vehicle Entry Automation 2) Robotic Pick-and-Place and 3) Smart Meeting Room Reconfiguration.

To evaluate system performance in practical scenarios, the following quantitative metrics were defined:

Code Volume Reduction (%): The reduction in the amount of manually written code due to automated design synthesis and code generation.

Engineering Time Reduction (%): The reduction in overall engineering time spent on designing, configuring, and validating control software.

Code Automation (%): The share of deployable control code automatically generated by the synthesis and code generation mechanisms.

Error Reduction (%): The decrease in testing and configuration errors resulting from automated validation and ontology-driven consistency checks.

Artifact Reusability (%): The proportion of reusable knowledge artifacts, such as ontologies and DSL specifications, that are used across multiple scenarios.

Synthesis Success Rate: The percentage of cases in which the system successfully synthesizes a valid control configuration.

Table 8.7 summarizes the collected results for the three use cases.

Table 8.7: Observed and Estimated Metrics Across Real-World Use Cases

Metric	Robotic Warehouse	Smart Meeting Room	Vehicle Entry Automation
Code Volume Reduction (%)	40%	25%	30%
Engineering Time Reduction (%)	70%	45%	70%
Code Automation (%)	80%	70%	60%
Error Reduction (%)	30%	30%	30%
Artifact Reusability (%)	75%	60%	65%
Synthesis Success Rate	100%	95%	100%

8.5.1 User Evaluation Experiment Setup

To assess the K-IDE platform’s usability and effectiveness, a mixed-method user study was conducted, combining structured interviews and surveys.

Objective: Evaluate user perception across five themes: efficiency, automation, flexibility, error reduction, and collaboration.

Participants:

- 10 interviewees: control engineers, developers, and domain experts.
- 60 survey respondents: mixed experience levels.

Participants were purposively sampled across domains (robotics, IoT, automation) to ensure representativeness and avoid bias.

Instruments:

- **Interviews:** Semi-structured, recorded and transcribed.
- **Survey:** Likert-scale and open-ended questions (e.g., ease of use, automation utility).

Procedure:

- All participants used the K-IDE platform in real projects.
- The interviews lasted between 30 and 45 minutes, and the surveys were completed online.

Analysis:

- *Qualitative:* Thematic coding of interview transcripts (deductive + inductive)

- *Quantitative*: Statistical summaries of survey responses (avg, min, max, box plots)

Bias Mitigation:

- Independent facilitators conducted sessions
- Participants were not involved in platform development

8.5.2 User Interviews

Structured interviews were conducted with ten participants, including control system engineers, domain experts, and software developers who had used the proposed platform in their respective projects. Participants were selected to ensure a representative distribution of domain experience and application diversity. The interviews focused on five key themes relevant to control software development. Summarized insights are provided below:

1. Impact on Development Process:

Participants reported measurable improvements in development workflows. Notably, K-IDE contributed to reduced development cycle times and improved responsiveness to evolving system requirements. These gains are attributed to the integrated modeling tools, validation mechanisms, and task-oriented interface, which facilitate systematic design and testing.

2. Use of Automated Design and Reconfiguration:

Respondents emphasized the utility of K-IDE's automation capabilities in adapting to changing operational conditions. Several examples were cited where automated reconfiguration enabled seamless system updates, such as the integration of new hardware devices (e.g., additional sensors) without requiring extensive manual redesign.

3. System Flexibility:

Participants acknowledged enhanced system flexibility, particularly in scenarios involving frequent device additions or changes in control strategy. K-IDE's support for modular control descriptions and dynamic configuration updates allowed systems to be rapidly adapted to new specifications.

4. Error Reduction and Validation:

Interviewees observed a noticeable reduction in configuration and runtime errors. K-IDE's built-in validation and verification mechanisms were effective in detecting inconsistencies early in the design phase, minimizing downstream errors and ensuring conformance with operational constraints.

Aspect	Average Rating
Efficiency	3.8
Flexibility	3.5
Error Reduction	4.0

Table 8.8: Average ratings from user interviews (on a scale of 1-5)

8.5.3 Questionnaires

Questionnaires were distributed to 60 users to gather quantitative data on K-IDE’s performance. The respondents included a mix of novice and experienced users, ensuring a broad range of perspectives. The questionnaire included the following questions:

1. On a scale of 1-10, how would you rate the platform’s ease of use?
2. How effective do you find the automated design and reconfiguration features?
3. How often do you use the visualization and analysis tools, and how helpful do you find them?

Question	Average Rating	Lowest Rating	Highest Rating
Ease of use	7.5	3	10
Effectiveness of automated design	7.3	2	10
Usefulness of visualization tools	8.0	3	10

Table 8.9: Average ratings from the questionnaire

8.5.4 Quantitative Evaluation and Comparative Analysis

To visualize participant-level feedback, Figure 8.9 presents box-and-swarm plots for both interview and questionnaire responses. The left plot shows the distribution of ratings from 10 interviewees regarding efficiency, flexibility, and error reduction. The right plot captures questionnaire responses from 60 users on ease of use, effectiveness of automation, and usefulness of visualization tools. The plots reveal central trends as well as inter-participant variability across participants, confirming broad satisfaction while identifying some areas for further improvement.

The results shown in these figures indicate a consistent reduction in manual development effort and perceived complexity when using the proposed knowledge-driven approach. This improvement can be attributed to the reuse of capability knowledge and automated synthesis,

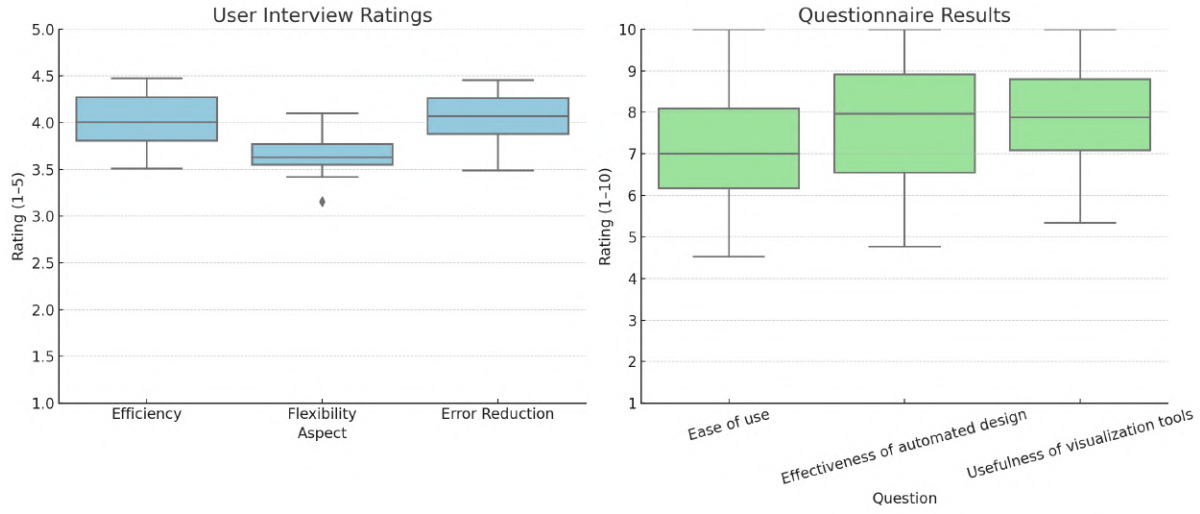


Figure 8.9: User Evaluation Results: Box and Swarm Plots of Interview and Questionnaire Feedback

rather than ad-hoc control logic construction. These findings support the thesis claim that explicit knowledge representation enables scalable and reusable control software development.

8.5.5 Performance Acceptance Criteria

To determine whether the system meets operational requirements, performance thresholds were adopted based on recent benchmarks in Industrial IoT and CPS research [171, 172]. The following criteria were used:

- **Latency:** Should be below 200 ms for soft real-time responsiveness.
- **Reconfiguration time:** Should not exceed 250 ms to support dynamic system adaptation.
- **CPU Overhead:** Should remain under 10% to prevent interference with concurrent tasks.

These thresholds are consistent with the performance needs of real-world CPS deployments and are used as benchmarks for evaluating the proposed platform's scalability and real-time suitability.

K-IDE was tested on increasing numbers of connected devices across representative scenarios. In the Smart Meeting Room setup, which involved 10 devices, the system achieved a latency of 50 ms, a reconfiguration time of 120 ms, and a CPU utilization of 3%. For the Robotic Arm scenario with 50 devices, latency increased to 60 ms, reconfiguration took 130 ms, and CPU usage rose to 4%. With 100 devices in the Vehicle Gate Automation scenario, the latency reached 75 ms, and the reconfiguration time was 140 ms with a CPU overhead of 5%. In the Smart Factory test with 500 devices, latency rose to 110 ms and reconfiguration time to 160 ms,

accompanied by 7% CPU usage. The most complex test, a City-scale IoT deployment with 1000 devices, resulted in 130 ms latency, 180 ms reconfiguration delay, and 9% CPU usage.

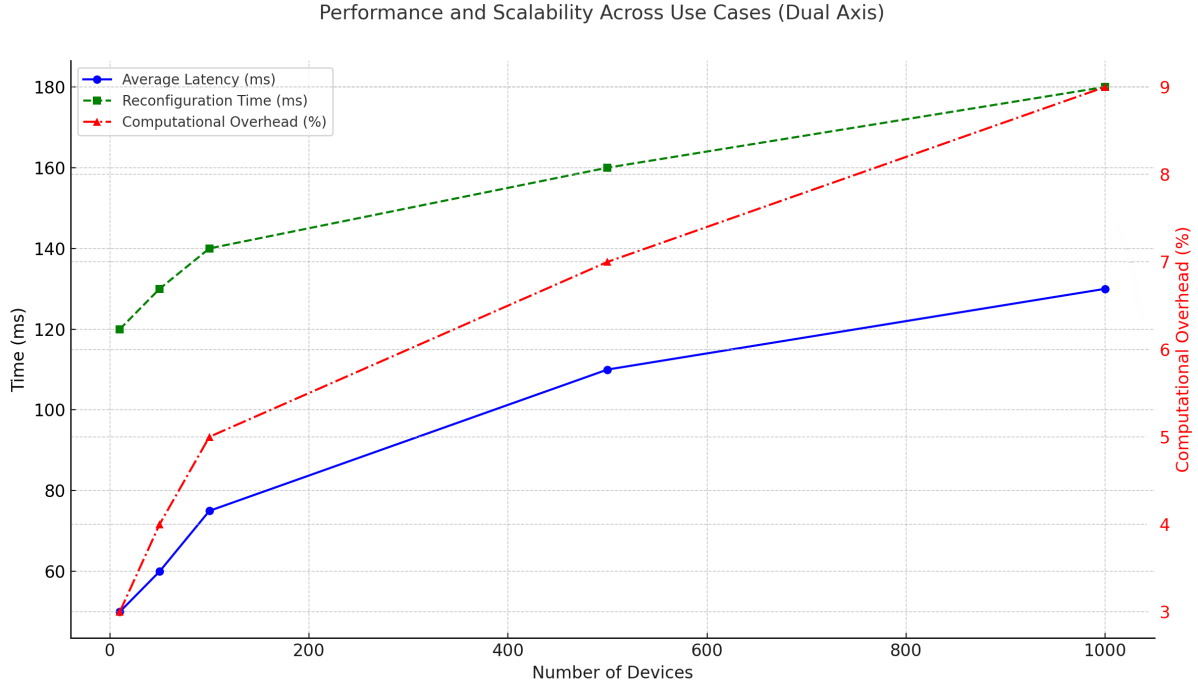


Figure 8.10: Performance and Scalability Across Use Cases

Latency and computational overhead increased predictably with system size. Importantly, even at the highest scale tested (1000 devices), the system maintained performance within the defined acceptable limits. This demonstrates the system’s suitability for large-scale deployment in soft real-time applications.

Figure 8.11 compares the proposed platform with two widely used alternatives: Node-Red and MaRCO.

In terms of latency, the proposed platform exhibited a slightly higher delay (95 ms) compared to Node-Red (50 ms) and MaRCO (65 ms), primarily due to the reasoning overhead introduced by the Java-based synthesis engine. The memory usage was also greater at 120 MB, attributed to the ontology and structured knowledge representation, whereas the alternatives ranged between 60 and 100 MB. However, K-IDE’s benefits come from a more advanced control logic synthesis process with computational complexity of $O(n \log n)$, enabling richer and dynamic reconfiguration strategies that are beyond the capabilities of the compared tools, which operate at linear complexity.

The key findings from the evaluation confirm that the proposed approach enhances several critical aspects of control system engineering. K-IDE increases engineering efficiency by automating routine tasks, reducing errors, and accelerating development cycles. Its design supports dynamic reconfiguration, which improves system flexibility and responsiveness to

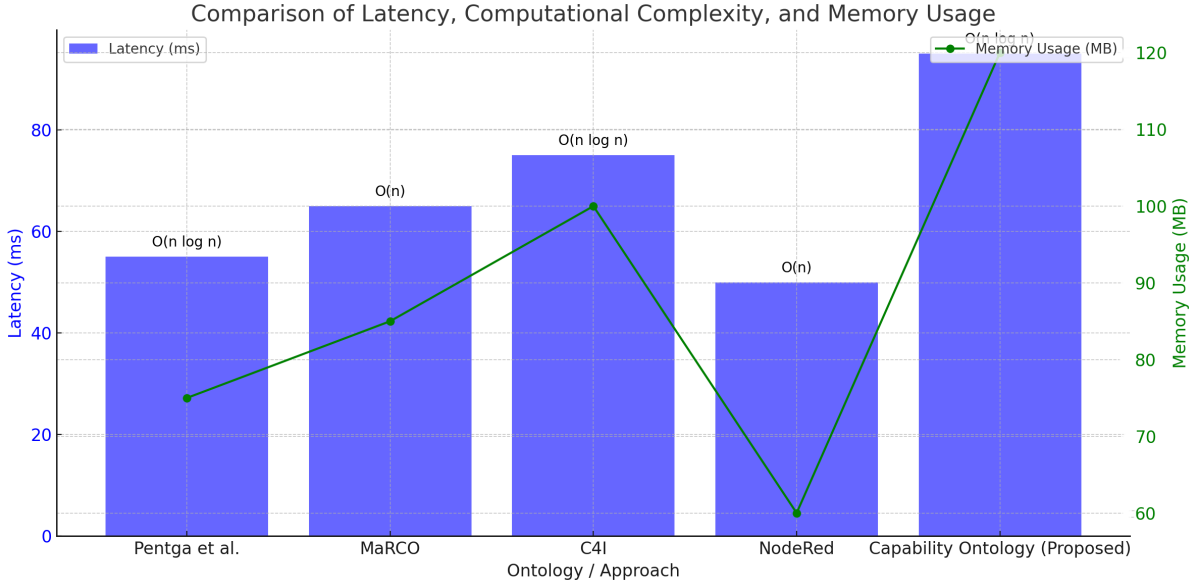


Figure 8.11: Latency, Memory, and Complexity Comparison

changes. The evaluation also demonstrates robust scalability, with performance maintained up to 1000 devices and consistent system behavior. K-IDE further promotes effective collaboration through features such as shared artifact libraries and integrated version control. Additionally, its modular architecture ensures compatibility and integration with existing tools and workflows across domains.

Despite these strengths, certain limitations remain. The user interface can benefit from further refinement to enhance usability, especially for first-time users. The automated design and synthesis features can also be extended to support more advanced use cases. Finally, optimizing K-IDE's reasoning speed and reducing latency for large-scale configurations remain key areas for future enhancement.

The generalizability of K-IDE is supported by its use of standardized ontologies and protocol-based architecture. Its modular and distributed reasoning design enables scalability across diverse application domains. In practice, K-IDE demonstrated consistent performance, achieving reduced in development time and manual effort. Additionally, it significantly reduced integration and runtime errors through automated validation and constraint checking.

8.6 Conclusion

The exploration of various use cases, including the dynamic reconfiguration of a smart meeting room, has provided detailed insights into the practical applicability of knowledge-driven approaches in control software development. The system's flexibility, adaptability, and intelligence have been illustrated through the precise representation of IoT device capabilities

and the demonstration of dynamic adjustments in response to real-time requirements. The structured composition of controllers, adherence to set protocols, and real-time interaction with devices affirm the robustness of the approach.

In the subsequent evaluation, the Knowledge-aided Integrated Development Environment (K-IDE) platform was assessed across critical dimensions of control system engineering. K-IDE demonstrated increased efficiency, faster development cycles, enhanced flexibility, and error reduction through user interviews, questionnaires, and practical testing in real-world scenarios. Its robust debugging tools, intuitive interface, dynamic reconfiguration capabilities, and collaboration environment underscored its adaptability and effectiveness across various applications.

Collectively, the experimental results demonstrate that control software can be systematically generated and reconfigured by leveraging explicit capability knowledge rather than manual programming. The observed reductions in effort, rework, and error rates provide empirical support for the central thesis claim that knowledge-driven approaches are effective for control software development and reconfiguration.

Chapter 9

Conclusion

This thesis presents a knowledge-driven approach to automating control software development and reconfiguration, addressing the challenges of scalability, cost, and reliability in industrial systems. The proposed framework enables efficient and adaptive control software generation by leveraging a Capability Ontology, Domain-Specific Languages (DSLs), and synthesis algorithms. The contributions are validated through real-world applications, demonstrating significant efficiency improvements in control software development and reconfiguration processes.

The proposed knowledge-driven control software development framework aligns with emerging paradigms in Industrial AI and Industry 5.0[173], which emphasize human-machine collaboration, intelligent automation, and adaptable control systems. By formalizing domain knowledge and enabling dynamic reconfiguration, the approach supports scalable, intelligent manufacturing and cyber-physical system integration, which are central to Industry 5.0 objectives.

The primary contributions of this thesis are as follows:

- **Capability Ontology:** A structured ontology was developed to represent device capabilities, facilitating knowledge reuse and automation in control software design.
- **Domain-Specific Languages (DSLs):** The creation of DSLs provided a syntactic framework for describing device capabilities and control requirements, reducing the complexity of the manual design.
- **Synthesis Algorithm:** The proposed algorithm automated the generation of control designs from high-level requirements, enabling rapid reconfiguration with minimal manual intervention.
- **Code Generation Engine:** A semantic code generation engine was introduced to translate synthesized designs into executable software, bridging the gap between design and implementation.

- **K-IDE:** The development of K-IDE integrated all proposed components into a unified platform, streamlining control software development and reconfiguration processes.

The validation of the proposed framework across three industrial setups, warehouse robotics, automated vehicle entry, and a smart meeting room, demonstrated significant improvements:

- Development time reduced by 60%.
- Reconfiguration effort minimized by 50%.
- Enhanced consistency and reliability in control software design.

These results highlight the potential of knowledge-driven approaches to revolutionize control software development, making it scalable, efficient, and adaptable to dynamic industrial requirements.

The thesis effectively addresses key challenges in control software development, including:

- Managing the scale and complexity of large industrial systems.
- Incorporating domain-specific knowledge into the control software.
- Ensuring safety, reliability, and real-time performance.
- Streamlining the frequent reconfiguration of control systems.

By automating traditionally manual processes, the proposed approach significantly reduces dependency on domain experts and mitigates errors associated with manual intervention.

Generalization to Additional Domains: While the proposed knowledge-driven framework is validated primarily in industrial automation contexts, its underlying principles are extensible to other complex cyber-physical domains. For example, in healthcare, the approach can support surgical robotics and patient monitoring systems where dynamic reconfiguration of medical devices is required. In the automotive sector, adaptive vehicle control and sensor fusion in autonomous driving present similar challenges in capability modeling and reconfiguration. Similarly, smart grid applications can benefit from automated control logic for dynamic energy balancing and integration of distributed renewable sources. Adapting the ontology and domain-specific languages to these sectors would require domain-specific capability modeling, regulatory knowledge incorporation, and specialized workflow abstractions.

9.1 Limitations

While the proposed framework offers substantial improvements, it is subject to certain limitations:

- **Middleware Dependency:** The framework assumes the existence of middleware to handle real-time constraints and communication.
- **Domain Generalization:** The approach has been validated primarily in industrial automation scenarios. Extending its applicability to other domains, such as healthcare or aerospace, requires further research.
- **Real-Time Performance Optimization:** The framework focuses on control design and code generation, with real-time optimization left to middleware solutions.

The findings and limitations of this research suggest several avenues for future work:

- **Real-Time Optimization:** Integration of real-time performance optimization into the proposed framework to handle time-critical operations more efficiently.
- **Broader Applicability:** Extension of the framework to diverse domains, such as healthcare, aerospace, and energy management, to assess its versatility and adaptability.
- **Advanced Reconfiguration Techniques:** Development of autonomous reconfiguration methods using AI and machine learning to reduce manual intervention further.
- **Dynamic Ontology Evolution:** Enhancements to the Capability Ontology to support dynamic updates and more complex systems, allowing it to adapt to evolving industrial requirements.
- **Integration with Edge and Cloud Computing:** Exploration of edge and cloud-based architectures to facilitate distributed control and scalability for larger systems.
- **Human-Computer Interaction (HCI):** Improving user interfaces and interaction models within K-IDE to support intuitive workflows for non-expert users.
- **Integration with Formal Methods:** Future work can explore the integration of formal verification methods and conflict resolution mechanisms for concurrent capability compositions.

This thesis provides a foundational step towards automating control software development using a knowledge-driven approach. By addressing the challenges of scalability, reliability, and adaptability, the proposed framework significantly enhances the efficiency of control software engineering. The demonstrated improvements underscore the potential of integrating domain knowledge and advanced algorithms to create a new control software design and reconfiguration paradigm. Future advancements in this area can further extend the impact of this thesis, paving the way for intelligent and autonomous industrial systems.

Published Journal Articles

1. Banerjee, Amar, and Venkatesh Choppella. "Knowledge Driven Synthesis Using Resource-Capability Semantics for Control Software Design." *IEEE Access*, vol. 11, pp. 52527–52539, 2023. *Publisher: IEEE*.
2. Banerjee, Amar, and Venkatesh Choppella. "Knowledge-aided Integrated Development Environment (K-IDE) for Control Software Development." *Computing in Science & Engineering*, 2024. *Publisher: IEEE*.
3. Banerjee, Amar, and Venkatesh Choppella. "A Knowledge-Driven Approach for Dynamic Reconfiguration of Control Design in Internet of Things and Cyber-Physical Systems." *IEEE Internet of Things Journal*, 2024. *Publisher: IEEE*.
4. Banerjee, Amar, and Venkatesh Choppella. "Control Software Engineering Approaches for Cyber-Physical Systems: A Systematic Mapping Study." *ACM Transactions on Cyber-Physical Systems*, 2024. *Publisher: ACM*.
5. Banerjee, Amar, and Venkatesh Choppella. "Challenges and opportunities in the industrial usage controller synthesis tools: A review of LTL-based opensource tools for automated control design." *Results in Control and Optimization*, 2025. *Publisher: Elseiver*.

Published Conference Proceedings

1. Banerjee, Amar, Venkatesh Choppella, Viswanath Kasturi, Swaminathan Natarajan, Padmalata V. Nistala, and Kesav Nori. "An Attempt at Explicating the Relationship between Knowledge, Systems and Engineering." In *Proceedings of the 11th Innovations in Software Engineering Conference*, pp. 1–11, 2018.
2. Chaudhuri, Subhrojyoti Roy, Amar Banerjee, Swaminathan Natarajan, Venkatesh Choppella, Arpan Pal, and P. Balamurali. "A Knowledge Centric Approach to Conceptualizing Robotic Solutions." In *Proceedings of the 12th Innovations in Software Engineering Conference (formerly known as India Software Engineering Conference)*, pp. 1–11, 2019.
3. Roy Chaudhuri, Subhrojyoti, Swaminathan Natarajan, Amar Banerjee, and Venkatesh Choppella. "Methodology to Develop Domain Specific Modeling Languages." In *Proceedings of the 17th ACM SIGPLAN International Workshop on Domain-Specific Modeling*, pp. 1–10, 2019.
4. Amar, B., Subhrojyoti R. C., Barnali B., Dhakshinamoorthy R., Rajesh N., and Venkatesh C. "Knowledge Driven Approach to Auto-Generate Digital Twins for Industrial Plants."

In Proceedings of the CIKM 2021 Workshops Co-located with 30th ACM International Conference on Information and Knowledge Management (CIKM 2021) Gold Coast, Queensland, Australia, November, pp. 1–5, 2021.

Manuscripts Under Review

- "Knowledge-Driven Control Software Engineering - A Literature Review." *IEEE Transactions on Human-Machine Systems*.
- "Knowledge Representation Language." *Elsevier Data & Knowledge Engineering*.

Appendices

Tables

Research Question	Description
RQ1	How actively is research being conducted control software engineering?
RQ2	The research approaches support which engineering phases?
RQ2.1	Which research solutions address multiple engineering phases?
RQ3	Which engineering methods are predominant in the construction of control software?
RQ4	Which engineering paradigms does control software engineering rely on?
RQ4.1	Which engineering phase is supported by which paradigm?
RQ5	What is the artefact type proposed to address problems in control software engineering?
RQ5.1	Which artefact type is suitable for which engineering phase?
RQ6	Which application domains are most studied?
RQ6.1	Which approaches have multiple target application domains?
RQ6.2	Which engineering phases are supported by the approaches for which target application domains?
RQ7	Which concerns are addressed by the research articles, using which paradigms and in which domains?

Table 1: Summary of research questions in literature survey

Sr. No	Base Search Strings
1	("control software engineering" OR "robot software engineering" OR "cyber-physical system software engineering" OR "internet of things software engineering") AND (("engineering" OR "development" OR "designing") AND ("environment" OR "platform" OR "approaches" OR "tools") AND ("robot" OR "internet of things" OR "cyber-physical system" OR "control" OR "distributed" OR "control systems"))
2	("control software" OR "control systems software" OR "cyber-physical software" OR "distributed systems software") OR "robot software" OR "orchestration software" OR "control and actuation software" OR "embedded software" AND (("engineering" OR "approach" OR "platform" OR "environment" OR "tool") OR ("validation" OR "design" OR "requirements"))

Table 2: Search strings used for literature search

Engineering Methods	Studies
Waterfall	[41, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195]
Agile	[196, 197, 198]

Table 3: Engineering methods used in the approaches from articles in the Literature Survey

Engineering Phases	Studies
Requirements	[166, 199, 200, 201]
Design	[32, 38, 41, 180, 185, 186, 192, 194, 195, 195, 199, 202, 203, 204, 205, 206, 207, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239]
Implementation Methods	[38, 174, 177, 182, 184, 187, 194, 205, 208, 211, 213, 218, 220, 222, 228, 237, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257]
Verification Methods	[181, 195, 198, 205, 209, 215, 219, 224, 249, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273]
Integration Methods	[229, 274, 275]
Operations Methods	[31, 195, 205, 226, 257, 263, 267, 268, 272, 276, 277]

Table 4: Engineering phases addressed by the approaches in the articles from literature survey

Engineering Paradigms	Studies
Model-Driven	[32, 41, 174, 176, 179, 180, 184, 185, 186, 187, 192, 194, 195, 195, 200, 201, 202, 203, 205, 206, 207, 207, 209, 210, 211, 212, 215, 220, 221, 222, 223, 224, 226, 228, 229, 230, 232, 233, 234, 236, 237, 238, 245, 253, 256, 257, 259, 261, 262, 264, 265, 266, 269, 271, 277, 278, 279]
Code-Driven	[38, 177, 178, 181, 182, 189, 204, 208, 213, 218, 239, 240, 241, 243, 244, 247, 248, 249, 250, 251, 254, 255, 257, 258, 260, 263, 267, 268, 272, 273, 275]
Knowledge-Driven	[38, 177, 178, 181, 182, 189, 204, 208, 213, 218, 239, 240, 241, 243, 244, 247, 248, 249, 250, 251, 254, 255, 257, 258, 260, 263, 267, 268, 272, 273, 275]

Table 5: Engineering paradigms used by the approaches in articles considered in the literature survey

Research Objective	Studies
Programming Framework	[177, 182, 204, 208, 218, 227, 236, 240, 241, 242, 248, 250, 251, 254]
Tool	[32, 185, 192, 194, 205, 209, 210, 211, 213, 215, 218, 222, 226, 230, 231, 237, 238, 244, 255, 256, 259, 260, 266, 269]
Concept	[31, 166, 174, 176, 179, 181, 184, 190, 193, 196, 197, 200, 201, 206, 207, 217, 220, 224, 225, 233, 234, 243, 244, 257, 263, 265, 267, 268, 270, 271, 275, 279, 280]
Language	[189, 223, 228, 246]
Formalism	[38, 41, 180, 202, 249, 258, 261, 264, 272, 273, 276]
Architecture	[187, 194, 195, 195, 199, 203, 205, 207, 209, 212, 216, 221, 229, 230, 232, 238, 239, 253, 262, 274, 277]

Table 6: Research objectives of the articles in the literature survey

Target Domain	Studies
Cyber Physical Systems	[166, 174, 177, 180, 181, 184, 188, 190, 192, 193, 195, 195, 196, 198, 199, 201, 202, 203, 204, 206, 207, 207, 209, 216, 219, 220, 221, 222, 223, 224, 225, 226, 230, 232, 233, 235, 236, 238, 239, 241, 243, 244, 245, 246, 247, 250, 253, 256, 258, 260, 261, 262, 263, 264, 266, 267, 268, 269, 270, 271, 273, 274, 275, 278, 279, 281]
Robotics & Mechatronics	[31, 32, 38, 41, 178, 185, 194, 200, 205, 208, 210, 211, 212, 213, 215, 217, 218, 228, 229, 234, 238, 240, 242, 245, 248, 249, 251, 252, 254, 255, 255, 257, 257, 259, 263, 265, 277]
Internet of Things	[174, 175, 182, 183, 187, 190, 193, 198, 221, 233, 237, 238, 239, 243, 253, 263, 274, 276]
Automotive	[196, 198, 220, 227, 231, 253, 264, 271]

Table 7: Target domains in articles from the literature survey

Concern	Studies
Safety and Security	[177, 180, 182, 183, 191, 198, 205, 215, 219, 231, 236, 239, 247, 248, 249, 253, 254, 263, 264, 266, 269, 271, 278, 280, 282]
Real-time Control	[174, 175, 177, 178, 181, 184, 188, 194, 195, 198, 205, 206, 207, 210, 213, 220, 221, 222, 223, 225, 228, 231, 237, 240, 244, 246, 252, 257, 278, 281]
Dynamic Adaptation	[41, 190, 199, 202, 214, 226, 232, 233, 243, 250, 255, 270, 276]
Modularity and Interfacing	[41, 175, 179, 184, 185, 186, 187, 192, 194, 200, 203, 204, 207, 207, 208, 210, 211, 212, 216, 218, 220, 221, 224, 229, 232, 234, 235, 238, 245, 248, 251, 252, 262]
Performance	[189, 195, 201, 236, 253, 259, 274, 279]
Fault Detection	[180, 197, 198, 215, 228, 230, 247, 258, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 271, 272, 273, 275, 277, 281, 282]
Context Awareness	[38, 166, 176, 183, 187, 196, 201, 206, 207, 209, 216, 219, 227, 239, 242, 250, 270, 279]
Modularity and Interfacing	[182, 209, 214, 215, 227, 246, 254, 276, 280]

Table 8: Concerns addressed in articles from the literature survey

Sources	Found	Unique	Relevant	Primaries
IEEE Digital Library	423	354	76	42
ACM Digital Library	364	211	81	34
Science Direct & Springer	216	77	62	39

Table 9: Search results from the literature study

Article Ref.	Year	Inclusion Criteria Met
Waibel et al. [35]	2011	Relevance, Methodological Rigor, Novelty and Originality
Beetz et al. [48]	2018	Relevance, Impact and Significance, Theoretical and Practical Relevance
Francesca et al. [38]	2014	Relevance, Methodological Rigor, Credibility and Quality of References
Sorouri [41]	2014	Relevance, Novelty and Originality, Thesis and Clarity
Nuzzo [42]	2015	Relevance, Impact and Significance, Thesis
Dai et al. [43]	2015	Relevance, Methodological Rigor, Theoretical and Practical Relevance
Zhou et al. [44]	2017	Relevance, Novelty and Originality, Credibility and Quality of References
Ming et al. [49]	2018	Relevance, Impact and Significance, Language and Clarity
dos Santos et al. [50]	2019	Relevance, Methodological Rigor, Publication Venue
Zhou et al. [52]	2020	Relevance, Novelty and Originality, Theoretical and Practical Relevance
Thuluva et al. [22]	2020	Relevance, Impact and Significance, Credibility and Quality of References

Table 10: Selection of articles for literature review

Sr. No	Title	Year	Citations	Article Type	Selection Rationale
1	Roboearth	2011	14	Journal Paper	Popular article in enabling knowledge creation for robotic workflows.
2	AutoMoDe: A novel approach to the automatic design of control software for robot swarms	2014	10	Journal Paper	Relevant paper.
3	A Compositional Approach to Control Software Design of Automation Systems Based on Mechatronic Modularity	2014	17	Thesis	Very relevant thesis which shows how knowledge can be used to automatically compose design.
4	Compositional design of cyber-physical systems using contracts	2015	18	Thesis	A very relevant thesis which shows how contracts knowledge can be used to automatically compose design.
5	Knowledge-driven service orchestration engine for flexible information acquisition in industrial cyber-physical systems	2016	7	Conference Paper	The article shows the knowledge approach to compose services for machine control. Presented in the industrial conference.
6	Model-based development of knowledge-driven self-reconfigurable machine control systems	2017	9	Journal Paper	Relevant paper.
7	Know rob 2.0, a 2nd generation knowledge processing framework for cognition-enabled robotic agents	2018	28	Conference Paper	Popular article and high cited publication from ICRA conference.
8	PDSIDES, a knowledge-based platform for decision support in the design of engineering systems	2018	23	Journal Paper	Relevant paper.
9	CONDENSE: contract based design synthesis	2019	4	Conference Paper	Very new work and shows how contracts knowledge can be used for synthesis. Presented in MODELS conference.
10	Knowledge-driven digital twin manufacturing cell towards intelligent manufacturing	2020	40	Journal Paper	Recent articles showing expansion of knowledge-driven approach applicable for digital twin control.
11	Semantic Node-RED for Rapid Development of Interoperable Industrial IoT Applications	2020	14	Journal Paper	Describes ontology based abstraction to design IoT applications.

Table 11: Paper list for literature review

Table 12: Questions related to respondent characteristics

Code	Question
A1	Age (in years)
A2	Gender
A3	Country
A4	Years of experience
A5	Role in control software development
A6	Years of experience in control software development

Table 13: General questions for demographic analysis of survey participants

Code	Question
B1	<i>Time is a constraint for control software development</i>
B2	<i>Extensive knowledge is required in control software development</i>
B3	<i>New techniques need to be implemented for control software</i>
B4	<i>IDEs are considered vital for predicting control components in complex scenarios</i>
B5	<i>Textbooks are essential for predicting associated control components based on the problem scenario</i>
B6	<i>Manual experience sharing is essential for predicting problem scenarios associated with control components</i>
B7	<i>Control components knowledge details are stored in text format</i>
B8	<i>Control components knowledge details are stored in image format</i>
B9	<i>Control components knowledge details are stored in coding format</i>
B10	<i>Control components knowledge details are stored in records format</i>
B11	<i>Time constraint persists while identifying control components in problematic scenarios</i>

Table 14: Likert scale-based questions to understand the role of knowledge and time consumed

Code	Question
B2.1	<i>It consumes more time for preparing and managing control design</i>
B2.2	<i>Control design architecture is highly domain-specific, limiting software translation</i>
B2.3	<i>Language dependency on control design is complex during software translation</i>
B2.4	<i>Data structure complexity hinders software development</i>
B2.5	<i>Evolution & Validation of translation into software is complex to track</i>

Table 15: Likert scale-based questions to understand challenges in control design and software development

Code	Question
C1.1	<i>High-level programming languages are required for control software development</i>
C1.2	<i>Are low-level languages a constraint in control software development</i>
C1.3	<i>There is reuse of knowledge skill associated with the program developed</i>
C1.4	<i>There is efficient information management across disciplines while developing the software</i>
C1.5	<i>Domain guidelines or standard codes are adhered to during control software development</i>

Table 16: Likert scale-based questions related to control design, code translation, and testing

Category	Frequency	Percent	Cumulative Percent
20-30 years	198	44.0%	44.0%
30-40 years	144	32.0%	76.0%
50 years and above	108	24.0%	100.0%
Total	450	100.0%	100.0%

Table 17: Age distribution of respondents

Gender	Frequency	Percent	Cumulative Percent
Male	324	72.0%	72.0%
Female	126	28.0%	100.0%
Total	450	100.0%	100.0%

Table 18: Gender distribution of respondents

Country	Frequency	Percent	Cumulative Percent
Asian countries	171	38.0%	38.0%
North America	126	28.0%	66.0%
South America	81	18.0%	84.0%
Other countries	72	16.0%	100.0%
Total	450	100.0%	100.0%

Table 19: Country distribution of respondents

Experience	Frequency	Percent	Cumulative Percent
Fresher	79	17.6%	17.6%
0-5 years	83	18.4%	36.0%
5-10 years	148	32.9%	68.9%
More than 10 years	140	31.1%	100.0%
Total	450	100.0%	100.0%

Table 20: Years of software engineering experience of survey respondents

Role	Frequency	Percent	Cumulative Percent
Domain expert	91	20.2%	20.2%
Designer	142	31.6%	51.8%
Developer	126	28.0%	79.8%
Tester	45	10.0%	89.8%
Deployment	46	10.2%	100.0%
Total	450	100.0%	100.0%

Table 21: Role of the respondents in control software development

Which testing component requires maximum time for development	Frequency	Percent	Valid Percent	Cumulative Percent
Design validation	128	28.4	28.4	28.4
Code testing	91	20.2	20.2	48.7
Hardware testing	103	22.9	22.9	71.6
Actual environmental testing	128	28.4	28.4	100.0
Total	450	100.0	100.0	

Table 22: Frequency, Percent, and Valid Percent Evaluation on time evaluation measurement for development using Testing component

What are the limitations associated with testing newly developed software	Frequency	Percent	Valid Percent	Cumulative Percent
Changing context dependency	80	17.8	17.8	17.8
Insufficient domain knowledge	97	21.6	21.6	39.3
Presence of diverse languages	81	18.0	18.0	57.3
Lack of formal model	118	26.2	26.2	83.6
Application domain is too specific	74	16.4	16.4	100.0
Total	450	100.0	100.0	

Table 23: Frequency, Percent, and Valid Percent Evaluation on the Limitation on Testing new Developed Software

Most Consuming Tasks in Software Development				
Task	Frequency	Percent	Valid Percent	Cumulative Percent
Understanding domain from expert	108	24.0	24.0	24.0
Designing based on knowledge	108	24.0	24.0	48.0
Development of software	93	20.7	20.7	68.7
Validation of developed software	76	16.9	16.9	85.6
Documentation and automation	65	14.4	14.4	100.0
Total	450	100.0	100.0	

Table 24: Cumulative Percentage Evaluation on most consuming tasks that might result in more time for software development, enhancing the overall cost

Figures

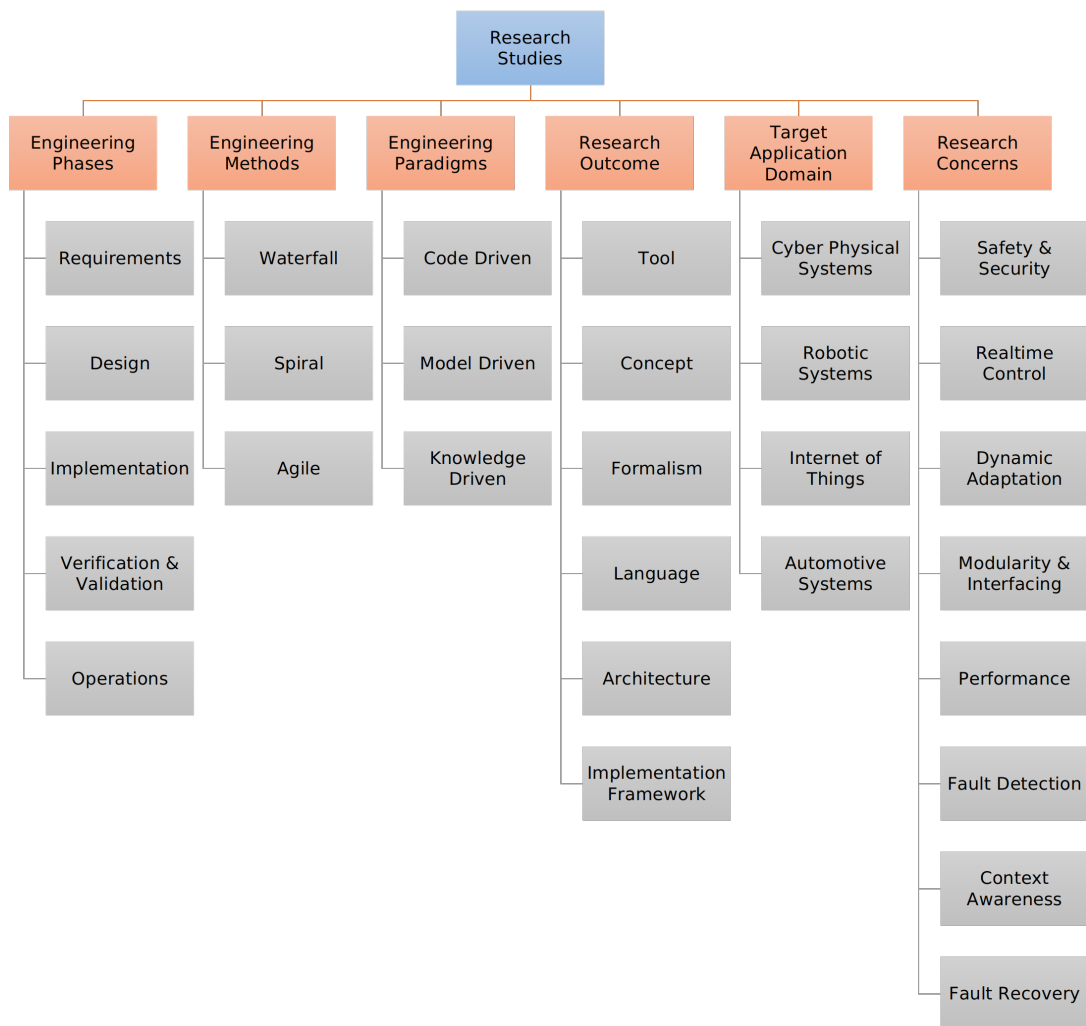
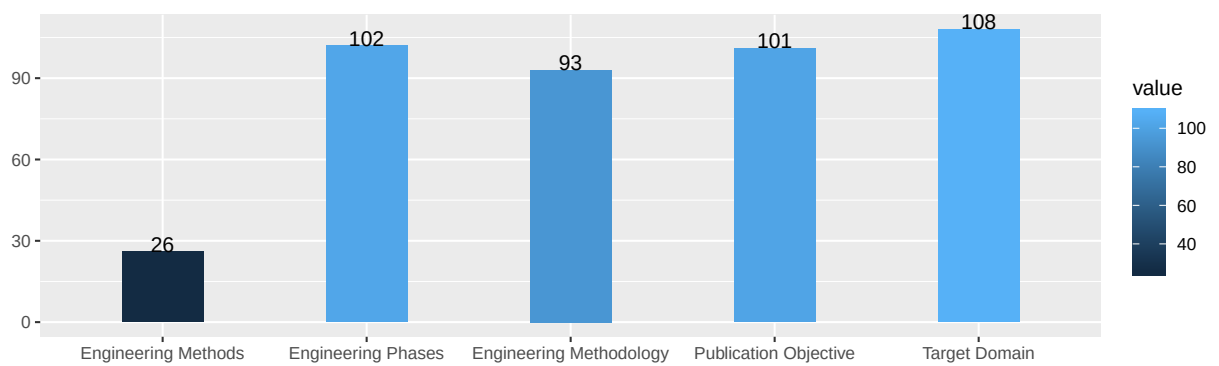
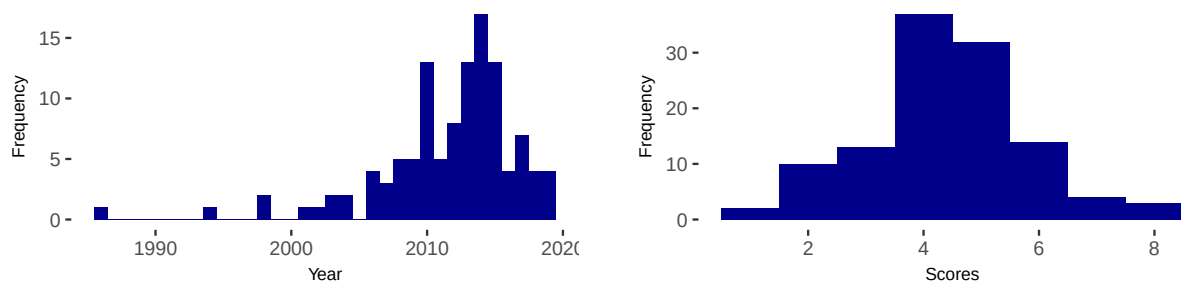


Figure 1: Classification Taxonomy



(a) Categorisation of articles



(b) Publications per year and score distribution

Figure 2: Trends in Control Software Engineering

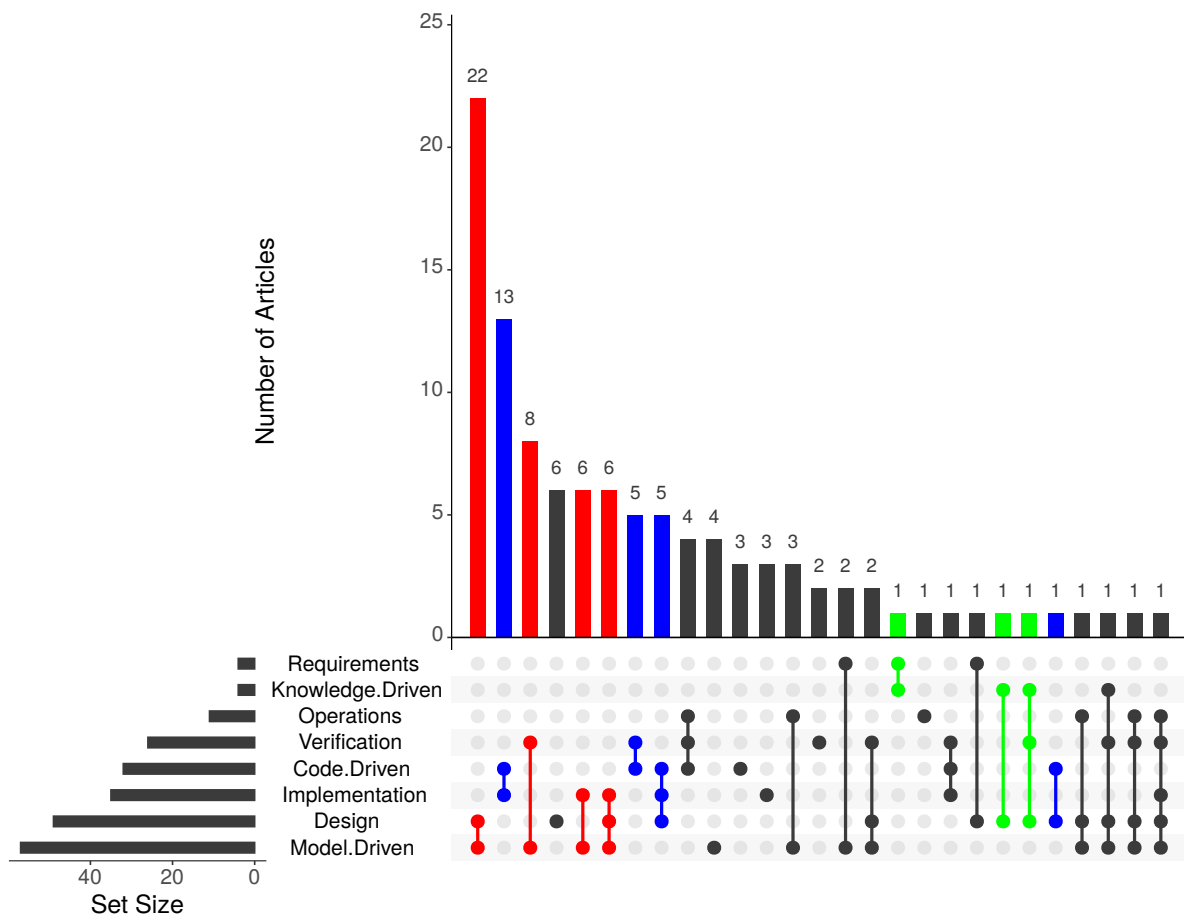


Figure 3: Publications supporting a specific engineering phase by using a specific engineering paradigm

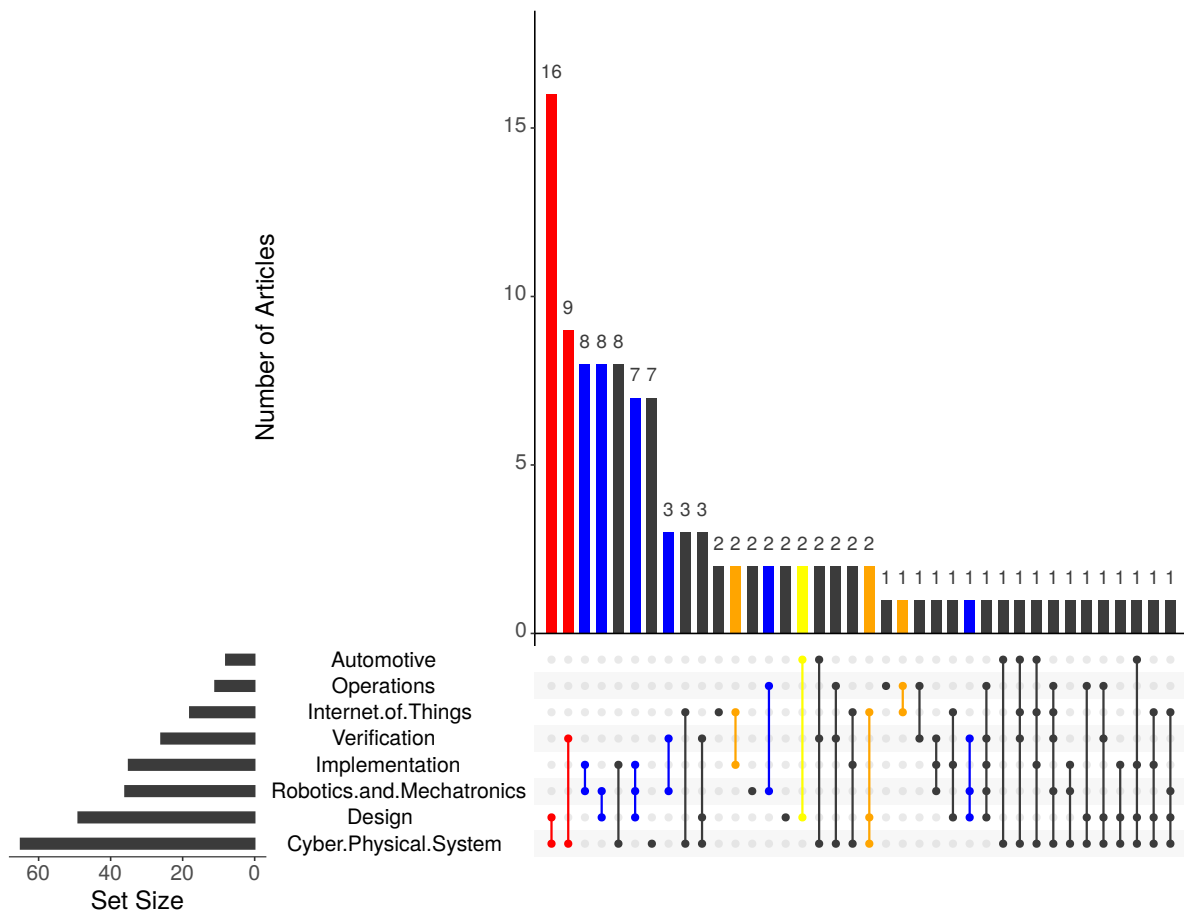
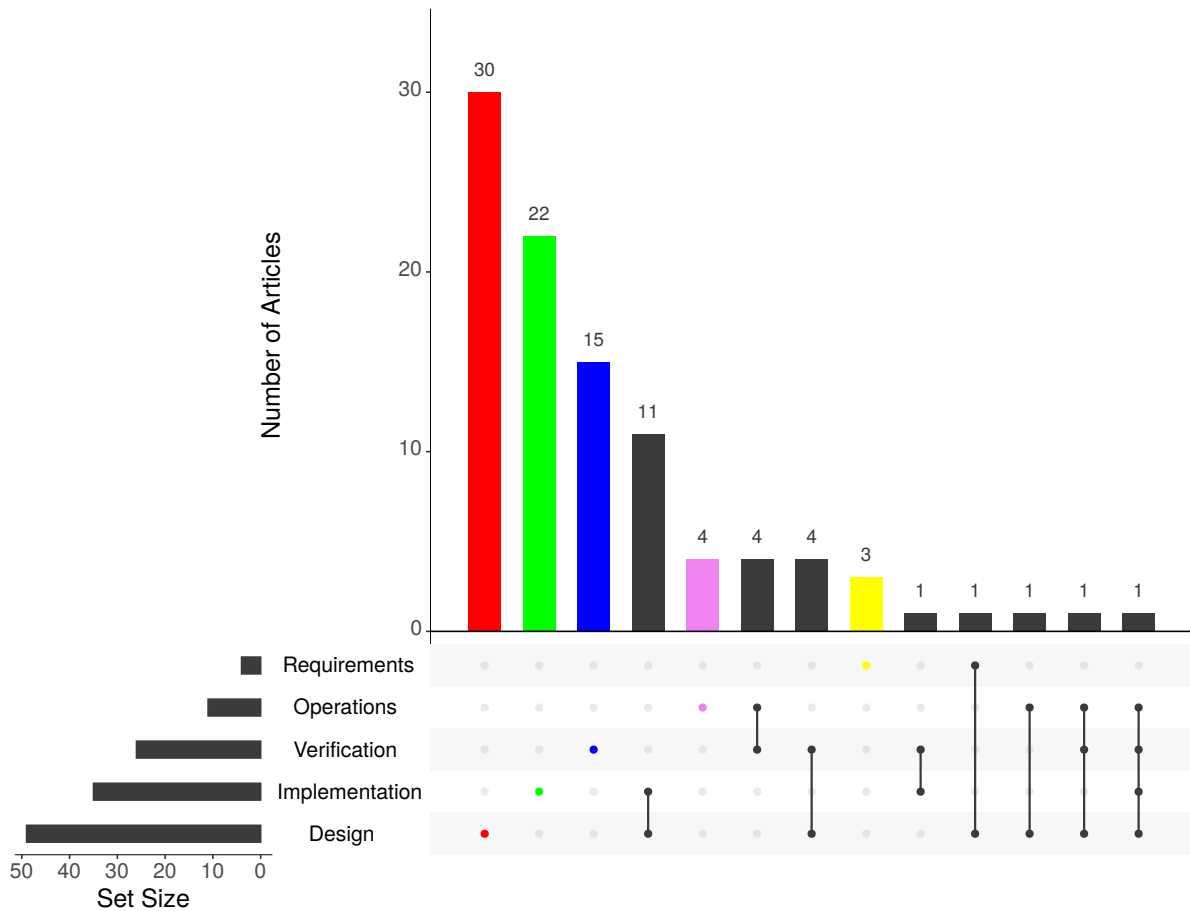
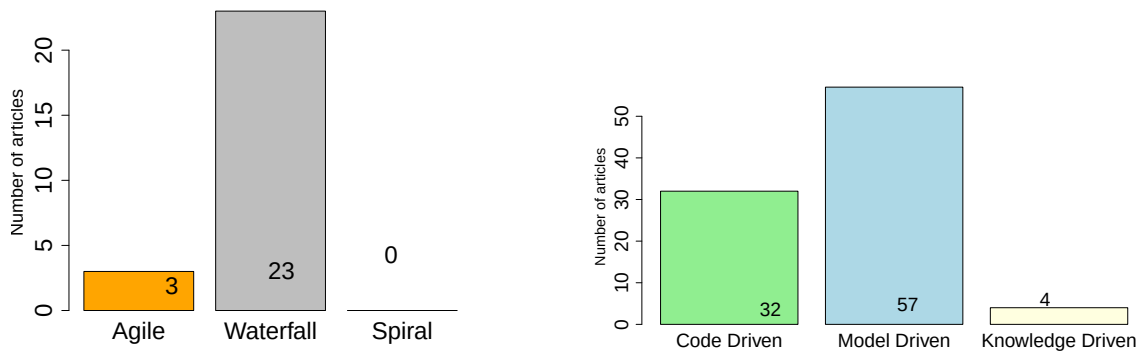


Figure 4: Publications supporting a specific engineering phase for a specific target application domain



(a) Literature distribution for Engineering Phases



(b) Distribution of Engineering Methodologies

(c) Distribution of Engineering Approaches

Figure 5: Trends in Control Software Engineering

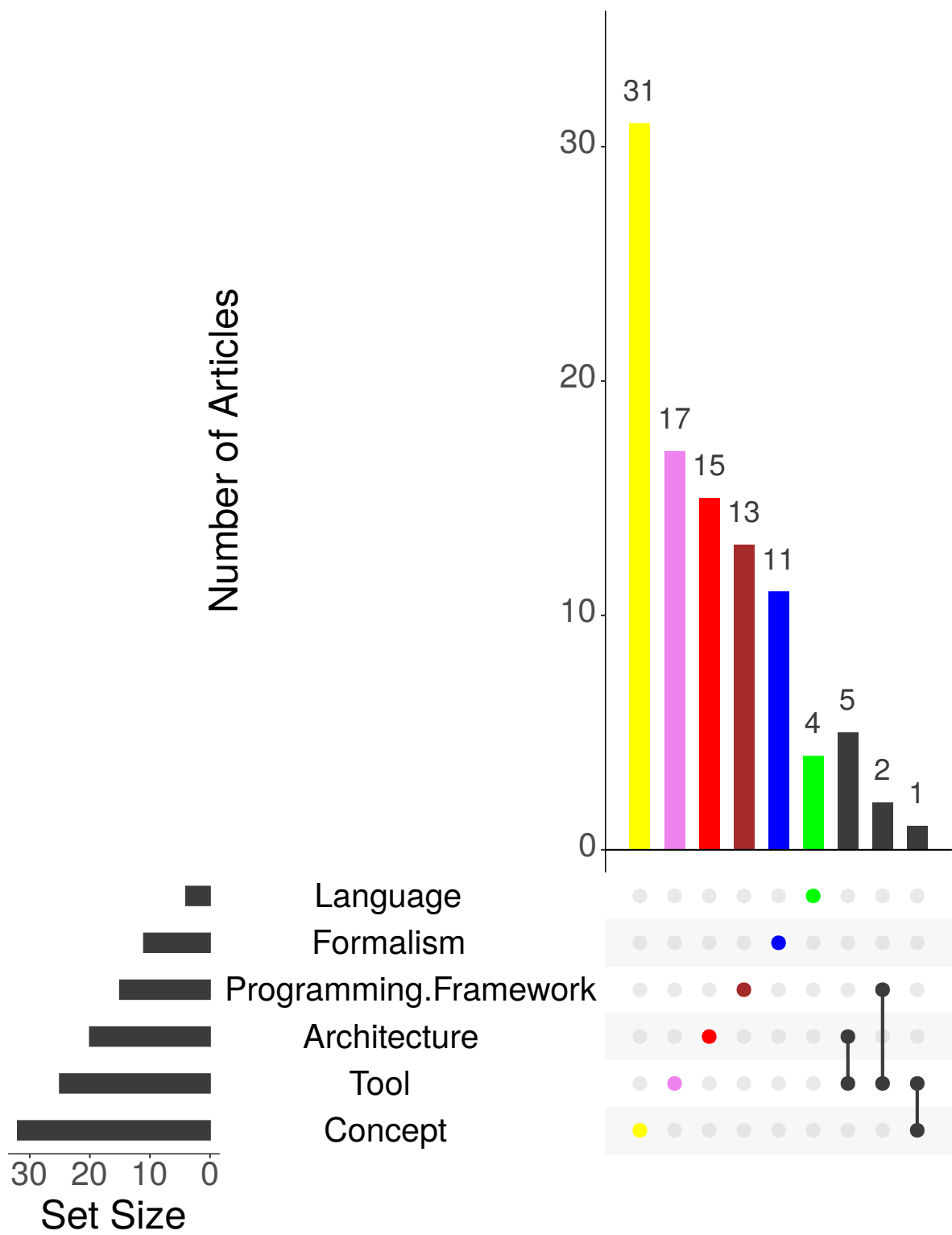


Figure 6: Research Objectives Distribution

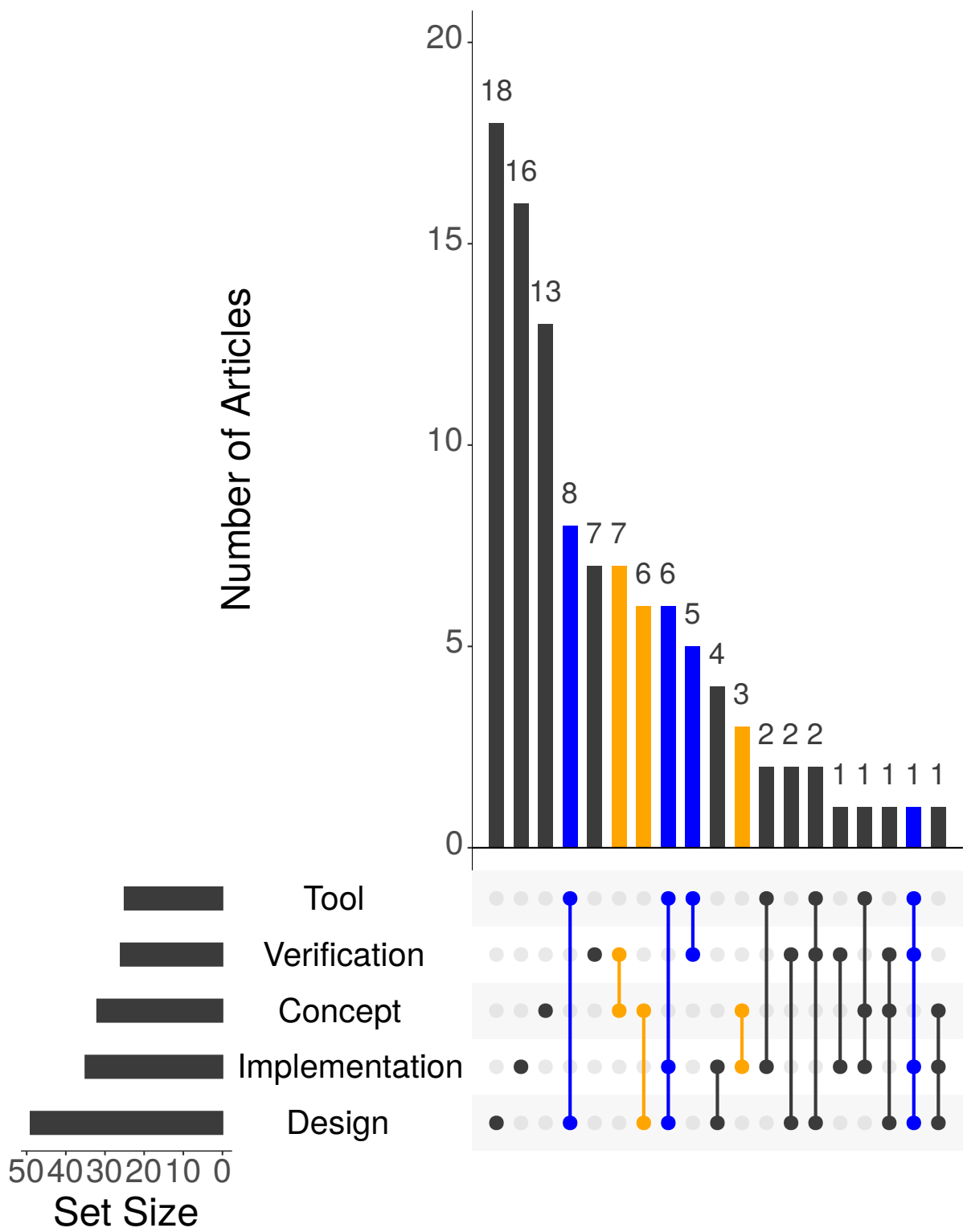


Figure 7: Tool vs Concept for Engineering Phases

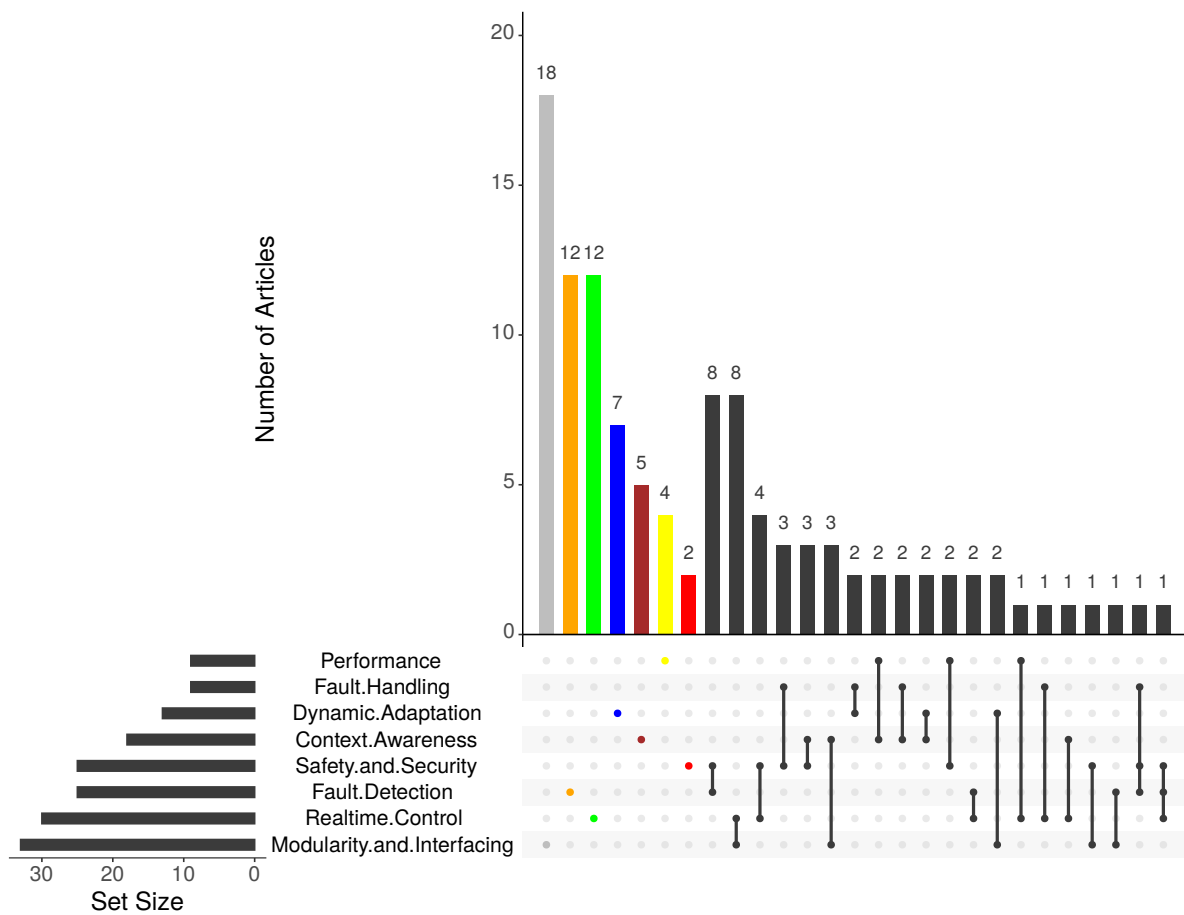


Figure 8: Identified concerns for control software

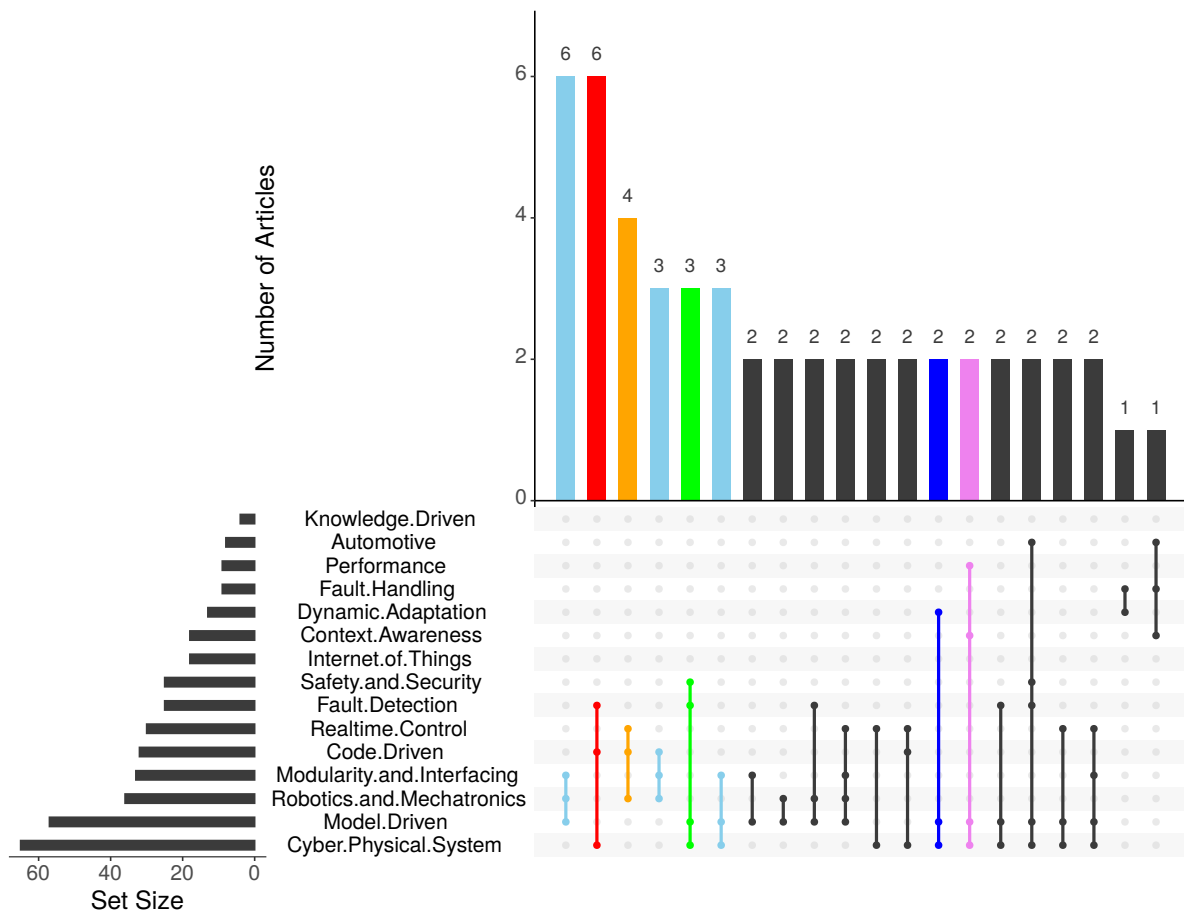


Figure 9: Concerns and relationships with target application domains and engineering paradigms

Ontology

```
<?xml version="1.0"?>
<rdf:RDF xmlns="http://iiit.serc.com/ontologies/capability.owl#"
  xml:base="http://iiit.serc.com/ontologies/capability.owl"
  xmlns:owl="http://www.w3.org/2002/07/owl#"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:xml="http://www.w3.org/XML/1998/namespace"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
  xmlns:swrl="http://www.w3.org/2003/11/swrl#"
  xmlns:swrla="http://swrl.stanford.edu/ontologies/3.3/swrla.owl#"
  xmlns:swrlb="http://www.w3.org/2003/11/swrlb#">
  <owl:Ontology rdf:about="http://iiit.serc.com/ontologies/capability.owl">
    <rdfs:comment>Ontology for representing capability knowledge</rdfs:comment>
    <rdfs:label>Capability Ontology</rdfs:label>
  </owl:Ontology>

  <!--
  //////////////////////////////////////
  //
  // Annotation properties
  //
  //////////////////////////////////////
  -->

  <!-- http://swrl.stanford.edu/ontologies/3.3/swrla.owl#isRuleEnabled -->
```

```

<owl:AnnotationProperty
  ↪ rdf:about="http://swrl.stanford.edu/ontologies/3.3/swrla.owl#isRuleEnabled"/>

<!--
////////////////////////////////////
//
// Object Properties
//
////////////////////////////////////
-->

<!-- http://example.com/ontologies/capability.owl#fulfilledBy -->

<owl:ObjectProperty rdf:about="http://example.com/ontologies/capability.owl#fulfilledBy">
  <owl:inverseOf rdf:resource="http://example.com/ontologies/capability.owl#hasCapability"/>
  <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Capability"/>
  <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#Device"/>
  <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#SessionType"/>
</owl:ObjectProperty>

<!-- http://example.com/ontologies/capability.owl#hasActivities -->

<owl:ObjectProperty rdf:about="http://example.com/ontologies/capability.owl#hasActivities">
  <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Workflow"/>
  <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#Activity"/>
</owl:ObjectProperty>

<!-- http://example.com/ontologies/capability.owl#hasCapability -->

```

```

<owl:ObjectProperty rdf:about="http://example.com/ontologies/capability.owl#hasCapability">
  <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Device"/>
  <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#Capability"/>
</owl:ObjectProperty>

<!-- http://example.com/ontologies/capability.owl#hasInteractions -->

<owl:ObjectProperty rdf:about="http://example.com/ontologies/capability.owl#hasInteractions">
  <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#SessionType"/>
  <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#Interaction"/>
</owl:ObjectProperty>

<!-- http://example.com/ontologies/capability.owl#hasInterface -->

<owl:ObjectProperty rdf:about="http://example.com/ontologies/capability.owl#hasInterface">
  <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Device"/>
  <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#Interface"/>
</owl:ObjectProperty>

<!-- http://example.com/ontologies/capability.owl#hasProperty -->

<owl:ObjectProperty rdf:about="http://example.com/ontologies/capability.owl#hasProperty">
  <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Device"/>
  <rdfs:range rdf:resource="http://iiit.searc.com/ontologies/capability.owl#Property"/>
</owl:ObjectProperty>

<!-- http://example.com/ontologies/capability.owl#implementsSession -->

<owl:ObjectProperty rdf:about="http://example.com/ontologies/capability.owl#implementsSession">

```

```

    <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Device"/>
    <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#SessionType"/>
</owl:ObjectProperty>

<!-- http://example.com/ontologies/capability.owl#nextActivity -->

<owl:ObjectProperty rdf:about="http://example.com/ontologies/capability.owl#nextActivity">
    <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Activity"/>
    <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#Activity"/>
</owl:ObjectProperty>

<!-- http://example.com/ontologies/capability.owl#nextInteraction -->

<owl:ObjectProperty rdf:about="http://example.com/ontologies/capability.owl#nextInteraction">
    <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Interaction"/>
    <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#Interaction"/>
</owl:ObjectProperty>

<!-- http://example.com/ontologies/capability.owl#previousActivity -->

<owl:ObjectProperty rdf:about="http://example.com/ontologies/capability.owl#previousActivity">
    <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Activity"/>
    <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#Activity"/>
</owl:ObjectProperty>

<!-- http://example.com/ontologies/capability.owl#requiredCapability -->

<owl:ObjectProperty rdf:about="http://example.com/ontologies/capability.owl#requiredCapability">
    <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Activity"/>
    <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#Capability"/>

```

```

</owl:ObjectProperty>

<!-- http://iiit.serc.com/ontologies/capability.owl#hasActions -->

<owl:ObjectProperty rdf:about="http://iiit.serc.com/ontologies/capability.owl#hasActions">
  <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Interaction"/>
  <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#Action"/>
</owl:ObjectProperty>

<!-- http://iiit.serc.com/ontologies/capability.owl#hasAlarms -->

<owl:ObjectProperty rdf:about="http://iiit.serc.com/ontologies/capability.owl#hasAlarms">
  <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Interface"/>
  <rdfs:range rdf:resource="http://iiit.serc.com/ontologies/capability.owl#Alarm"/>
</owl:ObjectProperty>

<!-- http://iiit.serc.com/ontologies/capability.owl#hasCommands -->

<owl:ObjectProperty rdf:about="http://iiit.serc.com/ontologies/capability.owl#hasCommands">
  <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Interface"/>
  <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#Command"/>
</owl:ObjectProperty>

<!-- http://iiit.serc.com/ontologies/capability.owl#hasData -->

<owl:ObjectProperty rdf:about="http://iiit.serc.com/ontologies/capability.owl#hasData">
  <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Interface"/>
  <rdfs:range rdf:resource="http://iiit.serc.com/ontologies/capability.owl#Datastream"/>
</owl:ObjectProperty>

```

```

<!-- http://iiit.serc.com/ontologies/capability.owl#hasEvents -->

<owl:ObjectProperty rdf:about="http://iiit.serc.com/ontologies/capability.owl#hasEvents">
  <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Interface"/>
  <rdfs:range rdf:resource="http://iiit.serc.com/ontologies/capability.owl#Event"/>
</owl:ObjectProperty>

<!-- http://iiit.serc.com/ontologies/capability.owl#hasResponses -->

<owl:ObjectProperty rdf:about="http://iiit.serc.com/ontologies/capability.owl#hasResponses">
  <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Interface"/>
  <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#Response"/>
</owl:ObjectProperty>

<!-- http://iiit.serc.com/ontologies/capability.owl#hasSession -->

<owl:ObjectProperty rdf:about="http://iiit.serc.com/ontologies/capability.owl#hasSession">
  <rdfs:domain
    ↪ rdf:resource="http://iiit.serc.com/ontologies/capability.owl#CapabilityInvocation"/>
  <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#SessionType"/>
</owl:ObjectProperty>

<!-- http://iiit.serc.com/ontologies/capability.owl#invokesCapability -->

<owl:ObjectProperty rdf:about="http://iiit.serc.com/ontologies/capability.owl#invokesCapability">
  <rdfs:domain rdf:resource="http://example.com/ontologies/capability.owl#Device"/>
  <rdfs:range rdf:resource="http://iiit.serc.com/ontologies/capability.owl#CapabilityInvocation"/>
</owl:ObjectProperty>

```

```

<!-- http://iiit.serc.com/ontologies/capability.owl#offersCapability -->

<owl:ObjectProperty rdf:about="http://iiit.serc.com/ontologies/capability.owl#offersCapability">
    <rdfs:domain
        ↪ rdf:resource="http://iiit.serc.com/ontologies/capability.owl#CapabilityInvocation"/>
    <rdfs:range rdf:resource="http://example.com/ontologies/capability.owl#Capability"/>
</owl:ObjectProperty>

<!--
////////////////////////////////////
//
// Data properties
//
////////////////////////////////////
-->

<!-- http://example.com/ontologies/capability.owl#name -->

<owl:DatatypeProperty rdf:about="http://example.com/ontologies/capability.owl#name">
    <rdfs:domain rdf:resource="http://www.w3.org/2002/07/owl#Thing"/>
    <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
</owl:DatatypeProperty>

<!-- http://iiit.serc.com/ontologies/capability.owl#value -->

<owl:DatatypeProperty rdf:about="http://iiit.serc.com/ontologies/capability.owl#value">
    <rdfs:domain rdf:resource="http://www.w3.org/2002/07/owl#Thing"/>
    <rdfs:range rdf:resource="http://www.w3.org/2000/01/rdf-schema#Literal"/>
</owl:DatatypeProperty>

```

```

<!--
////////////////////////////////////
//
// Classes
//
////////////////////////////////////
-->

<!-- http://example.com/ontologies/capability.owl#Action -->

<owl:Class rdf:about="http://example.com/ontologies/capability.owl#Action"/>

<!-- http://example.com/ontologies/capability.owl#Activity -->

<owl:Class rdf:about="http://example.com/ontologies/capability.owl#Activity">
  <rdfs:subClassOf rdf:resource="http://www.w3.org/2002/07/owl#Thing"/>
</owl:Class>

<!-- http://example.com/ontologies/capability.owl#Actuator -->

<owl:Class rdf:about="http://example.com/ontologies/capability.owl#Actuator">
  <rdfs:subClassOf rdf:resource="http://example.com/ontologies/capability.owl#Device"/>
</owl:Class>

<!-- http://example.com/ontologies/capability.owl#Capability -->

<owl:Class rdf:about="http://example.com/ontologies/capability.owl#Capability">

```

```

    <rdfs:subClassOf rdf:resource="http://www.w3.org/2002/07/owl#Thing" />
    <rdfs:comment>Class encapsulating the Requirement and Skill, depending upon whether it is
    ↪ exposed by Process entity or Resource entity, respectively.</rdfs:comment>
</owl:Class>

<!-- http://example.com/ontologies/capability.owl#Command -->

<owl:Class rdf:about="http://example.com/ontologies/capability.owl#Command">
    <rdfs:subClassOf rdf:resource="http://example.com/ontologies/capability.owl#Action" />
</owl:Class>

<!-- http://example.com/ontologies/capability.owl#Device -->

<owl:Class rdf:about="http://example.com/ontologies/capability.owl#Device">
    <rdfs:subClassOf rdf:resource="http://www.w3.org/2002/07/owl#Thing" />
    <rdfs:comment>Class representing a device.</rdfs:comment>
</owl:Class>

<!-- http://example.com/ontologies/capability.owl#Interaction -->

<owl:Class rdf:about="http://example.com/ontologies/capability.owl#Interaction" />

<!-- http://example.com/ontologies/capability.owl#Interface -->

<owl:Class rdf:about="http://example.com/ontologies/capability.owl#Interface" />

<!-- http://example.com/ontologies/capability.owl#Response -->

```

```

<owl:Class rdf:about="http://example.com/ontologies/capability.owl#Response">
  <rdfs:subClassOf rdf:resource="http://example.com/ontologies/capability.owl#Action"/>
</owl:Class>

<!-- http://example.com/ontologies/capability.owl#Sensor -->

<owl:Class rdf:about="http://example.com/ontologies/capability.owl#Sensor">
  <rdfs:subClassOf rdf:resource="http://example.com/ontologies/capability.owl#Device"/>
</owl:Class>

<!-- http://example.com/ontologies/capability.owl#SessionType -->

<owl:Class rdf:about="http://example.com/ontologies/capability.owl#SessionType">
  <rdfs:subClassOf rdf:resource="http://www.w3.org/2002/07/owl#Thing"/>
  <rdfs:comment>Class representing the session type (interaction protocol) of a
  ↪ device.</rdfs:comment>
</owl:Class>

<!-- http://example.com/ontologies/capability.owl#Workflow -->

<owl:Class rdf:about="http://example.com/ontologies/capability.owl#Workflow"/>

<!-- http://iiit.searc.com/ontologies/capability.owl#Alarm -->

<owl:Class rdf:about="http://iiit.searc.com/ontologies/capability.owl#Alarm">
  <rdfs:subClassOf rdf:resource="http://example.com/ontologies/capability.owl#Action"/>
</owl:Class>

```

```

<!-- http://iiit.serc.com/ontologies/capability.owl#CapabilityInvocation -->

<owl:Class rdf:about="http://iiit.serc.com/ontologies/capability.owl#CapabilityInvocation"/>


<!-- http://iiit.serc.com/ontologies/capability.owl#Datastream -->

<owl:Class rdf:about="http://iiit.serc.com/ontologies/capability.owl#Datastream">
    <rdfs:subClassOf rdf:resource="http://example.com/ontologies/capability.owl#Action"/>
</owl:Class>


<!-- http://iiit.serc.com/ontologies/capability.owl#Event -->

<owl:Class rdf:about="http://iiit.serc.com/ontologies/capability.owl#Event">
    <rdfs:subClassOf rdf:resource="http://example.com/ontologies/capability.owl#Action"/>
</owl:Class>


<!-- http://iiit.serc.com/ontologies/capability.owl#Property -->

<owl:Class rdf:about="http://iiit.serc.com/ontologies/capability.owl#Property">
    <rdfs:subClassOf rdf:resource="http://www.w3.org/2002/07/owl#Thing"/>
</owl:Class>


<!--
////////////////////////////////////
//
// General axioms
//
////////////////////////////////////
-->

```

```

<rdf:Description>
  <rdf:type rdf:resource="http://www.w3.org/2002/07/owl#AllDifferent"/>
  <owl:distinctMembers rdf:resource="http://www.w3.org/1999/02/22-rdf-syntax-ns#nil"/>
</rdf:Description>

<!--
////////////////////////////////////
//
// Rules
//
////////////////////////////////////
-->

<rdf:Description rdf:about="http://iiit.searc.com/ontologies/capability.owl#d">
  <rdf:type rdf:resource="http://www.w3.org/2003/11/swrl#Variable"/>
</rdf:Description>
<rdf:Description rdf:about="http://iiit.searc.com/ontologies/capability.owl#ci">
  <rdf:type rdf:resource="http://www.w3.org/2003/11/swrl#Variable"/>
</rdf:Description>
<rdf:Description rdf:about="http://iiit.searc.com/ontologies/capability.owl#c">
  <rdf:type rdf:resource="http://www.w3.org/2003/11/swrl#Variable"/>
</rdf:Description>
<rdf:Description>
  <swrla:isRuleEnabled
    ↪ rdf:datatype="http://www.w3.org/2001/XMLSchema#boolean">true</swrla:isRuleEnabled>
  <rdfs:comment>Infer if a device has capability</rdfs:comment>
  <rdfs:label>CapabilityInferencingRule</rdfs:label>
  <rdf:type rdf:resource="http://www.w3.org/2003/11/swrl#Imp"/>
  <swrl:body>
    <rdf:Description>
      <rdf:type rdf:resource="http://www.w3.org/2003/11/swrl#AtomList"/>
      <rdf:first>
        <rdf:Description>
          <rdf:type rdf:resource="http://www.w3.org/2003/11/swrl#ClassAtom"/>
          <swrl:classPredicate
            ↪ rdf:resource="http://example.com/ontologies/capability.owl#Device"/>

```

```

        <swrl:argument1
↳ rdf:resource="http://iiit.serc.com/ontologies/capability.owl#d"/>
        </rdf:Description>
    </rdf:first>
    <rdf:rest>
        <rdf:Description>
            <rdf:type rdf:resource="http://www.w3.org/2003/11/swrl#AtomList"/>
            <rdf:first>
                <rdf:Description>
                    <rdf:type rdf:resource="http://www.w3.org/2003/11/swrl#ClassAtom"/>
                    <swrl:classPredicate
↳ rdf:resource="http://iiit.serc.com/ontologies/capability.owl#CapabilityInvocation"/>
                    <swrl:argument1
↳ rdf:resource="http://iiit.serc.com/ontologies/capability.owl#ci"/>
                    </rdf:Description>
                </rdf:first>
                <rdf:rest>
                    <rdf:Description>
                        <rdf:type rdf:resource="http://www.w3.org/2003/11/swrl#AtomList"/>
                        <rdf:first>
                            <rdf:Description>
                                <rdf:type
↳ rdf:resource="http://www.w3.org/2003/11/swrl#ClassAtom"/>
                                <swrl:classPredicate
↳ rdf:resource="http://example.com/ontologies/capability.owl#Capability"/>
                                <swrl:argument1
↳ rdf:resource="http://iiit.serc.com/ontologies/capability.owl#c"/>
                                </rdf:Description>
                            </rdf:first>
                            <rdf:rest>
                                <rdf:Description>
                                    <rdf:type
↳ rdf:resource="http://www.w3.org/2003/11/swrl#AtomList"/>
                                    <rdf:first>
                                        <rdf:Description>
                                            <rdf:type
↳ rdf:resource="http://www.w3.org/2003/11/swrl#IndividualPropertyAtom"/>

```

```

        <swrl:propertyPredicate
↳ rdf:resource="http://iiit.serc.com/ontologies/capability.owl#invokesCapability"/>
        <swrl:argument1
↳ rdf:resource="http://iiit.serc.com/ontologies/capability.owl#d"/>
        <swrl:argument2
↳ rdf:resource="http://iiit.serc.com/ontologies/capability.owl#ci"/>
        </rdf:Description>
    </rdf:first>
    <rdf:rest>
        <rdf:Description>
            <rdf:type
↳ rdf:resource="http://www.w3.org/2003/11/swrl#AtomList"/>
            <rdf:first>
                <rdf:Description>
                    <rdf:type
↳ rdf:resource="http://www.w3.org/2003/11/swrl#IndividualPropertyAtom"/>
                    <swrl:propertyPredicate
↳ rdf:resource="http://iiit.serc.com/ontologies/capability.owl#offersCapability"/>
                    <swrl:argument1
↳ rdf:resource="http://iiit.serc.com/ontologies/capability.owl#ci"/>
                    <swrl:argument2
↳ rdf:resource="http://iiit.serc.com/ontologies/capability.owl#c"/>
                </rdf:Description>
            </rdf:first>
            <rdf:rest>
↳ rdf:resource="http://www.w3.org/1999/02/22-rdf-syntax-ns#nil"/>
            </rdf:Description>
        </rdf:rest>
    </rdf:Description>
</rdf:rest>
</rdf:Description>
</rdf:rest>
</rdf:Description>
</rdf:rest>
</rdf:Description>
</rdf:rest>
</rdf:Description>
</swrl:body>
<swrl:head>
    <rdf:Description>

```

```

        <rdf:type rdf:resource="http://www.w3.org/2003/11/swrl#AtomList"/>
        <rdf:first>
            <rdf:Description>
                <rdf:type rdf:resource="http://www.w3.org/2003/11/swrl#IndividualPropertyAtom"/>
                <swrl:propertyPredicate
↳ rdf:resource="http://example.com/ontologies/capability.owl#hasCapability"/>
                <swrl:argument1
↳ rdf:resource="http://iiit.serc.com/ontologies/capability.owl#d"/>
                <swrl:argument2
↳ rdf:resource="http://iiit.serc.com/ontologies/capability.owl#c"/>
            </rdf:Description>
        </rdf:first>
        <rdf:rest rdf:resource="http://www.w3.org/1999/02/22-rdf-syntax-ns#nil"/>
    </rdf:Description>
</swrl:head>
</rdf:Description>
</rdf:RDF>

<!-- Generated by the OWL API (version 4.5.25.2024-12-15T19:15:49Z) https://github.com/owlcs/owlapi -->

```

Algorithm

Algorithm 1 Algorithm for Composing Controllers

```
1: function COMPOSEMACHINES(machines, capabilities, session types)
2:   seq_list  $\leftarrow$  list of machines with sequential capabilities
3:   par_list  $\leftarrow$  list of machines with parallel capabilities
4:   composed_seq  $\leftarrow$  empty list
5:   for all  $m \in \textit{seq\_list}$  do
6:     m_session  $\leftarrow$  session type of machine  $m$ 
7:     m_capability  $\leftarrow$  capability of machine  $m$ 
8:     if previous machine in composed_seq has output that satisfies m_capability then
9:       append  $m$  to composed_seq
10:    else
11:      session_input  $\leftarrow$  input from the previous machine's output
12:      session_output  $\leftarrow$  run m_session with session_input
13:      if session_output satisfies m_capability then
14:        append  $m$  to composed_seq
15:      else
16:        raise SessionTypeError
17:      end if
18:    end if
19:  end for
20:  for all  $m \in \textit{par\_list}$  do
21:    m_session  $\leftarrow$  session type of machine  $m$ 
22:    m_capability  $\leftarrow$  capability of machine  $m$ 
23:    session_input  $\leftarrow$  input from the previous machine's output
24:    session_output  $\leftarrow$  run m_session with session_input
25:    if session_output satisfies m_capability then
26:      append  $m$  to composed_seq
27:    else
28:      raise SessionTypeError
29:    end if
30:  end for
31:  return composed_seq
32: end function
```

SPARQL Queries

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?subject ?label
WHERE {
    ?subject a <http://example.com/ontologies/capability.owl#Workflow> .
    ?subject rdfs:label ?label
}
```

Listing 1: SPARQL query to fetch suitable workflow

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
select DISTINCT ?activities ?neededCapability ?availableCapability ?device
    ↪ ?interface where {
    ?s a <http://example.com/ontologies/capability.owl#Workflow>.
    ?s <http://example.com/ontologies/capability.owl#hasActivitySequence> ?sequence.
    ?sequence rdf:rest*/rdf:first ?activities .
    ?activities <http://example.com/ontologies/capability.owl#requiredCapability>
    ↪ ?neededCapability .

    ?device <http://example.com/capabilities> ?availableCapability .
    ?device <http://example.com/deviceInterface> ?interface .
    ?availableCapability <http://example.com/capability> ?availableCapabilityName
    FILTER(?neededCapability=?availableCapabilityName)
```

```
    FILTER(?s=<{workflowURI}>)  
}
```

Listing 2: SPARQL query to fetch activities in a workflow and required capabilities

```
PREFIX ex: <http://example.com/>  
SELECT ?device ?capabilityName ?capabilities  
WHERE {  
    ?device ex:capabilities ?capabilities .  
    ?capabilities ex:capability ?capabilityName .  
    FILTER(?capabilities = <{capabilities}>)  
}
```

Listing 3: SPARQL query to fetch devices offering required capabilities

```
PREFIX ex: <http://example.com/>  
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>  
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>  
  
SELECT ?capabilities ?sessionAction ?type WHERE {  
    ?device ex:capabilities ?capabilities .  
    ?capabilities ex:capability ?capabilityName.  
    ?capabilities ex:sessionType ?sessionSeq .  
    ?sessionSeq rdf:rest*/rdf:first ?sessionAction .  
    ?sessionAction a ?type.  
  
    FILTER(?device=<{device}>)  
    FILTER(?capabilities=<{capability}>)  
}
```

Listing 4: SPARQL query to fetch interaction protocols for a capability

K-IDE - Reproducibility Configuration

Table 25: Tooling and Environment Details for Reproducibility

Component	Version / Details
K-IDE Platform	Custom Implementation
Eclipse IDE	Version 2023-03 (4.27)
Xtext Framework	Version 2.30
Sirius Framework	Version 7.0
Graph Database	GraphDB Version 10.1.1 (Ontotext)
Simulation Environment	GAZEBO Simulator Version 11.11
Robotics Middleware	ROS (Robot Operating System) Version Noetic Ninjemys
Operating System	Ubuntu 20.04 LTS
Hardware Environment	Intel Core i7 CPU, 32GB RAM
Knowledge Base Size	80 devices modeled, each with approx. 20 properties, 20 roles, and 1 skill per node

Glossary

1. **Adaptive Systems:** Systems that can change their behavior and structure dynamically.
2. **Agile Development:** Approach to software development emphasizing flexibility and collaboration.
3. **API (Application Programming Interface):** Set of rules for building software applications.
4. **Automated Design:** Utilization of algorithms for designing control systems with minimal human intervention.
5. **Capability Ontology:** Structured representation of the knowledge about devices' or components' capabilities.
6. **Cloud Computing:** Delivery of computing services over the Internet.
7. **Context Awareness:** Ability of a system to gather information about its environment.
8. **Continuous Integration:** Development practice requiring developers to integrate code into a shared repository several times a day.
9. **Control System Engineering:** Design and implementation of systems controlling physical processes.
10. **Cyber-Physical Systems:** Integrating computer algorithms with physical processes.
11. **Data Mining:** Process of discovering patterns in large datasets.
12. **Debugging Tools:** Software tools for identifying and fixing system errors.
13. **Digital Twin:** Digital representation of a physical object or system.
14. **Domain Expertise:** Expert knowledge in a specific subject area.
15. **Dynamic Reconfiguration:** On-the-fly changes to a system's configuration.
16. **Efficiency:** Effectiveness of a system with minimum expenditure of time and resources.

17. **Embedded Systems:** Computer systems with dedicated functions within larger systems.
18. **Fault Tolerance:** Continuation of system operation despite component failure.
19. **Flexibility:** Adaptability of a system to altered circumstances or conditions.
20. **Human-Computer Interaction:** Study of interaction between people and computers.
21. **IDE (Integrated Development Environment):** Comprehensive software development application.
22. **Integration Testing:** Testing of combined parts of an application to determine if they function correctly together.
23. **Interoperability:** Ability of systems to work together.
24. **IoT (Internet of Things):** Network of interconnected physical devices.
25. **Knowledge Base:** Centralized repository for information, such as data, information, rules.
26. **Knowledge-Driven Approaches:** Methods leveraging structured and unstructured knowledge.
27. **Knowledge Graph:** Graph-based structure representing knowledge in interconnected nodes and edges.
28. **Lifecycle Management:** Management of a product or system from inception through disposal.
29. **Machine Learning:** Algorithms that improve through experience and data.
30. **Maintenance:** Keeping a system in working order.
31. **Middleware:** Software providing services to software applications beyond OS capabilities.
32. **Ontology:** Formal naming and definition of types, properties, and interrelationships of entities.
33. **Open Source Software:** Software with source code accessible to the public.
34. **Optimization:** Process of making something as effective as possible.
35. **Qualitative Analysis:** Examination of non-numeric data.
36. **Quantitative Analysis:** Examination of measurable data.

- 37. **Real-Time Collaboration:** Simultaneous teamwork often supported by collaborative tools.
- 38. **Real-Time Systems:** Systems that require immediate response.
- 39. **Reliability:** Ability of a system to function under stated conditions for a specified period.
- 40. **Requirements Engineering:** Process of defining, documenting, and maintaining requirements.
- 41. **Robotic Pick-and-Place System:** Robots performing warehouse tasks.
- 42. **Robustness:** Ability of a system to handle errors during execution.
- 43. **Scalability:** Ability to handle increased loads by adding resources.
- 44. **Semantic Web:** Extension of the World Wide Web that enables machine understanding of content.
- 45. **Sensor Integration:** Combining different sensor inputs for a cohesive understanding.
- 46. **Simulation:** Imitation of a situation or process.
- 47. **Smart Meeting Room:** Intelligent devices in a meeting room adapting to user needs.
- 48. **Software Architecture:** High-level structure of a software system.
- 49. **Temperature Control:** Regulation of temperature within a defined environment.
- 50. **Usability:** Ease of use and learnability of a system.
- 51. **User Interviews:** Qualitative research to gain insights into user experiences.
- 52. **Version Control:** System recording file changes for recalling specific versions.
- 53. **Virtualization:** Creating virtual resources (servers, desktops, etc.).
- 54. **Visualization Tools:** Tools to create visual representations of data or processes.
- 55. **Zigbee:** Specification for high-level communication protocols using low-power digital radios.

Bibliography

- [1] Christian Krupitzer, Felix Maximilian Roth, Sebastian VanSyckel, Gregor Schiele, and Christian Becker. A survey on engineering approaches for self-adaptive systems. *Pervasive and Mobile Computing*, 17:184–206, 2015.
- [2] Samir K Srivastava. *Industry 4.0. Lucknow: BHU Engineer's Alumni*, 2016.
- [3] Alejandro Germán Frank, Lucas Santos Dalenogare, and Néstor Fabián Ayala. Industry 4.0 technologies: Implementation patterns in manufacturing companies. *International Journal of Production Economics*, 210:15–26, 2019.
- [4] Jon Hemmerdinger and David Kaminski-Morrow. Et302 crew'could not control aircraft': Ethiopian transport ministry says pilots were unable to prevent" uncommanded nose-down conditions in 737 max. *Flight International*, 2019.
- [5] Nilay K Roy, Michael A Ridge, Scott E Lennox, Rami Mangoubi, and Murali V Chaparala. Distributed system for management and control of aerial vehicle air traffic, April 11 2019. US Patent App. 16/153,241.
- [6] Andrey S Prokhorov, Maksim A Chudinov, and Sergei E Bondarev. Control systems software implementation using open source scada-system openscada. In *2018 IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering (EIConRus)*, pages 220–222. Ieee, 2018.
- [7] Shen Yin, Juan J Rodriguez-Andina, and Yuchen Jiang. Real-time monitoring and control of industrial cyberphysical systems: With integrated plant-wide monitoring and control framework. *IEEE Industrial Electronics Magazine*, 13(4):38–47, 2019.
- [8] Martin Törngren and Ulf Sellgren. Complexity challenges in development of cyber-physical systems. In *Principles of Modeling*, pages 478–503. Springer, 2018.
- [9] PJ Hall. The square kilometre array: An international engineering perspective. In *The Square Kilometre Array: An Engineering Perspective*, pages 5–16. Springer, 2005.
- [10] LJ Reinders. The international thermonuclear experimental reactor. In *The Fairy Tale of Nuclear Fusion*, pages 241–277. Springer, 2021.

- [11] Xin Tan, Minghui Zhou, and Brian Fitzgerald. Scaling open source communities: an empirical study of the linux kernel. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, pages 1222–1234, 2020.
- [12] Loh Poh Chuan, Ayob Johari, Mohd Helmy Abd Wahab, Danial Md Nor, Nik Shahidah Afifi Md Taujuddin, and Mohd Erdi Ayob. An rfid warehouse robot. In *2007 International Conference on Intelligent and Advanced Systems*, pages 451–456. Ieee, 2007.
- [13] Jiafeng Zhang, Hongyi Li, Georg Frey, and Zhiwu Li. Reconfiguration control of dynamic reconfigurable discrete event systems based on ncess. *IEEE Transactions on Control Systems Technology*, 2019.
- [14] Rajesh Pansare, Gunjan Yadav, and Madhukar R Nagare. Reconfigurable manufacturing system: a systematic review, meta-analysis and future research directions. *Journal of Engineering, Design and Technology*, 21(1):228–265, 2023.
- [15] SAE International. Sae j1772 standard for electric vehicle and reconfiguration processes. Technical report, SAE International, 2019. URL <https://www.sae.org/standards>.
- [16] McKinsey & Company. Automotive industry reconfiguration trends. Technical report, McKinsey & Company, 2020. URL <https://www.mckinsey.com/industries/automotive-and-assembly/our-insights>.
- [17] International Energy Agency (IEA). Energy efficiency and power plant reconfigurations. Technical report, International Energy Agency (IEA), 2020. URL <https://www.iea.org/reports/powerplant-efficiency-2020>.
- [18] McKinsey & Company. Energy system reconfigurations and optimization. Technical report, McKinsey & Company, 2020. URL <https://www.mckinsey.com/industries/electric-power-and-natural-gas/our-insights>.
- [19] F. Franceschini, M. Galetto, et al. Reconfiguration strategies in aerospace manufacturing. *Journal of Engineering Technology*, 2019.
- [20] McKinsey & Company. Mckinsey on semiconductors issue 8. https://www.mckinsey.com/~media/mckinsey/industries/semiconductors/our%20insights/mckinsey%20on%20semiconductors%20issue%208/mck_on_semiconductors_08_2021.pdf, 2021.
- [21] Semiconductor Industry Association (SIA). Sia state of the industry report 2022. https://www.semiconductors.org/wp-content/uploads/2022/11/SIA_State-of-Industry-Report_Nov-2022.pdf, 2022.
- [22] Aparna Saisree Thuluva, Darko Anicic, Sebastian Rudolph, and Malintha Adikari. Semantic node-red for rapid development of interoperable industrial iot applications. *Semantic Web*, 11(6):949–975, 2020.

- [23] Eeva Järvenpää, Niko Siltala, Otto Hylli, and Minna Lanz. The development of an ontology for describing the capabilities of manufacturing resources. *Journal of Intelligent Manufacturing*, 30(2):959–978, 2019.
- [24] Amar Banerjee and Venkatesh Choppella. Knowledge-aided integrated development environment (k-ide) for control software development. *Computing in Science & Engineering*, 2024.
- [25] Kai Petersen, Sairam Vakkalanka, and Ludwik Kuzniarz. Guidelines for conducting systematic mapping studies in software engineering: An update. *Information and Software Technology*, 64:1–18, 2015.
- [26] Yifan Wu. Is a dataframe just a table? In *10th Workshop on Evaluation and Usability of Programming Languages and Tools (PLATEAU 2019)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2020.
- [27] Shyamanta M Hazarika and Uday Shanker Dixit. Robotics: History, trends, and future directions. In *Introduction to Mechanical Engineering*, pages 213–239. Springer, Cham, 2018.
- [28] Richard M Murray, Karl J Astrom, Stephen P Boyd, Roger W Brockett, and Gunter Stein. Future directions in control in an information-rich world. *IEEE Control Systems Magazine*, 23(2):20–33, 2003.
- [29] Moritz Tenorth and Michael Beetz. Knowrob: A knowledge processing infrastructure for cognition-enabled robots. *The International Journal of Robotics Research*, 32(5):566–590, 2013.
- [30] Luis Riazuelo, Moritz Tenorth, Daniel Di Marco, Marta Salas, Dorian Gálvez-López, Lorenz Mösenlechner, Lars Kunze, Michael Beetz, Juan D Tardós, Luis Montano, et al. Roboearth semantic mapping: A cloud enabled knowledge-based approach. *IEEE Transactions on Automation Science and Engineering*, 12(2):432–443, 2015.
- [31] Georg Jäger, Christian A Mueller, Madhura Thosar, Sebastian Zug, and Andreas Birk. Towards robot-centric conceptual knowledge acquisition. *arXiv preprint arXiv:1810.03583*, 2018.
- [32] Alonso Diego, Vicente-Chicote Cristina, Ortiz Francisco, Pastor Juan, and Álvarez Bárbara. V3cmm: A 3-view component meta-model for model-driven robotic software development. *Journal of Software Engineering in Robotics*, 1(1):3–17, 2010.
- [33] Moritz Tenorth, Alexander Clifford Perzylo, Reinhard Lafrenz, and Michael Beetz. The roboearth language: Representing and exchanging knowledge about actions, objects,

- and environments. In *Robotics and Automation (ICRA), 2012 IEEE International Conference on*, pages 1284–1289. Ieee, 2012.
- [34] Morgan Quigley, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y Ng. Ros: an open-source robot operating system. In *ICRA workshop on open source software*, volume 3.2, page 5. Kobe, Japan, 2009.
 - [35] Markus Waibel, Michael Beetz, Javier Civera, Raffaello d’Andrea, Jos Elfring, Dorian Galvez-Lopez, Kai Häussermann, Rob Janssen, JMM Montiel, Alexander Perzylo, et al. Roboearth. *IEEE Robotics & Automation Magazine*, 18(2):69–82, 2011.
 - [36] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8(3-4):279–292, 1992.
 - [37] Chi Jin, Zeyuan Allen-Zhu, Sebastien Bubeck, and Michael I Jordan. Is q-learning provably efficient? *arXiv preprint arXiv:1807.03765*, 2018.
 - [38] Gianpiero Francesca, Manuele Brambilla, Arne Brutschy, Vito Trianni, and Mauro Birattari. Automode: A novel approach to the automatic design of control software for robot swarms. *Swarm Intelligence*, 8(2):89–112, 2014.
 - [39] Enrique Vidal, Franck Thollard, Colin De La Higuera, Francisco Casacuberta, and Rafael C Carrasco. Probabilistic finite-state machines-part i. *IEEE transactions on pattern analysis and machine intelligence*, 27(7):1013–1025, 2005.
 - [40] Valeriy Vyatkin. Intelligent mechatronic components: control system engineering using an open distributed architecture. In *EFTA 2003. 2003 IEEE Conference on Emerging Technologies and Factory Automation. Proceedings (Cat. No. 03TH8696)*, volume 2, pages 277–284. Ieee, 2003.
 - [41] Majid Sorouri. *A Compositional Approach to Control Software Design of Automation Systems Based on Mechatronic Modularity*. PhD thesis, ResearchSpace@ Auckland, 2014.
 - [42] Pierluigi Nuzzo. *Compositional design of cyber-physical systems using contracts*. PhD thesis, UC Berkeley, 2015.
 - [43] Wenbin Dai, Wanqi Huang, and Valeriy Vyatkin. Knowledge-driven service orchestration engine for flexible information acquisition in industrial cyber-physical systems. In *2016 IEEE 25th International Symposium on Industrial Electronics (ISIE)*, pages 1055–1060. Ieee, 2016.
 - [44] Nan Zhou, Di Li, Song Li, Shiyong Wang, and Chengliang Liu. Model-based development of knowledge-driven self-reconfigurable machine control systems. *IEEE Access*, 5: 19909–19919, 2017.

- [45] Gabor Karsai and Janos Sztipanovits. Model-integrated development of cyber-physical systems. In *IFIP International Workshop on Software Technologies for Embedded and Ubiquitous Systems*, pages 46–54. Springer, 2008.
- [46] Puneet Patwari, Subhrojyoti Roy Chaudhuri, Amar Banerjee, Swaminathan Natarajan, and Shreya Pandey. A complementary domain specific design environment aiding sysml. In *2016 IEEE International Symposium on Systems Engineering (ISSE)*, pages 1–8. Ieee, 2016.
- [47] Valeriy Vyatkin. The iec 61499 standard and its semantics. *IEEE Industrial Electronics Magazine*, 3(4):40–48, 2009.
- [48] Michael Beetz, Daniel Beßler, Andrei Haidu, Mihai Pomarlan, Asil Kaan Bozcuoğlu, and Georg Bartels. Know rob 2.0 - a 2nd generation knowledge processing framework for cognition-enabled robotic agents. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 512–519. Ieee, 2018.
- [49] Zhenjun Ming, Anand Balu Nellippallil, Yan Yan, Guoxin Wang, Chung Hyun Goh, Janet K Allen, and Farrokh Mistree. Pdsides - a knowledge-based platform for decision support in the design of engineering systems. *Journal of Computing and Information Science in Engineering*, 18(4), 2018.
- [50] César Augusto Ribeiro dos Santos, Amr Hany Saleh, Tom Schrijvers, and Mike Nicolai. Condense: contract based design synthesis. In *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems (MODELS)*, pages 250–260. Ieee, 2019.
- [51] Adnane Saoud, Pushpak Jagtap, Majid Zamani, and Antoine Girard. Compositional abstraction-based synthesis for cascade discrete-time control systems. *IFAC-PapersOnLine*, 51(16):13–18, 2018.
- [52] Guanghui Zhou, Chao Zhang, Zhi Li, Kai Ding, and Chuang Wang. Knowledge-driven digital twin manufacturing cell towards intelligent manufacturing. *International Journal of Production Research*, 58(4):1034–1051, 2020.
- [53] Ornela Dardha, Elena Giachino, and Davide Sangiorgi. Session types revisited. *Information and Computation*, 256:253–286, 2017.
- [54] Paulo Tabuada. *Verification and control of hybrid systems: a symbolic approach*. Springer Science & Business Media, 2009.
- [55] Louis M Rose, Nicholas Matragkas, Dimitrios S Kolovos, and Richard F Paige. A feature model for model-to-text transformation languages. In *2012 4th International Workshop on Modeling in Software Engineering (MISE)*, pages 57–63. Ieee, 2012.

- [56] Marc’Aurelio Ranzato, Sumit Chopra, Michael Auli, and Wojciech Zaremba. Sequence level training with recurrent neural networks. *arXiv preprint arXiv:1511.06732*, 2015.
- [57] J Richard Büchi. Alonzo church. logic, arithmetic, and automata. proceedings of the international congress of mathematicians, 15–22 august 1962, institut mittag-leffler, djursholm, sweden, 1963, pp. 23–35. *The Journal of Symbolic Logic*, 29(4):210–210, 1964.
- [58] Patrick Maier. Intuitionistic ltl and a new characterization of safety and liveness. In *International Workshop on Computer Science Logic*, pages 295–309. Springer, 2004.
- [59] Martin Leucker and César Sánchez. Regular linear temporal logic. In *International colloquium on theoretical aspects of computing*, pages 291–305. Springer, 2007.
- [60] Aaron Bohy, Véronique Bruyère, Emmanuel Filiot, Naiyong Jin, and Jean-François Raskin. Acacia+, a tool for ltl synthesis. In *International Conference on Computer Aided Verification*, pages 652–657. Springer, 2012.
- [61] Chih-Hong Cheng, Edward A Lee, and Harald Ruess. autocode4: Structural controller synthesis. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 398–404. Springer, 2017.
- [62] Chih-Hong Cheng, Chung-Hao Huang, Harald Ruess, and Stefan Stattelmann. G4ltl-st: Automatic generation of plc programs. In *International Conference on Computer Aided Verification*, pages 541–549. Springer, 2014.
- [63] Barbara Jobstmann and Roderick Bloem. Optimizations for ltl synthesis. In *2006 Formal Methods in Computer Aided Design*, pages 117–124. Ieee, 2006.
- [64] Tichakorn Wongpiromsarn, Ufuk Topcu, Necmiye Ozay, Huan Xu, and Richard M Murray. Tulip: a software toolbox for receding horizon temporal logic planning. In *Proceedings of the 14th international conference on Hybrid systems: computation and control*, pages 313–314, 2011.
- [65] Philipp J Meyer, Salomon Sickert, and Michael Luttenberger. Strix: Explicit reactive synthesis strikes back! In *International Conference on Computer Aided Verification*, pages 578–586. Springer, 2018.
- [66] The reactive synthesis competition. <http://www.syntcomp.org/>, 2019. [Online; accessed 3-April-2020].
- [67] Romain Brenguier, Guillermo A Pérez, Jean-François Raskin, and Ocan Sankur. Absynthe: abstract synthesis from succinct safety specifications. *arXiv preprint arXiv:1407.5961*, 2014.

- [68] Shahar Maoz and Yaniv Sa’ar. Aspectltl: an aspect language for ltl specifications. In *Proceedings of the tenth international conference on Aspect-oriented software development*, pages 19–30, 2011.
- [69] Shahar Maoz and Jan Oliver Ringert. Spectra: a specification language for reactive systems. *arXiv preprint arXiv:1904.06668*, 2019.
- [70] R Taylor and R Braun. Science with the square kilometer array: Motivation, key science projects, standards and assumptions. *arXiv preprint astro-ph/0409274*, 2004.
- [71] Y Gupta, B Ajithkumar, H Kale, S Nayak, S Sabhapathy, S Sureshkumar, RV Swami, JN Chengalur, SK Ghosh, CH Ishwara-Chandra, et al. The upgraded gmrt: opening new windows on the radio universe. *Current Science*, 113(4):707–714, 2017.
- [72] Norbert Holtkamp et al. An overview of the iter project. *Fusion Engineering and Design*, 82(5-14):427–434, 2007.
- [73] Amol Wakankar, Paritosh K Pandya, and Raj Mohan Matteplackel. *Dcsynth: Guided reactive synthesis with soft requirements*, pages 124–142. Springer, 2020.
- [74] Zahra Ramezani, Jonas Krook, Zhennan Fei, Martin Fabian, and Knut Akesson. Comparative case studies of reactive synthesis and supervisory control. In *2019 18th European Control Conference (ECC)*, pages 1752–1759. Ieee, 2019.
- [75] Shahar Maoz and Jan Oliver Ringert. Synthesizing a lego forklift controller in gr (1): A case study. *arXiv preprint arXiv:1602.01172*, 2016.
- [76] Henning Kagermann, Johannes Helbig, Ariane Hellinger, and Wolfgang Wahlster. *Recommendations for implementing the strategic initiative INDUSTRIE 4.0: Securing the future of German manufacturing industry; final report of the Industrie 4.0 Working Group*. Forschungsunion, 2013.
- [77] John Hutchinson, Mark Rouncefield, and Jon Whittle. Model-driven engineering practices in industry. In *Proceedings of the 33rd International Conference on Software Engineering*, pages 633–642, 2011.
- [78] Puneet Patwari, Subhrojyoti R Choudhury, Amar Banerjee, N Swaminathan, and Shreya Pandey. M&c domain map maker: an environment complimenting mde with m&c knowledge and ensuring solution completeness. In *Software and Cyberinfrastructure for Astronomy IV*, volume 9913, page 991341. International Society for Optics and Photonics, 2016.
- [79] Selma Kchir, Saadia Dhouib, Jérémie Tatibouet, Baptiste Gradoussoff, and Max Da Silva Simoes. Robotml for industrial robots: design and simulation of manipulation scenarios.

In *2016 IEEE 21st International Conference on Emerging Technologies and Factory Automation (ETFA)*, pages 1–8. Ieee, 2016.

- [80] Stephan Philippi. Automatic code generation from high-level petri-nets for model driven systems engineering. *Journal of Systems and Software*, 79(10):1444–1455, 2006.
- [81] H. Fadhlillah, Kevin Feichtinger, Kristof Meixner, Lisa Sonnleithner, Rick Rabiser, and Alois Zoitl. Towards multidisciplinary delta-oriented variability management in cyber-physical production systems. *ACM Transactions on Software Engineering and Methodology*, 2022. doi: 10.1145/3510466.3511273.
- [82] Michael Weser, Jürgen Bock, Siwara Schmitt, Alexander Perzylo, and Kathrin Evers. An ontology-based metamodel for capability descriptions. In *2020 25th IEEE international conference on emerging technologies and factory automation (ETFA)*, volume 1, pages 1679–1686. Ieee, 2020.
- [83] Munir Merdan, Timon Hoebert, Erhard List, and Wilfried Lepuschitz. Knowledge-based cyber-physical systems for assembly automation. *Production & Manufacturing Research*, 7(1):223–254, 2019.
- [84] Yazan Alsafi and Valeriy Vyatkin. Ontology-based reconfiguration agent for intelligent mechatronic systems in flexible manufacturing. *Robotics and Computer-Integrated Manufacturing*, 26(4):381–391, 2010.
- [85] Leonard Petnga and Mark Austin. An ontological framework for knowledge modeling and decision support in cyber-physical systems. *Advanced Engineering Informatics*, 30(1): 77–94, 2016.
- [86] Kaja Balzereit and Oliver Niggemann. Autoconf: new algorithm for reconfiguration of cyber-physical production systems. *IEEE Transactions on Industrial Informatics*, 19(1): 739–749, 2022.
- [87] Don Norman. *The design of everyday things: Revised and expanded edition*. Basic books, 2013.
- [88] Amar Banerjee and Venkatesh Choppella. Knowledge driven synthesis using resource-capability semantics for control software design. *IEEE Access*, 11:52527–52539, 2023.
- [89] Subhrojyoti Roy Chaudhuri, Amar Banerjee, N Swaminathan, Venkatesh Choppella, Arpan Pal, and P Balamurali. A knowledge centric approach to conceptualizing robotic solutions. In *Proceedings of the 12th Innovations in Software Engineering Conference (formerly known as India Software Engineering Conference)*, pages 1–11, 2019.

- [90] Amar Banerjee, Venkatesh Choppella, Viswanath Kasturi, Swaminathan Natarajan, Padmalata V Nistala, and Kesav Nori. An attempt at explicating the relationship between knowledge, systems and engineering. In *Proceedings of the 11th Innovations in Software Engineering Conference*, pages 1–11, 2018.
- [91] B Amar, RC Subhrojyoti, B Barnali, R Dhakshinamoorthy, N Rajesh, and C Venkatesh. Knowledge driven approach to auto-generate digital twins for industrial plants. In *Proceedings of the CIKM 2021 Workshops co-located with 30th ACM International Conference on Information and Knowledge Management (CIKM 2021) Gold Coast, Queensland, Australia, November*, pages 1–5, 2021.
- [92] Abdelkarim Erradi, Sriram Anand, and Naveen Kulkarni. Soaf: An architectural framework for service definition and realization. In *2006 IEEE International Conference on Services Computing (SCC'06)*, pages 151–158. Ieee, 2006.
- [93] Ji Han, Hannah Forbes, and Dirk Schaefer. An exploration of the relations between functionality, aesthetics and creativity in design. In *Proceedings of the Design Society: International Conference on Engineering Design*, volume 1, pages 259–268. Cambridge University Press, 2019.
- [94] Reinholt Geelink, Otto W Salomons, Fjodor van Slooten, Fred JAM van Houten, and Huub JJ Kals. Unified feature definition for feature based design and feature based manufacturing. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, volume 17018, pages 517–533. American Society of Mechanical Engineers, 1995.
- [95] Aparna Saisree Thuluva, Darko Anicic, and Sebastian Rudolph. Iot semantic interoperability with device description shapes. In *The Semantic Web: ESWC 2018 Satellite Events: ESWC 2018 Satellite Events, Heraklion, Crete, Greece, June 3-7, 2018, Revised Selected Papers 15*, pages 409–422. Springer, 2018.
- [96] Ian Horrocks, Peter F Patel-Schneider, Harold Boley, Said Tabet, Benjamin Grosz, Mike Dean, et al. Swrl: A semantic web rule language combining owl and ruleml. *W3C Member submission*, 21(79):1–31, 2004.
- [97] Rebecca Wirfs-Brock, Brian Wilkerson, and Lauren Wiener. *Designing Object-Oriented Software*. Prentice-Hall, 1990.
- [98] Laura Daniele, Frank den Hartog, and Jasper Roes. Created in close interaction with the industry: the smart appliances reference (saref) ontology. In *Formal Ontologies Meet Industry: 7th International Workshop, FOMI 2015, Berlin, Germany, August 5, 2015, Proceedings 7*, pages 100–112. Springer, 2015.

- [99] Victor Charpenay, Sebastian Käbisich, and Harald Kosch. Introducing thing descriptions and interactions: An ontology for the web of things. In *Sr+ Swit@ Iswc*, pages 55–66, 2016.
- [100] Armin Haller, Krzysztof Janowicz, Simon JD Cox, Maxime Lefrançois, Kerry Taylor, Danh Le Phuoc, Joshua Lieberman, Raúl García-Castro, Rob Atkinson, and Claus Stadler. The modular ssn ontology: A joint w3c and ogc standard specifying the semantics of sensors, observations, sampling, and actuation. *Semantic Web*, 10:9–32, 2019.
- [101] S Roy Chaudhuri, Amrit Lal Ahuja, Swaminathan Natarajan, and Harrick Vin. Model-driven development of control system software. In *The Low-Frequency Radio Universe*, volume 407, page 402, 2009.
- [102] S Roy Chaudhuri, H Hayatnagarkar, S Natarajan, et al. Integrated monitoring and control specification environment. *Proceedings of ICALEPCS, 2013, San Francisco, CA, USA*, 2013.
- [103] Dave Steinberg, Frank Budinsky, Ed Merks, and Marcelo Paternostro. *EMF: eclipse modeling framework*. Pearson Education, 2008.
- [104] Subhrojyoti Roy Chaudhuri, Swaminathan Natarajan, Amar Banerjee, and Venkatesh Choppella. Methodology to develop domain specific modeling languages. In *Proceedings of the 17th ACM SIGPLAN International Workshop on Domain-Specific Modeling*, pages 1–10, 2019.
- [105] B Amar, RC Subhrojyoti, B Barnali, R Dhakshinamoorthy, N Rajesh, and C Venkatesh. Knowledge driven approach to auto-generate digital twins for industrial plants. *Fourth Workshop on Knowledge-driven Analytics and Systems Impacting Human Quality of Life (KDAH-CIKM-2021)*, 2021.
- [106] Inc. Gartner. Best integrated development environment (ide) software reviews 2023: Gartner peer insights. <https://www.gartner.com/reviews/market/integrated-development-environment-ide-software>, 2023.
- [107] Deborah L McGuinness, Frank Van Harmelen, et al. Owl web ontology language overview. *W3C recommendation*, 10(10):2004, 2004.
- [108] Billy Jonathan, Rafayel Avetyan, and Stan Abeln. Create domain-specific language and syntax checker using xtext. *International Journal of Industrial Research and Applied Engineering*, 4(1):26–32, 2019.
- [109] Eugen Schindler, Klemens Schindler, Federico Tomassetti, and AM Sutii. Language workbench challenge 2016: the jetbrains meta programming system. In *LWC@ SLE 2016*

Language Workbench Challenge, Splash2016, 39 October-4 November 2016, Amsterdam, The Netherlands, 2016.

- [110] Puneet Patwari, G Muralikrishna, Amar Banerjee, and Subhrojyoti Roy Chaudhuri. Towards building reusability in control systems-a journey. In *Software and Cyberinfrastructure for Astronomy IV*, volume 9913, page 991341. International Society for Optics and Photonics, 2015. US Patent App. 16/26,314.
- [111] Anders Wallander, Lana Abadie, Haresh Dave, Franck Di Maio, Hitesh Kumar Gulati, Chandresh Hansalia, Didier Joonekindt, Jean-Yves Journeaux, Wolf-Dieter Klotz, Kirti Mahajan, et al. Iter instrumentation and control - status and plans. *Fusion Engineering and Design*, 85(3-4):529–534, 2010.
- [112] Camila Martínez, Jorge Avarias, and Patricio Galeas. Implementation of a new code generator, based on eclipse modeling framework, for the antennas control devices of the alma and llama observatories. *Aspc*, 522:481, 2020.
- [113] Justin Cooper and Dimitris Kolovos. Engineering hybrid graphical-textual languages with sirius and xtext: Requirements and challenges. In *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, pages 322–325. Ieee, 2019.
- [114] Miguel Morim Oliveira. *Praxis Market Drift*. PhD thesis, Instituto Politecnico do Porto (Portugal), 2020.
- [115] Arthur B Markman. *Knowledge representation*. Psychology Press, 2013.
- [116] Christopher Brewster and Kieron O’Hara. Knowledge representation with ontologies: Present challenges - future possibilities. *International Journal of Human-Computer Studies*, 65(7):563–568, 2007.
- [117] HealthIT.gov. Interoperability needs more than just standards. HealthIT.gov Blog, 2017.
- [118] Finra. Report on finra examination findings. Technical report, Financial Industry Regulatory Authority, 2018.
- [119] McKinsey & Company. Digital manufacturing: The revolution will be virtualized. Technical report, McKinsey Insights, 2020.
- [120] Senthil K Chandrasegaran, Karthik Ramani, Ram D Sriram, Imré Horváth, Alain Bernard, Ramy F Harik, and Wei Gao. The evolution, challenges, and future of knowledge representation in product design systems. *Computer-aided design*, 45(2):204–228, 2013.

- [121] Wim JC Verhagen, Pablo Bermell-Garcia, Reinier EC Van Dijk, and Richard Curran. A critical review of knowledge-based engineering: An identification of research challenges. *Advanced Engineering Informatics*, 26(1):5–15, 2012.
- [122] Zhe Chen, Yuehan Wang, Bin Zhao, Jing Cheng, Xin Zhao, and Zongtao Duan. Knowledge graph completion: A review. *Ieee Access*, 8:192435–192456, 2020.
- [123] James Manyika et al. Big data: The next frontier for innovation, competition, and productivity. Technical report, McKinsey Global Institute, 2011.
- [124] The Economist. Data is giving rise to a new economy. *The Economist*, 2017.
- [125] Bernard Marr. How much data do we create every day? the mind-blowing stats everyone should read. *Forbes*, 21:1–5, 2018.
- [126] Gartner. Top 10 data and analytics technology trends for 2020. Technical report, Gartner Report, 2020.
- [127] Cathy O’Neil. *Weapons of Math Destruction: How Big Data Increases Inequality and Threatens Democracy*. Crown, 2016.
- [128] Michael Chui et al. Notes from the ai frontier: Insights from hundreds of use cases. Technical report, McKinsey Global Institute, 2018.
- [129] Frederick Hayes-Roth, Donald A Waterman, and Douglas B Lenat. *Building expert systems*. Addison-Wesley, 1983.
- [130] Jay Liebowitz. *The handbook of applied expert systems*. cRc Press, 2019.
- [131] M Ross Quillian. Semantic memory. *Semantic Information Processing*, pages 227–270, 1968.
- [132] John F Sowa. Semantic networks. *Encyclopedia of artificial intelligence*, 2:1591–1598, 1987.
- [133] Thomas R Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.
- [134] Rudi Studer, V Richard Benjamins, and Dieter Fensel. Knowledge engineering: Principles and methods. *Data & Knowledge Engineering*, 25(1-2):161–197, 1998.
- [135] Michael Uschold and Michael Gruninger. Ontologies: Principles, methods and applications. *Knowledge Engineering Review*, 11(2):93–136, 1996.
- [136] Lisa Ehrlinger and Wolfram Wöß. Towards a definition of knowledge graphs. *SEMAN-TiCS (Posters, Demos, SuCCESS)*, 2016.

- [137] Heiko Paulheim. Knowledge graph refinement: A survey of approaches and evaluation methods. *Semantic Web*, 8(3):489–508, 2017.
- [138] Antonio Cicchetti, Davide Di Ruscio, Romina Eramo, and Alfonso Pierantonio. Automating co-evolution in model-driven engineering. *IEEE International Conference on Engineering of Complex Computer Systems*, pages 121–130, 2010.
- [139] MathWorks. *Simulink*. The MathWorks, Inc., 2020. URL <https://www.mathworks.com/products/simulink.html>.
- [140] Volkan Aslan and Suleyman Korukoglu. Model-based design and code generation for embedded software. *Procedia Technology*, 1:118–122, 2012.
- [141] Object Management Group. Mda guide version 1.0.1, 2003. URL <https://www.omg.org/cgi-bin/doc?omg/03-06-01.pdf>.
- [142] Anneke G Kleppe, Jos B Warmer, and Wim Bast. *MDA Explained: The Model Driven Architecture: Practice and Promise*. Addison-Wesley Professional, 2003.
- [143] Juha-Pekka Tolvanen and Steven Kelly. Model-driven development challenges and solutions: experiences with domain-specific modelling in industry. In *2016 4th International Conference on Model-Driven Engineering and Software Development (MODEL-SWARD)*, pages 711–719. Ieee, 2016.
- [144] Steven Kelly and Juha-Pekka Tolvanen. *Domain-specific modeling: enabling full code generation*. John Wiley & Sons, 2008.
- [145] Bran Selic. The pragmatics of model-driven development. *IEEE Software*, 20(5):19–25, 2003.
- [146] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, pages 5998–6008, 2017.
- [147] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. Why should i trust you? explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144, 2016.
- [148] Tolga Bolukbasi, Kai-Wei Chang, James Y Zou, Venkatesh Saligrama, and Adam T Kalai. Man is to computer programmer as woman is to homemaker? debiasing word embeddings. In *Advances in Neural Information Processing Systems*, pages 4349–4357, 2016.
- [149] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in nlp. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3645–3650, 2019.

- [150] Miltiadis Allamanis, Earl T Barr, Premkumar Devanbu, and Charles Sutton. A survey of machine learning for big code and naturalness. In *ACM Computing Surveys (CSUR)*, volume 51/4, pages 1–37. Acm, 2018.
- [151] Pierluigi Riti and Pierluigi Riti. External dsl. *Practical Scala DSLs: Real-World Applications Using Domain Specific Languages*, pages 59–69, 2018.
- [152] Waqas Ali, Muhammad Saleem, Bin Yao, Aidan Hogan, and Axel-Cyrille Ngonga Ngomo. A survey of rdf stores & sparql engines for querying knowledge graphs. *The VLDB Journal*, 31:1–26, 2022.
- [153] Google. Knowledge graph: Things, not strings. Google Official Blog, 2016.
- [154] Enrique S Burgos and Eduardo J Adam. Graphical user interface editor for octave applications. *Engineering Reports*, 2(11):e12269, 2020.
- [155] Swaminathan Natarajan and Subhrojyoti Roy Chaudhuri. A systems science basis for compositionality reasoning. In *Proc. Conference on Systems Engineering Research (CSER)*, Redondo Beach, CA, 01 2020.
- [156] Lorenzo Bettini. *Implementing domain-specific languages with Xtext and Xtend*. Packt Publishing Ltd, 2016.
- [157] Ewout Wal. Rosetta antlr: Ultimate grammar extractor. B.S. thesis, University of Twente, 2021.
- [158] Edward Sciore and Edward Sciore. Parsing. *Database Design and Implementation: Second Edition*, pages 239–266, 2020.
- [159] Alena Bastrykina, Vladimir Zyubin, and Andrey Rozov. Developing reflex ide kernel with xtext framework. In *2021 IEEE 22nd International Conference of Young Professionals in Electron Devices and Materials (EDM)*, pages 511–514. Ieee, 2021.
- [160] Jianan Yue. Transition from ebnf to xtext. *Alternation*, 1:1, 2014.
- [161] Lorenzo Bettini. Implementing java-like languages in xtext with xsemantics. In *Proceedings of the 28th Annual ACM Symposium on Applied Computing*, pages 1559–1564, 2013.
- [162] Sven Efftinge, Moritz Eysholdt, Jan Köhnlein, Sebastian Zarnekow, Robert von Massow, Wilhelm Hasselbring, and Michael Hanus. Xbase: implementing domain-specific languages for java. *ACM SIGPLAN Notices*, 48(3):112–121, 2012.
- [163] Carol Fairchild and Thomas L Harman. *ROS robotics by example*. Packt Publishing Ltd, 2016.

- [164] Barrett R. Bryant, Jeff Gray, and Marjan Mernik. Domain-specific software engineering. In *Proceedings of the FSE/SDP Workshop on Future of Software Engineering Research*, FoSER '10, page 65–68, New York, NY, USA, 2010. Association for Computing Machinery. ISBN 9781450304276. doi: 10.1145/1882362.1882376. URL <https://doi.org/10.1145/1882362.1882376>.
- [165] Scott Amyx. Internet of things open-source systems. *Internet of Things and Data Analytics Handbook*, pages 129–146, 2017.
- [166] Stefan Runde, Alexander Fay, and Wagner-Otto Wutzke. Knowledge-based requirement-engineering of building automation systems by means of semantic web technologies. In *2009 7th IEEE International Conference on Industrial Informatics*, pages 267–272. Ieee, 2009.
- [167] Douglas C Schmidt. Model-driven engineering. *Computer-IEEE Computer Society*, 39(2): 25, 2006.
- [168] Claudio Manconi. *Design and implementation of a software abstraction layer to extend the functionality of an automotive SoC with advanced I/O capabilities of a real time microcontroller*. PhD thesis, Politecnico di Torino, 2020.
- [169] Heiko Koziolk, Sten Grüner, Rhaban Hark, Virendra Ashiwal, Sofia Linsbauer, and Nafise Eskandani. Llm-based and retrieval-augmented control code generation. In *Proceedings of the 1st International Workshop on Large Language Models for Code*, pages 22–29, 2024.
- [170] Bianca Wiesmayr, Alois Zoitl, and Rick Rabiser. Assessing the usefulness of a visual programming ide for large-scale automation software. *Software and Systems Modeling*, pages 1–25, 2023.
- [171] Li Da Xu, Weijun He, and Shancang Li. Industrial internet of things: Architecture, advances and challenges. *Computers in Industry*, 101:1–12, 2018. doi: 10.1016/j.compind.2018.04.002.
- [172] James Moyne, Satish Ramaswamy, and Senthil Rathinam. Cybersecurity for smart manufacturing: Challenges and solutions. *Journal of Manufacturing Systems*, 56:61–74, 2020. doi: 10.1016/j.jmsy.2020.04.007.
- [173] João Barata and Ina Kayser. Industry 5.0—past, present, and near future. *Procedia Computer Science*, 219:778–788, 2023.
- [174] C. Yang, V. Vyatkin, and C. Pang. Model-driven development of control software for distributed automation: a survey and an approach. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 44(3):292–305, 2013.

- [175] Martin Wirsing, Matthias Hölzl, Nora Koch, and Philip Mayer. *Software Engineering for Collective Autonomic Systems: The ASCENS Approach*, volume 8998. Springer, 2015.
- [176] Birgit Vogel-Heuser, Daniel Schütz, Timo Frank, and Christoph Legat. Model-driven engineering of manufacturing automation software projects—a sysml-based approach. *Mechatronics*, 24(7):883–897, 2014.
- [177] Kleanthis Thramboulidis. A cyber-physical system-based approach for industrial automation systems. *Computers in Industry*, 72:92–102, 2015.
- [178] Seno Darmawan Panjaitan. *Development process for distributed automation systems based on elementary mechatronic functions*. Shaker, 2008.
- [179] Jens Otto, Steffen Henning, and Oliver Niggemann. Why cyber-physical production systems need a descriptive engineering approach—a case study in plug & produce. *Procedia Technology*, 15:295–302, 2014.
- [180] Pierluigi Nuzzo. *Compositional Design of Cyber-Physical Systems Using Contracts*. PhD thesis, EECS Department, University of California, Berkeley, Aug 2015. URL <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2015/EECS-2015-189.html>.
- [181] Hausi Muller, John Mylopoulos, and Marin Litoiu. Engineering cyber physical systems. In *Proceedings of the 25th Annual International Conference on Computer Science and Software Engineering*, Cascon ’15, pages 328–332, Riverton, NJ, USA, 2015. IBM Corp. URL <http://dl.acm.org/citation.cfm?id=2886444.2886514>.
- [182] Brice Morin, Nicolas Harrand, and Franck Fleurey. Model-based software engineering to tame the iot jungle. *IEEE Software*, 34(1):30–36, 2017.
- [183] Xabier Larrucea, Annie Combelles, John Favaro, and Kunal Taneja. Software engineering for the internet of things. *IEEE Software*, 34(1):24–28, 2017.
- [184] Gabor Karsai, Janos Sztipanovits, Akos Ledeczi, and Ted Bapty. Model-integrated development of embedded software. *Proceedings of the IEEE*, 91(1):145–164, 2003.
- [185] Choulsoo Jang, Seung-Ik Lee, Seung-Woog Jung, Byoungyoul Song, Rockwon Kim, Sunghoon Kim, and Cheol-Hoon Lee. Opros: A new component-based robot software platform. *ETRI journal*, 32(5):646–656, 2010.
- [186] Steffen Henning, Oliver Niggemann, Jens Otto, and Sebastian Schriegel. A descriptive engineering approach for cyber-physical systems. In *Proceedings of the 2014 IEEE Emerging Technology and Factory Automation (ETFA)*, pages 1–4. Ieee, 2014.

- [187] Taoufik Ben Hassine, Oualid Khayati, and Henda Ben Ghezala. An iot domain meta-model and an approach to software development of iot solutions. In *2017 International Conference on Internet of Things, Embedded Systems and Communications (IINTEC)*, pages 32–37. Ieee, 2017.
- [188] Robert Harrison, Daniel Vera, and Bilal Ahmad. Engineering methods and tools for cyber-physical automation systems. *Proceedings of the IEEE*, 104(5):973–985, 2016.
- [189] David Gay, Philip Levis, Robert Von Behren, Matt Welsh, Eric Brewer, and David Culler. The nesc language: A holistic approach to networked embedded systems. *Acm Sigplan Notices*, 49(4):41–51, 2014.
- [190] Rogério De Lemos, Holger Giese, Hausi A Müller, Mary Shaw, Jesper Andersson, Marin Litoiu, Bradley Schmerl, Gabriel Tamura, Norha M Villegas, Thomas Vogel, et al. Software engineering for self-adaptive systems: A second research roadmap. In *Software Engineering for Self-Adaptive Systems II*, pages 1–32. Springer, 2013.
- [191] Federico Ciccozzi, Ivica Crnkovic, Davide Di Ruscio, Ivano Malavolta, Patrizio Pelliccione, and Romina Spalazzese. Model-driven engineering for mission-critical iot systems. *IEEE software*, 34(1):46–53, 2017.
- [192] Arquimedes Canedo, Mohammad Abdullah Al Faruque, and Jan H Richter. Multi-disciplinary integrated design automation tool for automotive cyber-physical systems. In *2014 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1–2. Ieee, 2014.
- [193] Tomas Bures, Danny Weyns, Christian Berger, Stefan Biffl, Marian Daun, Thomas Gabor, David Garlan, Ilias Gerostathopoulos, Christine Julien, Filip Krikava, et al. Software engineering for smart cyber-physical systems—towards a research agenda: report on the first international workshop on software engineering for smart cps. *ACM SIGSOFT Software Engineering Notes*, 40(6):28–32, 2015.
- [194] Herman Bruyninckx, Markus Klotzbücher, Nico Hochgeschwender, Gerhard Kraetzschmar, Luca Gherardi, and Davide Brugali. The brics component model: a model-based development paradigm for complex robotics software systems. In *Proceedings of the 28th Annual ACM Symposium on Applied Computing*, pages 1758–1764. Acn, 2013.
- [195] Amel Bennaceur, Carlo Ghezzi, Kenji Tei, Timo Kehler, Danny Weyns, Radu Calinescu, Shahram Dustdar, Zhenjiang Hu, Shinichi Honiden, Fuyuki Ishikawa, Zhi Jin, Jeffrey Kramer, Marin Litoiu, Michele Loreti, Gabriel Moreno, Hausi Müller, Laura Nenzi, Bashar Nuseibeh, Liliana Pasquale, Wolfgang Reisig, Heinz Schmidt, Christos

- Tsigkanos, and Haiyan Zhao. Modelling and Analysing Resilient Cyber-Physical Systems. In *14th Symposium on Software Engineering for Adaptive and Self-Managing Systems*, Montréal, Canada, May 2019. URL <https://hal.archives-ouvertes.fr/hal-02104308>.
- [196] Samoel Mirachi, Valdir da Costa Guerra, Adilson Marques da Cunha, Luiz Alberto Vieira Dias, and Emilia Villani. Applying agile methods to aircraft embedded software: an experimental analysis. *Software: Practice and Experience*, 47(11):1465–1484, 2017.
- [197] Ulrik Eklund and Jan Bosch. Applying agile development in mass-produced embedded systems. In *International Conference on Agile Software Development*, pages 31–46. Springer, 2012.
- [198] Clément Duffau, Bartosz Grabiec, and Mireille Blay-Fornarino. Towards embedded system agile development challenging verification, validation and accreditation: Application in a healthcare company. In *2017 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, pages 82–85. Ieee, 2017.
- [199] João Pimentel, Márcia Lucena, Jaelson Castro, Carla Silva, Emanuel Santos, and Fernanda Alencar. Deriving software architectural models from requirements models for adaptive systems: the stream-a approach. *Requirements Engineering*, 17(4):259–281, 2012.
- [200] Elizabeth Cha, Anca D Dragan, and Siddhartha S Srinivasa. Perceived robot capability. In *2015 24th IEEE International Symposium on Robot and Human Interactive Communication (RO-MAN)*, pages 541–548. Ieee, 2015.
- [201] Raian Ali, Fabiano Dalpiaz, and Paolo Giorgini. A goal-based framework for contextual requirements modeling and analysis. *Requirements Engineering*, 15(4):439–458, 2010.
- [202] Ji Zhang and Betty HC Cheng. Model-based development of dynamically adaptive software. In *Proceedings of the 28th international conference on Software engineering*, pages 371–380. Acm, 2006.
- [203] Hongyan Xia, Jonathan Woodruff, Hadrien Barral, Lawrence Esswood, Alexandre Joanou, Robert Kovacsics, David Chisnall, Michael Roe, Brooks Davis, Edward Napierala, et al. Cherirtos: A capability model for embedded devices. In *International Conference on Computer Design*, 2018.
- [204] Andreas Wortmann and Martin Beet. Domain specific languages for efficient satellite control software development. In *DASIA 2016-Data Systems In Aerospace*, volume 736, 2016.
- [205] Nikolaus Vahrenkamp, Mirko Wächter, Manfred Kröhnert, Kai Welke, and Tamim Asfour. The robot software framework armarx. *it-Information Technology*, 57(2):99–111, 2015.

- [206] Kleanthis C Thramboulidis. Using uml in control and automation: a model driven approach. In *2nd IEEE International Conference on Industrial Informatics, 2004. INDIN'04. 2004*, pages 587–593. Ieee, 2004.
- [207] Thomas Strasser, Christoph Sunder, and Antonio Valentini. Model-driven embedded systems design environment for the industrial automation sector. In *2008 6th IEEE International Conference on Industrial Informatics*, pages 1120–1125. Ieee, 2008.
- [208] Azamat Shakhimardanov, Nico Hochgeschwender, and Gerhard K Kraetzschmar. Component models in robotics software. In *Proceedings of the 10th Performance Metrics for Intelligent Systems Workshop*, pages 82–87. Acm, 2010.
- [209] Syed Imran Shafiq, Cesar Sanin, Edward Szczerbicki, and Carlos Toro. Virtual engineering object/virtual engineering process: a specialized form of cyber physical system for industrie 4.0. *Procedia Computer Science*, 60:1146–1155, 2015.
- [210] Christian Schlegel, Andreas Steck, and Alex Lotz. Robotic software systems: From code-driven to model-driven software development. In *Robotic Systems-Applications, Control and Programming*. InTech, 2012.
- [211] Christian Schlegel, Andreas Steck, Davide Brugali, and Alois Knoll. Design abstraction and processes in robotics: From code-driven to model-driven engineering. In *International Conference on Simulation, Modeling, and Programming for Autonomous Robots*, pages 324–335. Springer, 2010.
- [212] Jan Oliver Ringert, Bernhard Rumpe, and Andreas Wortmann. A case study on model-based development of robotic systems using montiarc with embedded automata. *Tagungsband des Dagstuhl-Workshops*, page 30, 2014.
- [213] Giorgio Metta, Paul Fitzpatrick, and Lorenzo Natale. Yarp: yet another robot platform. *International Journal of Advanced Robotic Systems*, 3(1):8, 2006.
- [214] M Masin, Francesca Palumbo, H Myrhaug, JA de Oliveira Filho, M Pastena, Maxime Pelcat, Luigi Raffo, Francesco Regazzoni, AA Sanchez, A Toffetti, et al. Cross-layer design of reconfigurable cyber-physical systems. In *Proceedings of the Conference on Design, Automation & Test in Europe*, pages 740–745. European Design and Automation Association, 2017.
- [215] Damian M Lyons, Ronald C Arkin, Paramesh Nirmal, Shu Jiang, and T-M Liu. A software tool for the design of critical robot missions with performance guarantees. *Procedia Computer Science*, 16:888–897, 2013.
- [216] Jay Lee, Behrad Bagheri, and Hung-An Kao. A cyber-physical systems architecture for industry 4.0-based manufacturing systems. *Manufacturing letters*, 3:18–23, 2015.

- [217] Hitoshi Komoto and Tetsuo Tomiyama. A framework for computer-aided conceptual design and its application to system architecting of mechatronics products. *Computer-Aided Design*, 44(10):931–946, 2012.
- [218] Dayang NA Jawawi, Rosbi Mamat, and Safaai Deris. A component-oriented programming for embedded mobile robot software. *International Journal of Advanced Robotic Systems*, 4(3):40, 2007.
- [219] David Hästbacka and Seppo Kuikka. Semantics enhanced engineering and model reasoning for control application development. *Multimedia tools and applications*, 65(1):47–62, 2013.
- [220] David Hästbacka, Timo Vepsäläinen, and Seppo Kuikka. Model-driven development of industrial process control applications. *Journal of Systems and Software*, 84(7):1100–1113, 2011.
- [221] Arne Haber. *MontiArc-Architectural Modeling and Simulation of Interactive Distributed Systems*, volume 24. Shaker Verlag GmbH, 2016.
- [222] Giovanni Godena, Tomaž Lukman, and Gregor Kandare. A new approach to control systems software development. In *Case Studies in Control*, pages 363–406. Springer, 2013.
- [223] P Fritzson. A cyber-physical modeling language and the openmodelica environment. In *Proceedings of the International Wireless Communications and Mobile Computing Conference, Turkey, July*, pages 4–8, 2011.
- [224] John Fitzgerald, Carl Gamble, Peter Gorm Larsen, Kenneth Pierce, and Jim Woodcock. Cyber-physical systems design: formal foundations, methods and integrated tool chains. In *2015 IEEE/ACM 3rd FME Workshop on Formal Methods in Software Engineering*, pages 40–46. Ieee, 2015.
- [225] Antonio Filieri, Martina Maggio, Konstantinos Angelopoulos, Nicolás D’Ippolito, Ilias Gerostathopoulos, Andreas Berndt Hempel, Henry Hoffmann, Pooyan Jamshidi, Evangelia Kalyvianaki, Cristian Klein, et al. Software engineering meets control theory. In *Proceedings of the 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*, pages 71–82. IEEE Press, 2015.
- [226] Ahmed Elkhodary, Naeem Esfahani, and Sam Malek. Fusion: a framework for engineering self-tuning self-adaptive software systems. In *Proceedings of the eighteenth ACM SIGSOFT international symposium on Foundations of software engineering*, pages 7–16. Acm, 2010.

- [227] Giovanni Di Orio, José Barata, Carlos Sousa, and Luis Flores. Control system software design methodology for automotive industry. In *2013 IEEE International Conference on Systems, Man, and Cybernetics*, pages 3848–3853. Ieee, 2013.
- [228] Saadia Dhouib, Selma Kchir, Serge Stinckwich, Tewfik Ziadi, and Mikal Ziane. Robotml, a domain-specific language to design, simulate and deploy robotic applications. In *International Conference on Simulation, Modeling, and Programming for Autonomous Robots*, pages 149–160. Springer, 2012.
- [229] Máira Martins Da Silva, Olivier Brûls, Wim Desmet, and Hendrik Van Brussel. Integrated structure and control design for mechatronic systems with configuration-dependent dynamics. *Mechatronics*, 19(6):1016–1025, 2009.
- [230] Lucas Cordeiro, Carlos Mar, Eduardo Valentin, Fabiano Cruz, Daniel Patrick, Raimundo Barreto, and Vicente Lucena. A platform-based software design methodology for embedded control systems: An agile toolkit. In *15th Annual IEEE International Conference and Workshop on the Engineering of Computer Based Systems (ecbs 2008)*, pages 408–417. Ieee, 2008.
- [231] Craig B Chapman and Martyn Pinfold. The application of a knowledge based engineering approach to the rapid design and analysis of an automotive structure. *Advances in engineering software*, 32(12):903–912, 2001.
- [232] Isidro Calvo, Eva Portillo, Oier García de Albéniz, Aintzane Armentia, Marga Marcos, Elisabet Estévez, Ricardo Marau, Luis Almeida, and Paulo Pedreiras. Towards an infrastructure model for composing and reconfiguring cyber-physical systems. In *International Conference on Ubiquitous Computing and Ambient Intelligence*, pages 282–289. Springer, 2012.
- [233] Yuriy Brun, Ron Desmarais, Kurt Geihs, Marin Litoiu, Antonia Lopes, Mary Shaw, and Michael Smit. A design space for self-adaptive systems. In *Software Engineering for Self-Adaptive Systems II*, pages 33–50. Springer, 2013.
- [234] Davide Brugali and Azamat Shakhimardanov. Component-based robotic engineering (part ii). *IEEE Robotics & Automation Magazine*, 17(1):100–112, 2010.
- [235] Marcello Bonfe, Cesare Fantuzzi, and Cristian Secchi. Design patterns for model-based automation software design and implementation. *Control Engineering Practice*, 21(11): 1608–1619, 2013.
- [236] D Biermann, J Gausemeier, H-P Heim, S Hess, M Petersen, A Ries, and T Wagner. A framework for the computer-aided planning and optimisation of manufacturing pro-

- cesses for components with functional graded properties. In *AIP Conference Proceedings*, volume 1593-1, pages 762–765. Aip, 2014.
- [237] Malcolm Bain. Sentilo-sensor and actuator platform for smart cities. *Retr. Febr*, 20:2015, 2014.
- [238] Noriaki Ando, Takashi Suehiro, and Tetsuo Kotoku. A software platform for component based rt-system development: Openrtm-aist. In *International Conference on Simulation, Modeling, and Programming for Autonomous Robots*, pages 87–98. Springer, 2008.
- [239] H. A. Müller. Software engineering for the industrial internet: Situation-aware smart applications. In *2013 15th IEEE International Symposium on Web Systems Evolution (WSE)*, pages 1–1, Sep. 2013. doi: 10.1109/wse.2013.6642408.
- [240] Amanda Whitbrook. *Programming Mobile Robots with Aria and Player: A Guide to C++ Object-oriented Control*. Springer Science & Business Media, 2009.
- [241] Paul Soulier, Depeng Li, and John R Williams. A survey of language-based approaches to cyber-physical and embedded system development. *Tsinghua Science and Technology*, 20(2):130–141, 2015.
- [242] Reid Simmons and David Apfelbaum. A task description language for robot control. In *Intelligent Robots and Systems, 1998. Proceedings., 1998 IEEE/RSJ International Conference on*, volume 3, pages 1931–1937. Ieee, 1998.
- [243] Stepan Shevtsov, Danny Weyns, and Martina Maggio. Self-adaptation of software using automatically generated control-theoretical solutions. In *Engineering Adaptive Software Systems*, pages 35–55. Springer, 2019.
- [244] Kay Römer. Threads2events: An automatic code generation approach. In *Proceedings of the 6th Workshop on Hot Topics in Embedded Networked Sensors*, 2010.
- [245] Jan Oliver Ringert, Alexander Roth, Bernhard Rumpe, and Andreas Wortmann. Code generator composition for model-driven engineering of robotics component & connector systems. *arXiv preprint arXiv:1505.00904*, 2015.
- [246] Laurent Réveillère. *Building Efficient Distributed Systems: A Domain-Specific Language Based Approach*. PhD thesis, Université Sciences et Technologies-Bordeaux I, 2011.
- [247] Muthu Ramachandran. Secure software development of cyber-physical and iot systems. In *Encyclopedia of Information Science and Technology, Fourth Edition*, pages 7525–7538. IGI Global, 2018.

- [248] Tim Niemueller, Alexander Ferrein, Daniel Beck, and Gerhard Lakemeyer. Design principles of the component-based robot software framework fawkes. In *International Conference on Simulation, Modeling, and Programming for Autonomous Robots*, pages 300–311. Springer, 2010.
- [249] Douglas C MacKenzie and Ronald C Arkin. Formal specification for behavior-based mobile robots. In *Mobile Robots VIII*, volume 2058, pages 94–105. International Society for Optics and Photonics, 1994.
- [250] Svein Hallsteinsen, Kurt Geihs, Nearchos Paspallis, Frank Eliassen, Geir Horn, Jorge Lorenzo, Alessandro Mamelli, and George Angelos Papadopoulos. A development framework and methodology for self-adapting applications in ubiquitous computing environments. *Journal of Systems and Software*, 85(12):2840–2859, 2012.
- [251] Alessandro Farinelli, Giorgio Grisetti, and Luca Iocchi. Design and implementation of modular software for programming mobile robots. *International Journal of Advanced Robotic Systems*, 3(1):7, 2006.
- [252] Elisabet Estévez, Alejandro Sánchez-García, Javier Gámez-García, Juan Gómez-Ortega, and Silvia Satorres-Martínez. A novel model-driven approach to support development cycle of robotic systems. *The International Journal of Advanced Manufacturing Technology*, 82(1-4):737–751, 2016.
- [253] Sandeep Chopra, Harish C Sharma, Pradeep Semwal, and Sanjay Sharma. Software model for quality controlled component based software system. *International Journal of Advanced Research in Computer Science and Software Engineering*, 4(8), 2014.
- [254] Rodney Brooks. A robust layered control system for a mobile robot. *IEEE journal on robotics and automation*, 2(1):14–23, 1986.
- [255] Tucker Balch. The teambots environment for multi-robot systems development. *Working notes of Tutorial on Mobile Robot Programming Paradigms, ICRA*, 2002.
- [256] Richard B Kieburtz, Laura McKinney, Jeffrey M Bell, James Hook, Alex Kotov, Jeffrey Lewis, Dino P Oliva, Tim Sheard, Ira Smith, and Lisa Walton. A software engineering experiment in software component generation. In *Proceedings of IEEE 18th International Conference on Software Engineering*, pages 542–552. IEEE, 1996.
- [257] Jonathan Aldrich, David Garlan, Christian Kaestner, Claire Le Goues, Anahita Mohseni-Kabir, Ivan Ruchkin, Selva Samuel, Bradley Schmerl, Christopher Steven Timperley, Manuela Veloso, et al. Model-based adaptation for robotics software. *IEEE Software*, 36(2):83–90, 2019.

- [258] Lichen Zhang, Jifeng He, and Wensheng Yu. Test case generation from formal models of cyber physical system. *International Journal of Hybrid Information Technology*, 6(3):15–24, 2013.
- [259] Johannes Wienke, Dennis Wigand, Norman Koster, and Sebastian Wrede. Model-based performance testing for robotics software components. In *2018 Second IEEE International Conference on Robotic Computing (IRC)*, pages 25–32. Ieee, 2018.
- [260] Jiafu Wan, Hui Suo, Hehua Yan, and Jianqi Liu. A general test platform for cyber-physical systems: unmanned vehicle with wireless sensor network navigation. *Procedia Engineering*, 24:123–127, 2011.
- [261] Valeriy Vyatkin and Hans-Michael Hanisch. Verification of distributed control systems in intelligent manufacturing. *Journal of Intelligent Manufacturing*, 14:123–136, 2003.
- [262] Lenardo C Silva, Mirko Perkusich, Frederico M Bublitz, Hyggo O Almeida, and Angelo Perkusich. A model-based architecture for testing medical cyber-physical systems. In *Proceedings of the 29th Annual ACM Symposium on Applied Computing*, pages 25–30. Acm, 2014.
- [263] Viet Hoa Nguyen, François Fouquet, Noël Plouzeau, and Olivier Barais. A process for continuous validation of self-adapting component based systems. In *Proceedings of the 7th Workshop on Models@ run. time*, pages 32–37. Acm, 2012.
- [264] Oscar Ljungkrantz, Knut Akesson, Martin Fabian, and Chengyin Yuan. Formal specification and verification of industrial control logic components. *IEEE Transactions on Automation Science and Engineering*, 7(3):538–548, 2010.
- [265] Jae-Hee Lim, Suk-Hoon Song, Jung-Rye Son, Tae-Yong Kuc, Hong-Seong Park, and Hong-Seok Kim. An automated test method for robot platform and its components. *International Journal of Software Engineering and Its Applications*, 4(3):9–18, 2010.
- [266] Tariq M King, Alain E Ramirez, Rodolfo Cruz, and Peter J Clarke. An integrated self-testing framework for autonomic computing systems. *Jcp*, 2(9):37–49, 2007.
- [267] Aaron Kane, Thomas Fuhrman, and Philip Koopman. Monitor based oracles for cyber-physical system testing: Practical experience report. In *2014 44th annual ieee/ifip international conference on dependable systems and networks*, pages 148–155. Ieee, 2014.
- [268] Huang-Ming Huang, Terry Tidwell, Christopher Gill, Chenyang Lu, Xiuyu Gao, and Shirley Dyke. Cyber-physical systems for real-time hybrid structural testing: a case study. In *Proceedings of the 1st ACM/IEEE international conference on cyber-physical systems*, pages 69–78. Acm, 2010.

- [269] Jie He, Yishuang Geng, Yadong Wan, Shen Li, and Kaveh Pahlavan. A cyber physical test-bed for virtualization of rf access environment for body sensor network. *IEEE Sensors Journal*, 13(10):3826–3836, 2013.
- [270] Thomas Hartmann, François Fouquet, Jacques Klein, Grégory Nain, and Yves Le Traon. Reactive security for smart grids using models@ run. time-based simulation and reasoning. In *International Workshop on Smart Grid Security*, pages 139–153. Springer, 2014.
- [271] Reinhard Hametner, Dietmar Winkler, Thomas Östreicher, Stefan Biffl, and Alois Zoitl. The adaptation of test-driven software processes to industrial automation engineering. In *2010 8th IEEE International Conference on Industrial Informatics*, pages 921–927. Ieee, 2010.
- [272] Antonio Filieri and Giordano Tamburrelli. Probabilistic verification at runtime for self-adaptive systems. In *Assurances for Self-Adaptive Systems*, pages 30–59. Springer, 2013.
- [273] Bahareh Badban, Martin Fränzle, Jan Peleska, and Tino Teige. Test automation for hybrid systems. In *Proceedings of the 3rd international workshop on Software quality assurance*, pages 14–21. Acm, 2006.
- [274] Jeffrey Yan and Valeriy Vyatkin. Distributed software architecture enabling peer-to-peer communicating controllers. *IEEE Transactions on Industrial Informatics*, 9(4):2200–2209, 2013.
- [275] Miguel Jiménez, Norha M. Villegas, Gabriel Tamura, and Hausi A. Müller. Deployment specification challenges in the context of large scale systems. In *Proceedings of the 27th Annual International Conference on Computer Science and Software Engineering, Cascon '17*, pages 220–226, Riverton, NJ, USA, 2017. IBM Corp. URL <http://dl.acm.org/citation.cfm?id=3172795.3172821>.
- [276] Muhammed Cagri Kaya, Alperen Eroglu, Alper Karamanlioglu, Ertan Onur, Bedir Tekinerdogan, and Ali H Dogru. Runtime adaptability of ambient intelligence systems based on component-oriented approach. In *Guide to Ambient Intelligence in the IoT Environment*, pages 69–92. Springer, 2019.
- [277] Nico Hochgeschwender, Luca Gherardi, Azamat Shakhirmardanov, Gerhard K Kraetzschmar, Davide Brugali, and Herman Bruyninckx. A model-based approach to software deployment in robotics. In *Intelligent Robots and Systems (IROS), 2013 IEEE/RSJ International Conference on*, pages 3907–3914. Ieee, 2013.
- [278] Oliver Niggemann and Björn Kroll. On the applicability of model based software development to cyber physical production systems. In *Proceedings of the 2014 IEEE Emerging Technology and Factory Automation (ETFA)*, pages 1–4. Ieee, 2014.

- [279] Mohammad Abdullah Al Faruque and Fereidoun Ahourai. A model-based design of cyber-physical energy systems. In *2014 19th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 97–104. Ieee, 2014.
- [280] Quanyan Zhu and Tamer Basar. Game-theoretic methods for robustness, security, and resilience of cyberphysical control systems: games-in-games principle for optimal cross-layer resilient control systems. *IEEE Control Systems Magazine*, 35(1):46–65, 2015.
- [281] Terry Tidwell, Xiuyu Gao, Huang-Ming Huang, Chenyang Lu, Shirley Dyke, and Christopher Gill. Towards configurable real-time hybrid structural testing: a cyber-physical system approach. In *2009 IEEE International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing*, pages 37–44. Ieee, 2009.
- [282] Nathan Koenig and Andrew Howard. Design and use paradigms for gazebo, an open-source multi-robot simulator. In *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)(IEEE Cat. No. 04CH37566)*, volume 3, pages 2149–2154. Ieee, 2004.